BEGINNER'S GUIDE:

HOW MACHINES SENSE, CONNECT, AND ACT

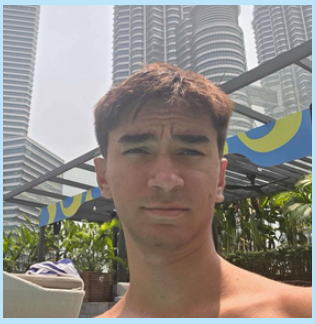
BLUE IMPULSE

MEET THE TEAM

ABOUT US:

We are BlueImpulse, the masterminds behind this course. We consist of 5 members from different backgrounds and ethnicities, working together to create a learning experience that is memorable and informative for students.

MEMBERS:



Danial

Greetings, my name is Danial Khan. I'm from Russia and I did research on learning methods and behaviorist AI.

Nazif

I am Nazif. I am from Bangladesh. I am doing visual works for the team. Driven by a relentless curiosity, I have explored and built across multiple domains. I basically worked on polishing and reconstructing everything, including the main repository.

Safa

Hi! My name is Safa Nadeem, and I'm from Japan and Pakistan. I was the submission lead for this project, and one of the members on the research team. I made sure all the documents were submitted on time, coordinated fellow members' research, and contributed to forming the structure and foundation for this course.

Tian

Hi, my name is Tian. I'm from Thailand. I am the visual design leader. What I do is that I design the color palette, the layout, and many more things that has to do with art.

Edzel

Hello my name is Edzel Lanvin Hennessy or you could call me by 黄金贤, I'm Indonesian, and my role in this group project is Research / Content Team.

CONTACT:

Email:

- danialkhanmay@gmail.com
- abdullahnajif5@gmail.com
- safa-nadeem@outlook.com
- pichayakan@gmail.com
- edzellahbodo@gmail.com
- pichayakan@gmail.com

Overview of the module

Our collection of learning materials consists of three main parts: this study guide PDF, a selection of video tutorials, and a PowerPoint.

First of all, the Study Guide is the main learning source. For each lesson, the learner or the person in charge of teaching this module should read through a chapter, stopping momentarily at the subtopics. They should pause and attempt the questions in “Pause and Try” mentioned at the bottom of every page. These are designed to develop and test the learning in concrete steps, so any misunderstandings can be addressed and fixed then and there, before moving to the next topic. This ensures a well-balanced, helix-style approach to learning.

Next, the video tutorials are in-depth explanations on selected topics touched on briefly by the study guide. They provide a deeper, out-of-syllabus look into the dynamics of certain ideas and questions that the reader could have.

Overview of the module

Finally, there is a PowerPoint attached, which bridges together the main three topics—hardware, networking and behaviorist AI—in the example of a real-life rover that went to Mars, Sojourner. Here, the reader explores how Sojourner implemented these three ideas in a mission to collect information on the Martian environment. This approach of taking the topics beyond standard, textbook material to active, real-life examples ensures that the readers would be able to relate better.

Therefore, we aim that the usage of these three components of the learning materials in a cohesive manner engraves the core concepts and cements the understanding of the reader, thereby ensuring a fun, comprehensive learning experience.

2/2

Safa, Danial, Tian, Nazif and Edzel

About this guide

This study guide introduces **high-school students and university freshmen** who already have some basic computer science knowledge to our module: **Hardware, Networks and Behaviorist AI**. It explains how machines *sense* the world, *connect* to one another and *act* on what they learn. These three ideas give the guide its title.

Three parts, used together

The module is built from three learning materials. You can move between them in any order:

1. **This study guide** — six chapters of explanations, worked examples, diagrams and exercises.
2. **Video tutorials** — short narrated lessons that show a process in motion (program execution, reinforcement learning, hardware for AI and networking). Each chapter shows the link at the topic it belongs to.
3. **RoverLab, an interactive game** — build a small machine, connect its network, train its behaviour and test it on a mission. The Appendix explains how to play.

How each chapter works

Every chapter opens with a coloured **divider**, followed by its **learning goals** and **key terms**. The subtopics keep their numbers (1.1, 1.2 …) so you can find them in the Contents. Most pages end with a short **Pause & Try** question: answer it before you turn the page. Each chapter closes with a **Practice** page that brings the whole chapter together.

Reading the page

Window boxes explanations → **Folder boxes** pictures and diagrams → **Pill shapes** steps and flows

Text on the background introduces an idea; the boxes break it into parts.

Play RoverLab: Build, Connect, Train
https://roverlab-blue-impulse.vercel.app

All video tutorials
https://github.com/user-github-me/cs-ai-project-01/tree/main/assets/tutorials/stage2

Contents

1 The Fundamentals 9

- 1.1 Number Systems & Data Representation 13
- 1.2 Multimedia Representation & Compression 19
- 1.3 Computer Architecture & The Von Neumann Model 25
- 1.4 Operating Systems & Software Architecture 30
- 1.5 Language Translators & Code Execution 34

2 Behaviorism & Behaviorist AI 42

- 2.1 Foundations of Behaviorism 46
- 2.2 Behaviorist AI & Behavior-Based Robotics 51
- 2.3 Introduction to Reinforcement Learning (RL) 56
- 2.4 Markov Decision Processes (MDPs) Made Simple 64

3 Hardware 72

- 3.1 Input, Processing and Output Hardware 76
- 3.2 Computer Memory and Storage 83
- 3.3 Processing Devices and CPU Architecture 89
- 3.4 Parallel Processing and Computer Systems 92
- 3.5 Logic Circuits and Boolean Algebra 98
- 3.6 How Hardware Enables AI 103

4 Networking 107

- 4.1 Network Architecture & Topologies 113
- 4.2 Client Computations & Cloud Technologies 119
- 4.3 Hardware Required to Support a LAN 121
- 4.4 Transmission Media & Collision Handling 124
- 4.5 Internet Infrastructure, URLs & DNS 128
- 4.6 Protocols & The TCP/IP Protocol Stack 132
- 4.7 Network Security, Privacy & Data Integrity 137
- 4.8 Switching Mechanisms: Circuit vs. Packet Switching 142
- 4.9 Cryptography, Quantum Security & Digital Certificates 145

Contents

5 Applications 154

- 5.1 Applications of Hardware, Networks and Behaviorist AI in Real-Life Situations 157
- 5.2 Smart Homes 161
- 5.3 Healthcare 163
- 5.4 Transportation 165
- 5.5 Industry & Manufacturing 168
- 5.6 Education 170
- 5.7 Conclusion 172

6 Strengths & Limitations 177

- 6.1 Introduction to Integrated Smart Systems 181
- 6.2 Embedded Hardware: Capabilities & Limitations 183
- 6.3 Deep Dive: Sensor Integration and Hardware Reliability 186
- 6.4 Network Architectures: Connectivity & Infrastructure 189
- 6.5 Resource Optimization: Edge vs. Cloud Trade-Offs 191
- 6.6 Behaviorist AI & Adaptive Agents 193
- 6.7 Predictive Analytics vs. Behavioral Modeling 195
- 6.8 Key Limitations of Behaviorist AI Models 197
- 6.9 Ethical, Security, and Practical Concerns 200

A Appendix 205

- Appendix · RoverLab: Build, Connect, Train 205

DAWN

THE FUNDAMENTALS

1

CHAPTER 1 · BEFORE YOU BEGIN

Learning goals

This chapter explains how a computer represents numbers, text, pictures and sound, how its processor runs a program, and how software — from the operating system to the translators that turn code into machine code — makes the hardware useful. Everything in the later chapters builds on these ideas: sensor readings, network messages and AI programs are all stored in binary and executed by a processor like the one you will meet here.

Learning goals

You will learn about:

- Binary magnitudes, binary prefixes and decimal prefixes
- Binary, denary and hexadecimal number systems
- How to carry out binary addition and subtraction
- The use of hexadecimal and binary-coded decimal (BCD) number systems
- Image resolution and colour depth
- Representation of sound in a computer
- Effects of changing sampling rate and resolution on sound quality
- The need for file compression methods (such as lossy and lossless)
- The basic Von Neumann model of a computer system
- Source code vs machine code
- The difference between high-level and low-level languages
- Compilers and interpreters
- Why computers need an operating system
- The fetch–execute cycle
- The purpose and role of the registers PC, MDR, MAR, ACC and CIR
- Key management tasks such as memory management, hardware management, security management and process management
- The need for utility software such as disk formatter, virus checker, defragmentation software, repair software, file compression and back-up software
- The need for these language translators: assemblers, compilers, interpreters
- Benefits and drawbacks of using compilers or interpreters
- Features of a typical IDE
- The purpose of interrupts
- Connection of computers to peripheral devices such as USB, HDMI and VGA
- The relationship between assembly language and machine code
- The different stages of a two-pass assembler
- Tracing sample assembly language code

CHAPTER 1 · BEFORE YOU BEGIN

Key terms

Key terms

Binary: Base-two number system that uses only the values 0 and 1.

Bit: Abbreviation for binary digit.

Sign and magnitude: The left-most bit gives the sign (0 = positive, 1 = negative); the other bits give the value.

One's complement: Every binary digit is reversed so that negative numbers can be represented.

Two's complement: Every bit is reversed and 1 is added in the right-most position to represent a negative number.

Hexadecimal: Base-16 number system using the digits 0–9 and the letters A–F.

Memory dump: Contents of computer memory output to a screen or printer.

Binary-coded decimal (BCD): Uses 4 bits to represent each denary digit.

ASCII code: Coding system for the characters on a keyboard plus control codes.

Character set: The list of characters defined by computer hardware and software.

Unicode: Coding system for all the languages of the world; its first 128 characters match ASCII.

Von Neumann architecture: Computer architecture that introduced the stored-program concept in the 1940s.

Arithmetic logic unit (ALU): Carries out all arithmetic and logical operations in the processor.

Control unit: Synchronises the flow of data and programs by sending control signals along the control bus.

System clock: Produces timing signals on the control bus to keep components in step.

Immediate access store (IAS): Holds the data and programs the control unit needs to access.

Accumulator (ACC): General-purpose register that stores values during an operation.

Register: Small, very fast storage inside the processor that holds data or instructions during the fetch–execute cycle.

Status register: Records the outcome of an arithmetic or logic operation using flags.

Flag: A single bit in the status register, e.g. N = 1 shows that a result is negative.

Address bus: Carries memory addresses; unidirectional (one way only).

Data bus: Carries data between processor, memory and input/output devices; bidirectional (both ways).

Control bus: Carries signals from the control unit to all other components.

Word: A group of bits treated as a single unit by the processor.

Clock cycle: One pulse of the system clock; 3.5 GHz means 3.5 billion cycles every second.

Overclocking: Running the system clock faster than the factory setting.

Cache memory: Small, high-speed memory that keeps frequently used data close to the CPU.

BIOS: Basic input/output system; start-up software that checks the hardware and loads the operating system.

Interrupt: A signal that asks the CPU to stop and deal with an event.

Compiler: Translates a whole high-level program into machine code before it runs.

Interpreter: Translates and runs a high-level program one statement at a time.

Assembler: Translates assembly language into machine code.

CHAPTER 1 · BEFORE YOU BEGIN

Key terms (continued)

Key terms (continued)

Nibble: A group of four bits; one hexadecimal digit.

Byte: A group of eight bits.

Denary: The base-10 number system we use every day.

Binary prefix: A prefix based on powers of 2, e.g. kibi (Ki) = 2^{10} = 1 024.

Decimal prefix: A prefix based on powers of 10, e.g. kilo (k) = 10^3 = 1 000.

Overflow: A result that needs more bits than are available to store it.

Pixel: The smallest element of a bitmap image; it shows a single colour.

Image resolution: The number of pixels in an image, written as width × height.

Colour depth: The number of bits used to store the colour of one pixel.

Bitmap image: An image stored as a grid of pixels.

Vector graphic: An image stored as a drawing list of shapes and their properties.

Sampling rate: The number of sound samples taken per second, in hertz (Hz).

Sampling resolution: The number of bits used to store each sound sample.

ADC / DAC: Analogue-to-digital converter / digital-to-analogue converter.

Lossless compression: Makes a file smaller in a way that lets the original be rebuilt exactly.

Lossy compression: Makes a file smaller by permanently removing data the user is unlikely to notice.

Run-length encoding (RLE): Lossless method that stores a run of identical values as one value and a count.

Program counter (PC): Holds the address of the next instruction to be fetched.

Memory address register (MAR): Holds the address of the memory location being read from or written to.

Memory data register (MDR): Holds the data or instruction just read from memory, or about to be written to it.

Current instruction register (CIR): Holds the instruction being decoded and executed.

Interrupt service routine (ISR): The program code that deals with one type of interrupt.

Peripheral: A device connected to the computer from outside, e.g. a printer or a webcam.

Operating system (OS): Software that manages the hardware and provides a platform and an interface for programs and users.

Device driver: Software that lets the operating system control one particular device.

Virtual memory: Part of secondary storage used as extra memory when RAM is full.

Utility software: Programs that maintain, protect and tidy up a computer.

Defragmentation: Reorganising a hard disk so that the blocks of each file sit next to each other.

Source code: A program as written by the programmer.

Machine code: Binary instructions that a processor can execute directly.

High-level language: A programming language close to human language and independent of the hardware.

Low-level language: A language close to the hardware: machine code or assembly language.

Mnemonic: A short code for an instruction, such as LOAD or ADD.

Opcode / operand: The part of an instruction that says what to do / the value or address it acts on.

Symbol table: A table of labels and their addresses, built by an assembler.

Trace table: A table that records how values change as a program runs, one step at a time.

IDE: Integrated development environment: one application with an editor, a translator, a run-time environment and a debugger.

Breakpoint: A marker that pauses a program at a chosen line so that its values can be checked.

1.1 NUMBER SYSTEMS & DATA REPRESENTATION

Binary system

A computer is built from billions of **transistors**, tiny electronic switches with two stable states — *off* and *on* — written **0** and **1**. A number system that uses just these two digits is called **binary**, or base 2. Everything a computer stores is ultimately a pattern of 0s and 1s.

Bits, nibbles and bytes

A single binary digit is a **bit** (symbol **b**); four bits make a **nibble** and eight bits make a **byte** (symbol **B**). Mind the case: a 100 Mb/s internet connection moves at most 100 ÷ 8 = 12.5 MB per second. Each extra bit doubles the number of possible patterns, so n bits give 2^n patterns.

Bits	Patterns	Example use
1	2	on / off, true / false
4	16	one hexadecimal digit
8	256	one byte, e.g. one ASCII character

KB or KiB? Two ways to count bytes

Storage can be measured with **decimal (SI) prefixes**, based on powers of 10, or **binary (IEC) prefixes**, based on powers of 2. Because "kilobyte" was used for both 1 000 and 1 024 bytes, the binary prefixes kibi, mebi, gibi and tebi were introduced in 1998.

Decimal prefix	Value	Binary prefix	Value
kilobyte (kB)	10^3 = 1 000	kibibyte (KiB)	2^{10} = 1 024
megabyte (MB)	10^6	mebibyte (MiB)	2^{20} = 1 048 576
gigabyte (GB)	10^9	gibibyte (GiB)	2^{30}
terabyte (TB)	10^{12}	tebibyte (TiB)	2^{40}

Drive makers use decimal prefixes, but memory comes in powers of two. So a 500 GB drive shows in an operating system as about **465.7 GiB**: 500 000 000 000 ÷ 2^{30} ≈ 465.7.

Pause & Try

a) How many bytes are in 3 MiB? b) A USB stick is sold as 16 GB. Explain why an operating system reports about 14.9 GiB.

1.1 NUMBER SYSTEMS & DATA REPRESENTATION

Binary, Denary and Hexadecimal systems

A number system writes quantities with a fixed set of digits. Its **base** tells you how many digits it has and how much each column is worth. Denary (base 10) is the system we use every day; computers work in binary (base 2); programmers often read binary through **hexadecimal** (base 16).

All three use **place value**: each column is worth *base* times the column on its right. In denary, 245 means 2 hundreds, 4 tens and 5 ones; binary uses powers of 2 and hexadecimal powers of 16.

Same number, three systems

System	Digits	Column values (right to left)	245 written in it
Denary (base 10)	0–9	1, 10, 100, 1000 …	245 = 2×100 + 4×10 + 5
Binary (base 2)	0, 1	1, 2, 4, 8, 16, 32, 64, 128 …	1111 0101 = 128+64+32+16+4+1
Hexadecimal (base 16)	0–9, A–F	1, 16, 256, 4096 …	F5 = 15×16 + 5

Binary → denary

Write the column values above the bits, then add up the values wherever there is a 1.

128	64	32	16	8	4	2	1
0	0	1	0	1	1	0	1

0010 1101 = 32 + 8 + 4 + 1 = **45**

Denary → binary

Subtract the biggest column value that fits, write 1 there, and repeat with what is left. For **156**:

- 156 − 128 = 28 → 1 in the 128 column
- 28 − 16 = 12 → 1 in the 16 column
- 12 − 8 = 4 → 1 in the 8 column
- 4 − 4 = 0 → 1 in the 4 column

Every other column gets 0, so 156 = **1001 1100**.

Another method is to divide by 2 again and again and read the remainders from the bottom up. Always check your answer by converting it back.

Pause & Try

Convert 200 and 99 into 8-bit binary, then check each answer by converting it back into denary.

1.1 NUMBER SYSTEMS & DATA REPRESENTATION

Working with hexadecimal

Hexadecimal (hex) is base 16: after 0–9 it uses **A to F** for ten to fifteen, and its columns are 1, 16, 256 … As $16 = 2^4$, **one hex digit always stands for exactly four bits** (a nibble).

The sixteen hex digits

0 **0000** denary 0	1 **0001** denary 1	2 **0010** denary 2	3 **0011** denary 3	4 **0100** denary 4	5 **0101** denary 5	6 **0110** denary 6	7 **0111** denary 7
8 **1000** denary 8	9 **1001** denary 9	A **1010** denary 10	B **1011** denary 11	C **1100** denary 12	D **1101** denary 13	E **1110** denary 14	F **1111** denary 15

Converting with hex

- **Binary → hex:** split the bits into nibbles from the right; convert each one: 0010 1101 → **2D**.
- **Hex → binary:** replace each digit with its four bits: 3F → 0011 1111.
- **Hex → denary:** multiply each digit by its column value: 2D = 2 × 16 + 13 × 1 = **45**.
- **Denary → hex:** divide by 16: 172 ÷ 16 = 10 r 12. Ten is A and twelve is C, so 172 → **AC**.

A memory dump: each pair of hexadecimal digits is one byte.

Why engineers use hexadecimal

- It is **shorter**: one hex digit replaces four bits.
- It converts to and from binary **directly**, nibble by nibble.
- It causes **fewer copying errors** than long strings of 0s and 1s.
- You meet it in memory dumps, colour codes (#FF0000 is pure red) and MAC addresses.

Pause & Try

Convert the byte 1011 0110 into denary and into hexadecimal, showing your column values.

1.1 NUMBER SYSTEMS & DATA REPRESENTATION

Representing negative values

A byte on its own cannot hold a minus sign — it only holds bits. To store negative numbers, a computer agrees on a rule for what the left-most bit means. There are three common rules; we will follow the number **−5** through each of them in 8 bits.

Sign and magnitude

The left-most bit is the **sign** (0 = +, 1 = −); the other 7 bits hold the size.

+5 = 0000 0101 and −5 = **1**000 0101.

It is easy to read, but there are two zeros (0000 0000 and 1000 0000) and ordinary addition gives wrong answers.

One's complement

Write +5, then **flip every bit**: each 0 becomes 1 and each 1 becomes 0.

+5 = 0000 0101 → −5 = 1111 1010.

Better for arithmetic, but it still has two zeros: 0000 0000 and 1111 1111.

Two's complement, step by step

Step 1 — write the positive number
+5 = 0000 0101

↓

Step 2 — flip every bit
1111 1010

↓

Step 3 — add 1
1111 1011 = −5

Check it: in two's complement the left-most column is worth **−128** instead of +128. So 1111 1011 = −128 + 64 + 32 + 16 + 8 + 2 + 1 = −5. An 8-bit two's complement number ranges from **−128** (1000 0000) to **+127** (0111 1111), and there is only one zero.

Reading a negative number: a 1 in the left-most bit tells you the number is negative. Start from −128 and add the other column values: 1110 0110 = −128 + 64 + 32 + 4 + 2 = **−26**.

Positive numbers look the same in all three systems. Almost every processor uses two's complement: it has only one zero, and it lets the CPU subtract by **adding** a negative number (next page).

Pause & Try

Write −53 as an 8-bit two's complement number. Show all three steps, then check your answer using the −128 column.

1.1 NUMBER SYSTEMS & DATA REPRESENTATION

Binary addition and subtraction

Computers add binary numbers column by column, from right to left — exactly as you add denary numbers on paper. The only difference is when you carry: in denary you carry when a column reaches 10; in binary you carry as soon as a column reaches **2**.

The four rules

Column	Total	Write	Carry
0 + 0	0	0	0
0 + 1	1	1	0
1 + 1	$2 = 10_2$	0	1
1 + 1 + 1	$3 = 11_2$	1	1

The last rule is needed when a carry arrives in a column that already holds two 1s.

Worked example: 53 + 27

carry	1	1	1	1	1	1		*denary*
	0	0	1	1	0	1	0	1	53
+	0	0	0	1	1	0	1	1	27
	0	1	0	1	0	0	0	0	80

Six columns produce a carry. Check in denary: 64 + 16 = 80.

Overflow

An 8-bit register can hold values only up to 1111 1111 = 255. If a result needs a ninth bit, that bit is lost — this is called **overflow**. For example, 180 + 100 = 280, but 1011 0100 + 0110 0100 = 1 0001 1000, and the eight bits that remain, 0001 1000, mean 24. The CPU sets a flag in the status register so that a program can detect the error.

Subtracting: 35 – 18

A CPU subtracts by adding a negative number: 35 − 18 = 35 + (−18).

1. Write 18: 0001 0010
2. Make it negative with two's complement (flip, add 1): 1110 1110
3. Add it to 35: 0010 0011 + 1110 1110 = 1 0001 0001
4. Ignore the carry out of the left-most bit: 0001 0001 = **17**

Pause & Try

a) Add 0100 1010 and 0010 0111, showing your carries, and check your answer in denary. b) Use two's complement to calculate 45 − 27 in 8 bits.

1.1 NUMBER SYSTEMS & DATA REPRESENTATION

BCD and ASCII

Pure binary is not the only way to store information as bits. Two other codes are used every day: **BCD** for decimal digits and **ASCII** for text.

Binary-coded decimal system (BCD)

In BCD, each denary digit is stored **separately** as its own 4-bit binary number. The number is not converted as a whole: 259 becomes 0010 0101 1001 — one nibble each for the 2, the 5 and the 9. (In pure binary, 259 is 1 0000 0011, a different pattern.) A nibble has 16 patterns but there are only ten digits, so 1010 to 1111 are never used. To read BCD, convert each nibble: 0100 0000 0111 → 4, 0, 7 → 407.

Where BCD is used

- **Digital clocks, calculators and meters:** each nibble drives one digit of the display, so no conversion is needed.
- **Money:** a value such as 0.1 cannot be stored exactly in pure binary, so banking systems that store each decimal digit avoid small rounding errors.

ASCII system

To store text, every character in a **character set** is given a number code. **ASCII** (American Standard Code for Information Interchange) uses 7 bits, giving 2^7 = 128 codes: English letters in upper and lower case, the digits 0–9, punctuation and control codes such as *new line*. The codes are in order, so if 'A' is 65 then 'D' is 68. **Extended ASCII** uses 8 bits for 256 characters, and **Unicode** has codes for more than 150 000 characters from all the world's writing systems — its first 128 codes are the same as ASCII.

Some ASCII codes

Character	Denary	Binary	Hex
A	65	0100 0001	41
a	97	0110 0001	61
7	55	0011 0111	37

What do the bits mean? Read 0100 0001 as a binary number and it is **65**; read it as ASCII and it is the letter **A**; read it as BCD and it is **41**. The bits never change — only the rule used to read them. That is why files have types (.txt, .jpg, .mp3): they tell the computer which rule to use.

Pause & Try

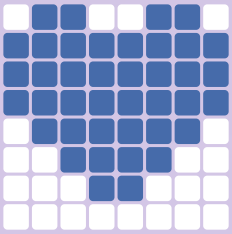
a) Write 4 052 in BCD. b) Using 'A' = 65, write the ASCII code for 'G' in denary and in 8-bit binary.

1.2 MULTIMEDIA REPRESENTATION & COMPRESSION

Digital images

Zoom into any photo on a screen and you will see tiny squares called **pixels** (*picture elements*). Each pixel has one colour, stored as a binary number; a grid of pixels like this is a **bitmap image**.

A tiny bitmap, pixel by pixel



```
0110 0110
1111 1111
1111 1111
1111 1111
0111 1110
0011 1100
0001 1000
0000 0000
```

row 1

row 8

1 = blue **0 = white**

8 × 8 = 64 pixels
1 bit per pixel
64 bits = 8 bytes

Each row of pixels is stored as one byte.

Image resolution

Image resolution is the number of pixels in an image, written as width × height. A Full HD photo is 1920 × 1080 = 2 073 600 pixels (about 2 megapixels). More pixels capture finer detail, but every extra pixel adds to the file size.

Colour depth

Colour depth is the number of bits per pixel: *n* bits give 2^n colours.

Bits per pixel	Colours
1	2 (black and white)
4	16
8	256
24	16 777 216 (“true colour”)

Worked example: how big is a bitmap?

File size (bits) = width × height × colour depth. A 1920 × 1080 photo in true colour (8 bits each for red, green and blue) needs 1920 × 1080 × 24 = 49 766 400 bits ÷ 8 = 6 220 800 bytes ÷ 1 048 576 ≈ **5.93 MiB**. Real files are slightly bigger: they also store a *header* with the width, height and colour depth.

Pause & Try

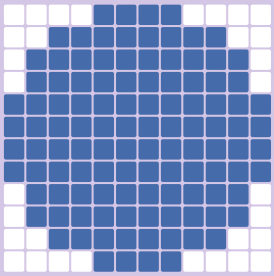
An image is 800 × 600 pixels with a colour depth of 8 bits. a) How many different colours can each pixel be? b) Calculate the file size in kibibytes.

1.2 MULTIMEDIA REPRESENTATION & COMPRESSION

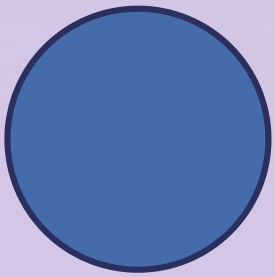
Bitmap and vector graphics

There are two completely different ways to store a picture. A **bitmap** stores the colour of every single pixel. A **vector graphic** stores instructions for drawing the picture: a list of shapes and their properties. The difference shows most clearly when you enlarge an image.

Zoom in on a circle



Bitmap, enlarged:
jagged, blocky edges



Vector, enlarged:
smooth at any size

Drawing list (what is stored)

```
circle
  centre: (60, 60)
  radius: 60
  fill colour: blue
  line: navy, 3 px
```

The computer redraws the shape from this list whenever it is shown.

Bitmap images

- Stored as a grid of pixels. The file size depends on resolution × colour depth, not on what the picture shows.
- Enlarging makes every pixel bigger, so edges look jagged — the image becomes **pixelated**.
- Ideal for photographs and scanned images, where neighbouring pixels can all be different.
- File types: BMP, PNG, JPEG, GIF.

Vector graphics

- Stored as a **drawing list**: each object (line, circle, rectangle, curve, text) with properties such as position, size, line colour and fill colour.
- The computer redraws the shapes at any size, so they stay sharp.
- The file size depends on how many objects there are.
- Ideal for logos, icons, diagrams, maps and fonts — but not for photos.
- File types: SVG, EPS.

Pause & Try

A company logo and a photo of the company's staff must both be printed on a 6-metre banner. For each one, explain whether it should be stored as a bitmap or as a vector graphic.

1.2 MULTIMEDIA REPRESENTATION & COMPRESSION

Multimedia

Multimedia means combining different types of media — text, images, sound, animation and video — in one product, such as a game, a website or an online lesson. You have seen how text and images become binary; a video is simply a sequence of images (often 24 to 60 every second) with a soundtrack. Sound needs a different approach, because it is not made of separate pieces.

Sound

Sound is a **vibration** that travels through the air as a wave of pressure changes. The wave's **amplitude** (height) is its loudness, and its **frequency** — waves per second, in hertz (Hz) — is its pitch; people hear from about 20 Hz to 20 000 Hz. A sound wave is **analogue**: it changes smoothly and continuously, with endless possible values. A computer can store only a limited list of binary numbers, so the wave must be converted.

Recording: from air to binary

Sound wave air vibrates → **Microphone** makes a voltage → **ADC** takes samples → **Binary data** saved in a file

1. The **microphone** turns the pressure changes into a matching electrical signal — still analogue.
2. The **analogue-to-digital converter (ADC)** measures the height of the signal at regular, very short intervals. Each measurement is a **sample**.
3. Each sample is rounded to the nearest level that the available bits can store, and saved as a binary number.

Playback: from binary back to air

Binary data read from the file → **DAC** rebuilds a voltage → **Speaker** vibrates the air

A **digital-to-analogue converter (DAC)** turns the numbers back into a changing voltage, which an amplifier and speaker turn into sound. How often we sample, and how many bits each sample gets, decide the quality — see the next page.

Pause & Try

Explain why sound has to be converted before a computer can store it, and name the device that converts it in each direction.

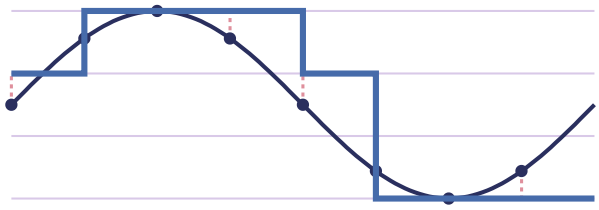
1.2 MULTIMEDIA REPRESENTATION & COMPRESSION

Sampling rate and resolution

Two settings decide how closely a recording matches the real sound: how **often** the ADC takes a sample, and how **precisely** each sample is stored. Below, the navy curve is the real wave, the dots are samples and the blue steps are what the computer stores.

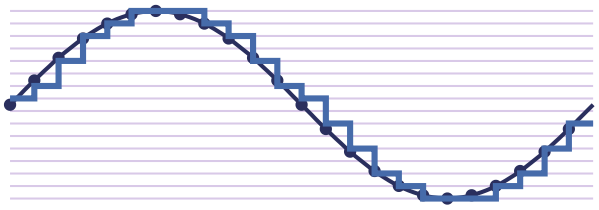
Sampling the same wave twice

Low sampling rate and resolution
2 bits per sample



8 samples per wave · 4 levels

Higher sampling rate and resolution
4 bits per sample



24 samples per wave · 16 levels

Sampling rate

The number of samples taken **per second**, in hertz; music CDs use 44 100 Hz. A higher rate captures quicker changes in the wave, such as high-pitched sounds, so the recording is more accurate — but there are more samples to store, so the file is bigger.

Sampling resolution

The number of **bits per sample** (bit depth). *n* bits give 2^n levels: 8 bits give 256 and 16 bits (CD quality) 65 536. More levels mean each sample is rounded less, so the sound is more accurate with less hiss — but the file is bigger.

Worked example: how big is a recording?

File size (bits) = sampling rate × sampling resolution × channels × seconds. One minute of CD-quality stereo: 44 100 × 16 × 2 × 60 = 84 672 000 bits = 10 584 000 bytes ≈ **10.1 MiB**. A phone call (8 000 Hz, 8 bits, mono) needs only 480 000 bytes a minute — but voices sound muffled.

Pause & Try

A 20-second mono recording uses a sampling rate of 8 000 Hz and a sampling resolution of 16 bits. a) Calculate the file size in kilobytes. b) State one advantage and one disadvantage of increasing the sampling rate.

1.2 MULTIMEDIA REPRESENTATION & COMPRESSION

Compression methods

Media files are big. One minute of CD-quality sound takes about 10 MB, and one uncompressed 4K video frame (3840 × 2160 pixels in true colour) takes about 25 MB — at 30 frames a second, that is 750 MB for every second of video. **Compression** makes files smaller, using one of two families of methods — **lossless** or **lossy** — so that they:

- take up less storage space;
- download and stream faster, using less bandwidth and mobile data;
- fit within limits, such as the size allowed for an email attachment.

Lossless or lossy?

	Lossless	Lossy
What happens	Repeated patterns are stored more efficiently; no data is lost	Data that people are unlikely to notice is removed for ever
Can the original be rebuilt?	Yes, exactly	No, only a close copy
How much smaller?	Moderately	Much: often a tenth of the size
Examples	ZIP, PNG, GIF, FLAC	JPEG, MP3, AAC, MP4
Best for	Text, programs, medical scans	Photos, music, streamed video

Two copies of one photo: one has been through lossy compression. At this size, can you tell which?

Choosing a method

Ask yourself: **does every bit matter?** If a single changed bit would break the file — a bank record, a legal contract, an app — it must be compressed losslessly. If the file is only for human eyes and ears, lossy compression gives far smaller files with very little visible or audible difference.

Pause & Try

Choose lossy or lossless compression for each file and justify your choice: a) a program file; b) a holiday photo for social media; c) a spreadsheet of exam results.

1.2 MULTIMEDIA REPRESENTATION & COMPRESSION

Lossless and lossy techniques

Lossless methods look for **repetition** and store it more efficiently. Lossy methods remove the parts of an image or a sound that human senses notice **least**.

Lossless 1 · Run-length encoding (RLE)

RLE replaces a **run** of identical values with one copy of the value and a count. This row of 16 pixels becomes just 3 pairs (6 values):

5 white **6 blue** **5 white**

Text works the same way: AAAAAABBBCCCCC becomes 6A 3B 5C. RLE suits icons and cartoons with large areas of one colour, but not photos: ABCD would become 1A 1B 1C 1D — bigger than before!

Lossless 2 · Dictionary methods

A repeated sequence is stored once; each later copy becomes a short reference to it. Ideal when a word such as *computer* appears again and again. Used in ZIP, PNG and GIF.

Lossless 3 · Huffman coding

Common characters get **short** binary codes and rare ones longer codes, instead of 8 bits each: in English, *e* might get 3 bits and *z* 10. ZIP and PNG use it with a dictionary method.

Lossy techniques

- **JPEG images:** works on 8 × 8 pixel blocks, reducing fine detail and colour information the eye hardly notices (our eyes are more sensitive to brightness than to colour).
- **MP3 and AAC sound:** removes sounds people cannot hear, and quiet sounds *masked* by louder ones; a song shrinks to about a tenth of its CD size.
- **Video (MP4):** most of a scene stays the same, so only the **changes** between frames are stored, and each frame is compressed like a JPEG.
- **Lower settings:** a lower resolution, colour depth or sampling rate also shrinks files.

Pause & Try

Use RLE to compress AAAABBBBBBCCDDDDD and count how many values you save. Then explain why RLE works badly on a photograph.

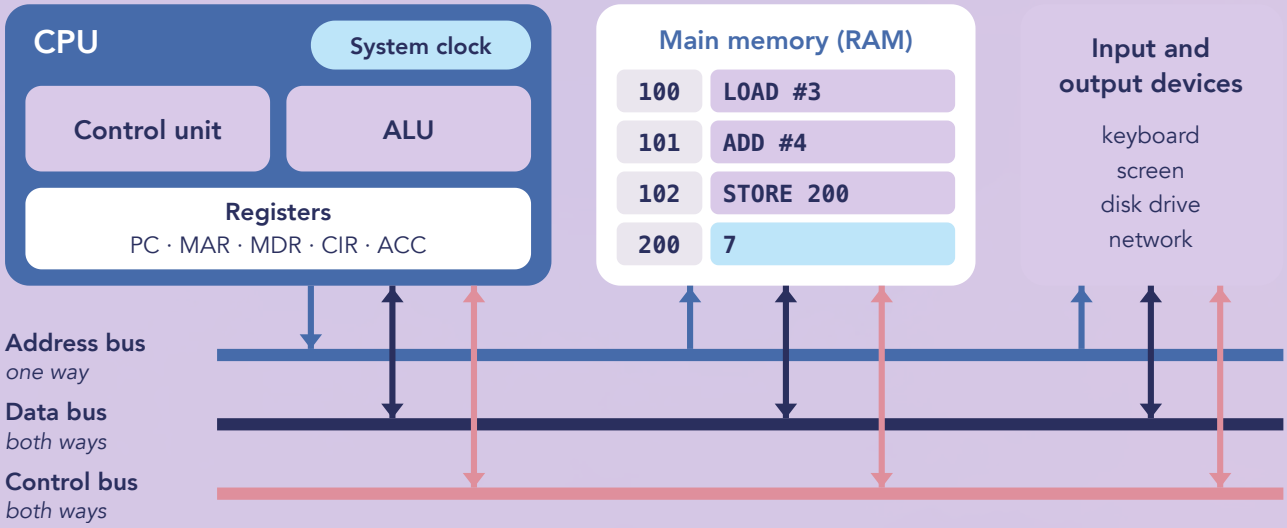
1.3 COMPUTER ARCHITECTURE & THE VON NEUMANN MODEL

Process execution

A program is a list of instructions; **process execution** is how the CPU carries them out. Almost every computer follows a design described by John von Neumann in 1945: the **Von Neumann architecture**.

Its key idea is the **stored-program concept**: instructions and data are stored **together in the same main memory**, and the CPU fetches the instructions one at a time. Early computers such as ENIAC had to be rewired for each new job; a stored-program computer just loads a different program.

The Von Neumann model



Inside the CPU

- **Control unit (CU):** decodes instructions and sends signals telling other parts what to do, timed by the **system clock**.
- **Arithmetic logic unit (ALU):** does arithmetic and logic (AND, OR, NOT, comparisons).
- **Registers:** tiny, very fast stores for the values in use now.

Outside the CPU

- **Main memory** (RAM, or **immediate access store**): holds the program and its data; every location has an **address**.
- **Buses:** parallel wires carrying addresses, data and control signals.
- **Input and output devices:** let data in and results out.

Pause & Try

Explain what is meant by the stored-program concept, and describe the job of the control unit and of the ALU.

1.3 COMPUTER ARCHITECTURE & THE VON NEUMANN MODEL

Registers, buses and the clock

Registers are the fastest storage in a computer: a handful of tiny locations inside the CPU, each holding one word (for example, 64 bits). The five below are **special-purpose registers**, each with one job in running a program.

Five special-purpose registers

Register	What it holds
PC · program counter	the address of the **next** instruction to fetch
MAR · memory address register	the address of the memory location being read or written
MDR · memory data register	data or an instruction just read from memory, or about to be written to it
CIR · current instruction register	the instruction being decoded and executed
ACC · accumulator	the value being worked on, and results from the ALU

The **status register** records the result of each operation using single-bit **flags**, such as N (negative), Z (zero), C (carry) and V (overflow).

The three buses

- **Address bus:** carries addresses from the MAR to memory, **one way** only (unidirectional). Its width sets how many locations it can reach: 16 wires give 2^{16} = 65 536 addresses.
- **Data bus:** carries data and instructions **both ways** (bidirectional) between the CPU, memory and I/O. A 64-bit data bus moves 64 bits at a time.
- **Control bus:** carries signals such as *memory read*, *memory write*, clock pulses and interrupt requests, both ways.

The system clock

The clock sends out a steady stream of pulses. Each pulse is one **clock cycle**, and every step of the CPU's work takes one or more cycles: a 3.5 GHz clock gives 3.5 billion cycles a second.

Overclocking runs the clock faster than the maker's setting: quicker, but the CPU gets hotter and may become unstable. More **cache** memory, more **cores** and wider buses also improve performance.

Pause & Try

a) Which register holds the address of the next instruction? b) How many memory locations can a 24-bit address bus reach? c) Explain why the data bus must be bidirectional.

1.3 COMPUTER ARCHITECTURE & THE VON NEUMANN MODEL

Fetch–execute cycle

Now the parts work together. The CPU runs a program by repeating one loop billions of times a second: **fetch** the next instruction from memory, **decode** it, **execute** it. Let us follow one instruction: the PC holds **100**, and memory location 100 contains ADD 201 — "add the number stored at address 201 to the accumulator".

Who does what

- **PC** — address of the next instruction
- **MAR** — address being read or written
- **MDR** — the data or instruction just read
- **CIR** — the instruction being decoded
- **ACC** — results from the ALU

Fetch (steps 1–4) copies the instruction from memory into the CPU. **Decode** (step 5): the control unit splits it into the *opcode* (ADD: what to do) and the *operand* (201: what to use). **Execute** (step 6): the control unit sends signals so that the right parts do the work.

One trip around the loop

1 · PC → MAR
MAR = 100, sent on the address bus

↓

2 · memory → MDR
ADD 201 returns on the data bus

↓

3 · PC + 1
PC = 101, the next instruction

↓

4 · MDR → CIR
CIR = ADD 201

↓

5 · decode
opcode ADD, operand 201

↓

6 · execute
ALU adds the value at 201 to ACC

The PC is increased **before** the instruction is executed, so the CPU already points at the next instruction while it works on this one. After step 6 the loop repeats — unless the instruction was a **jump**, which puts a new address into the PC. At the end of each cycle the CPU also checks for an **interrupt**: our next topic.

Watch the tutorial: Program execution (Video 05)
https://github.com/user-github-me/cs-ai-project-01/blob/main/assets/tutorials/stage2/videos/BlueImpulse_Video05_Program_Execution.mp4

Pause & Try

Put the six fetch–execute steps in order without looking, then explain why the PC is incremented before the instruction is executed.

1.3 COMPUTER ARCHITECTURE & THE VON NEUMANN MODEL

Interrupts

The fetch–execute cycle runs one instruction after another. But what if you press a key or a printer runs out of paper while the CPU is busy? The device sends an **interrupt**: a signal asking the CPU to pause and deal with an event. Without interrupts the CPU would waste time checking every device in turn (*polling*); with them, it does useful work and responds only when needed.

Where interrupts come from

- **Hardware:** a key press, a printer out of paper, data arriving from a network.
- **Timer:** a regular clock signal, so the OS can share CPU time.
- **Software:** an error such as division by zero, or a request for an OS service.

Priorities

Each interrupt has a **priority**: a power failure matters more than a key press. A low-priority interrupt waits while something more important is handled, and during critical work the CPU can briefly ignore (*mask*) interrupts.

How the CPU handles an interrupt

1 · Check
at the end of each fetch–execute cycle, the CPU checks for a waiting interrupt

↓

2 · Save
if its priority is higher than the current task, the registers (including the PC) are saved

↓

3 · Jump
the PC is loaded with the address of the right interrupt service routine (ISR)

↓

4 · Service
the ISR deals with the event, e.g. reads the code of the key that was pressed

↓

5 · Restore and resume
the saved values are put back and the program carries on where it stopped

Pause & Try

A user presses a key while a program is running. Describe what happens from the moment the keyboard sends an interrupt until the program carries on.

1.3 COMPUTER ARCHITECTURE & THE VON NEUMANN MODEL

Peripherals and ports

A **peripheral** is any device connected to a computer that is not part of its core — the CPU and main memory. Peripherals let you put data in, get results out and keep data safe. Each one plugs into a **port** and is controlled by the operating system through a small program called a **device driver**.

Common peripherals

Peripheral	Type	What it does
Keyboard	input	each key press sends a code, such as an ASCII code
Mouse or touchpad	input	sends movements and clicks to control a pointer
Microphone	input	turns sound into a signal for the ADC
Webcam or scanner	input	captures images as a grid of pixels
Monitor	output	shows images made of pixels
Printer	output	puts text and images onto paper
Speakers or headphones	output	turn the DAC's signal back into sound
External drive or USB stick	storage	keeps files when the power is off and carries them between computers

USB

Universal Serial Bus. A *serial* connection: the bits travel one after another. USB is **plug-and-play**: when you plug a device in, the operating system recognises it and loads its driver automatically. USB also supplies power, so it can charge a phone. It connects keyboards, mice, printers, flash drives and phones; newer devices use the reversible USB-C plug.

HDMI and VGA

HDMI (High-Definition Multimedia Interface) carries **digital** video *and* sound in one cable, at HD and 4K resolutions — for TVs, monitors, projectors and games consoles.

VGA (Video Graphics Array) is an older, **analogue**, video-only connection with a 15-pin plug. Its picture loses sharpness at high resolutions and over long cables, but it is still found on older projectors.

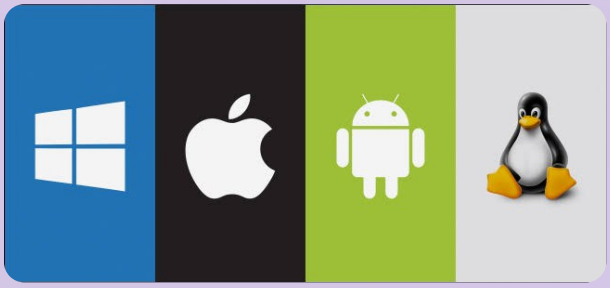
Pause & Try

Explain what a peripheral is, giving one input, one output and one storage example. Then state two differences between HDMI and VGA.

1.4 OPERATING SYSTEMS & SOFTWARE ARCHITECTURE

Why computers need an OS

Hardware on its own does nothing: every action needs software to control it. The most important piece of software on any computer is the **operating system (OS)**. When you switch on, the **BIOS** — a small program kept in memory on the motherboard that is not erased when the power is off — checks the hardware and then loads the OS from storage into RAM. From then until you shut down, the OS is in charge.



Windows, Apple's macOS and iOS, Android and Linux are widely used operating systems.

What the OS does for you

- Gives you a **user interface** — windows, icons and menus, or a command line — to start programs and manage files.
- **Hides the hardware's complexity:** a program simply asks the OS to *save this file*, and the OS deals with the disk.
- **Shares out resources** — CPU time, memory, storage and devices — between all the programs that are running.
- Organises **files** into folders and keeps the system and its users' data **safe**.

Without an operating system, every program would need its own code to drive the screen, the keyboard and the disk, and two programs could not safely run at the same time. To do its job, the OS carries out four key management tasks. The next two pages look at each of them in turn.

The OS's four management tasks

Task	What it manages
Memory management	which program uses which part of RAM
Process management	which program uses the CPU, and for how long
Hardware management	every device, through drivers, queues and buffers
Security management	who can log in and what each user may access

Pause & Try

Give three reasons why a computer needs an operating system.

1.4 OPERATING SYSTEMS & SOFTWARE ARCHITECTURE

Memory and process management

Memory management

Every running program needs space in RAM for its instructions and data. Many programs share limited memory, so the OS organises it carefully.

Allocating and protecting

- **Allocates** each program its own area of RAM when it starts, and **frees** it when the program closes.
- Records which parts of memory are in use, and by which program.
- **Protects** each area, so one program cannot read or overwrite another's memory — or the OS's own.

Virtual memory

When RAM is full, the OS moves blocks of data that are not needed right now (**pages**) onto the disk, and brings them back when needed. This **virtual memory** lets you run more programs than RAM alone could hold — but disks are slow, so constant swapping makes the computer crawl (*disk thrashing*).

Process management

A **process** is a running program. One CPU core can execute only one process at a time, yet you can stream music, browse and download at once. The OS's **scheduler** makes this possible:

Sharing the CPU

- Each process gets the CPU for a short **time slice** (a few milliseconds); then a timer interrupt passes the CPU on. The switching is so fast that programs seem to run together — **multitasking**.
- Processes have **priorities**: the program you are typing in must respond at once; a background update can wait.
- A process waiting for input or output is **blocked**, so it does not waste CPU time.
- The OS also tries to prevent **deadlock**: two processes each waiting for a resource the other holds.

Ready waiting for a turn → **Running** using the CPU → **Blocked** waiting for I/O, then ready again

Pause & Try

Explain how an operating system lets several programs appear to run at the same time on one CPU core, and describe what virtual memory is used for.

1.4 OPERATING SYSTEMS & SOFTWARE ARCHITECTURE

Hardware and security management

Hardware management

The OS controls every device connected to the computer, from the disk inside the case to the printer on the desk.

Device drivers

Every device is different, so the OS uses a **device driver** for each one: a small program that translates the OS's general commands, such as *print this page*, into the exact signals that particular printer understands. When you plug in a new USB device, the OS identifies it and loads the right driver (plug-and-play).

Queues and buffers

Devices are much slower than the CPU. The OS keeps a **queue** of jobs for each device — several documents can wait in the print queue — and holds data in a **buffer** (a temporary area of memory) so that the CPU can carry on while the device works. It also handles the **interrupts** that devices send.

Program asks to print → **Print queue** waits its turn → **Device driver** printer commands → **Printer** prints from a buffer

Security management

The OS protects the computer and its data from people who should not have access, and from damage.

How the OS keeps a computer secure

- **User accounts:** each user logs in with a username and a password, PIN or fingerprint; the OS can lock an account after several wrong attempts.
- **Access rights:** every file and folder has permissions (for example read, write or delete), and only administrators can change system settings or install software.
- **Security updates:** the OS downloads patches that close weaknesses attackers could use.
- **Built-in protection:** a firewall blocks unwanted network traffic, and many operating systems include virus checking and can encrypt the whole disk.

Pause & Try

Describe two ways an operating system keeps a computer secure, and explain what a device driver does.

1.4 OPERATING SYSTEMS & SOFTWARE ARCHITECTURE

Utility software

Utility software (or *utilities*) means programs that maintain, protect and tidy up a computer. Some come with the operating system; others are installed separately. Here are six you should know.

Disk formatter

Prepares a new hard disk, SSD or USB stick for use by setting up a **file system** — the structure the OS uses to record where each file is stored. Formatting removes access to any files already on the drive, so it is also used to wipe a drive clean.

Virus checker

Scans files and memory for **malware** by comparing them with a database of known virus *signatures* and watching for suspicious behaviour. Infected files are deleted or **quarantined**. It must be updated often to recognise new viruses.

Defragmentation software

On a hard disk, files are saved in blocks. As files are created, deleted and grow, a file's blocks become scattered (**fragmented**), so the read head must jump around and loading slows down. Defragmenting moves each file's blocks next to each other. SSDs have no moving parts and do not need this.

Repair software

Checks a disk for problems — damaged areas or errors in the file system's records — and either fixes them or marks the bad areas so that they are not used again. Examples include *Check Disk* in Windows and *Disk Utility* in macOS.

File compression

Makes files smaller, usually with lossless methods such as ZIP, to save storage space and to make them quicker to send. Several files can also be packed into a single archive.

Back-up software

Copies files to another device or to the cloud, so that they can be restored if the originals are deleted, damaged, stolen or locked by ransomware. Back-ups can run automatically on a schedule, e.g. *File History* (Windows) or *Time Machine* (macOS).

Pause & Try

A hard disk has become slow and some files will not open. Name two utilities that could help, and explain what each one does.

1.5 LANGUAGE TRANSLATORS & CODE EXECUTION

From source code to machine code

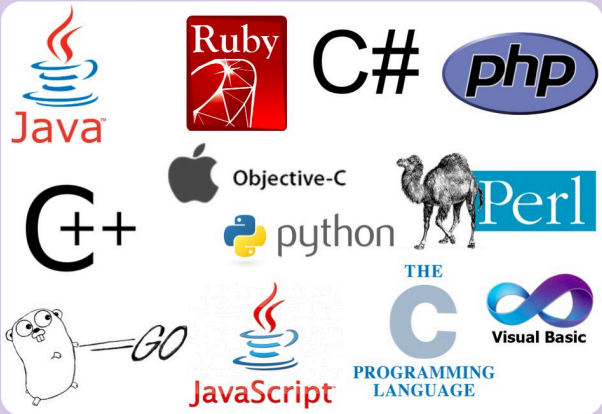
A processor can only execute **machine code**: instructions in binary, taken from the set of instructions its circuits are built to carry out. People, however, write programs in programming languages. The program as the programmer writes it is called **source code**, and a **translator** is a program that converts source code into machine code that the processor can run.

High-level languages

Python, Java, C++ and JavaScript use English-like words and mathematical notation. One line, such as `total = price + tax`, may become several machine-code instructions. High-level languages are **portable** (not tied to one type of processor) and much easier to write, read and debug.

Low-level languages

Machine code and **assembly language** work directly with the processor's own instructions, registers and memory addresses, so each type of processor has its own. They are harder to write, but give precise control and small, fast programs — useful for device drivers and embedded systems.



A few of the hundreds of high-level languages. Each one needs a translator before a processor can run its programs.

Three types of translator

- **Assembler:** translates assembly language into machine code. *Best for* low-level programs that control the hardware directly.
- **Compiler:** translates a whole high-level program into machine code before it runs. *Best for* finished programs that are given to users and must run fast.
- **Interpreter:** translates and runs a high-level program one statement at a time. *Best for* writing, learning and testing code, and for short scripts.

Pause & Try

Explain the difference between source code and machine code. Then say which translator you would use to run a short script you are still testing, and why.

1.5 LANGUAGE TRANSLATORS & CODE EXECUTION

Compilers and interpreters

Compilers and interpreters both translate high-level languages, but they work in very different ways — and that changes how you write, test and share a program.

A compiler

Source code
the whole program

↓

Compiler
checks and translates it all; lists every error

↓

Executable file
machine code, saved

↓

Run
as often as you like, without the compiler

An interpreter

Read one statement

↓

Check and translate it
stops at the first error

↓

Execute it
straight away

↓

Next statement
repeated every time the program runs

Benefits and drawbacks

	Compiler	Interpreter
Translates	Once, before the program runs	Every time the program runs
Errors	Listed together; nothing runs until all are fixed	Stops at the first error: easier debugging
Speed	Fast: already machine code	Slower: translated while it runs
Sharing	Users get only the executable; the source code stays private	Users need the interpreter and the source code

Many languages mix the two: Java compiles to *bytecode*, which a virtual machine interprets or compiles "just in time". Developers often test with an interpreter, then compile the finished version.

Pause & Try

A games company is about to sell a finished game, and a student is learning to program. Recommend a compiler or an interpreter for each, and justify your choices.

1.5 LANGUAGE TRANSLATORS & CODE EXECUTION

Assembly and machine code

Machine code is what the CPU executes. Each instruction has an **opcode** (*what to do*) and an **operand** (*what to use*: a number or a memory address). Binary is hard for people to write, so programmers use **assembly language**, where each opcode becomes a **mnemonic** such as LOAD or ADD, and **labels** can replace addresses.

Each assembly instruction becomes exactly one machine-code instruction (one-to-one); a high-level statement often becomes many. Assembly is tied to one processor family's instruction set.

Our teaching instruction set

Mnemonic	Opcode	What it does
LOAD #n	0001	put the number n into ACC
LOAD a	0010	copy the value stored at address a into ACC
ADD #n	0011	add the number n to ACC
SUB #n	0100	subtract the number n from ACC
STORE a	0101	copy ACC into address a
OUT	0110	output the value in ACC
JUMP a	0111	jump: put address a into the PC
JUMPZ a	1000	jump to address a only if ACC is 0
HALT	1111	stop the program

= a number to use directly; no # = an address. This set is invented for teaching, as in Video 05.

From assembly to binary

Each instruction is 16 bits: a 4-bit opcode, then a 12-bit operand.

STORE 200
assembly language

→

0101 0000 1100 1000
machine code

Pause & Try

Using the teaching instruction set, write the 16-bit machine code for LOAD #9, and explain the difference between LOAD #9 and LOAD 9.

1.5 LANGUAGE TRANSLATORS & CODE EXECUTION

The two-pass assembler

An **assembler** translates assembly language into machine code. There is one problem: a program may use a label *before* the line where it is defined. In the program below, line 103 says JUMPZ DONE, but the assembler does not reach the label DONE until line 105. This is called a **forward reference**. The solution is to read the program twice — a **two-pass assembler**.

A countdown program

Address	Label	Instruction
100		LOAD COUNT
101	LOOP	OUT
102		SUB #1
103		JUMPZ DONE
104		JUMP LOOP
105	DONE	HALT
106	COUNT	3

Symbol table built in pass 1

Label	Address
LOOP	101
DONE	105
COUNT	106

Line 106 is not an instruction: it reserves a memory location called COUNT, which starts with the value 3.

Pass 1 — find the labels

- Reads the program one line at a time and removes comments.
- Counts addresses, so it knows where each line will be stored.
- Checks that every mnemonic is in the instruction set.
- Adds each label to the **symbol table** with its address. A forward reference, such as DONE on line 103, is added first and given its address when the label is found.

Pass 2 — produce machine code

- Reads the program again from the start.
- Replaces each mnemonic with its opcode, and each label with its address from the symbol table: JUMPZ DONE becomes JUMPZ 105.
- Converts every operand to binary and outputs the finished machine code (the **object code**), or reports errors such as a label that was never defined.

Pause & Try

Describe what happens in each pass of a two-pass assembler, and explain why an assembler that read the program only once could not translate line 103.

1.5 LANGUAGE TRANSLATORS & CODE EXECUTION

Tracing a program

A **trace table** records how the values in registers and memory change as a program runs, one instruction at a time. Tracing lets you predict exactly what a program will do — and find mistakes — without a computer. Here is the countdown program from the previous page, starting with the PC at 100 and COUNT = 3.

Trace table

Address	Instruction	ACC	Output
100	LOAD COUNT	3	
101	OUT		3
102	SUB #1	2	
103	JUMPZ DONE		
104	JUMP LOOP		
101	OUT		2
102	SUB #1	1	
103	JUMPZ DONE		
104	JUMP LOOP		
101	OUT		1
102	SUB #1	0	
103	JUMPZ DONE		
105	HALT		

How to trace

1. Start at the first address. Write one row for every instruction that is **executed**, not for every line of the listing — a loop produces repeated rows.
2. Use the instruction set to work out what each instruction does, and write down any value that changes.
3. For a jump, the PC gets the new address: JUMP LOOP sends the CPU back to 101.
4. For JUMPZ, check ACC first: it jumps only when ACC is 0. On the third pass ACC is 0, so the CPU jumps to 105 and halts.

The output is 3, 2, 1. The listing has only seven lines, yet thirteen instructions were executed, because the loop ran three times.

Pause & Try

Trace the program again with COUNT = 5 and line 102 changed to SUB #2. What is output, and what problem do you notice?

1.5 LANGUAGE TRANSLATORS & CODE EXECUTION

Features of a typical IDE

An **integrated development environment (IDE)** is one application that brings together everything a programmer needs: an editor to write the code, a translator and run-time environment to run it, and tools to find and fix errors. IDLE, Thonny, Visual Studio Code, PyCharm and Xcode are common examples.

A typical IDE window, paused at a breakpoint

total.py

```
1  total = 0
2  for n in range(1, 4):
3      total = total + n
4      print(total)
5
6  print("Done")
```

red dot = breakpoint;
highlighted line runs next

Variables (watch)

```
total = 3
n = 2
```

Output

```
1
```

Features for writing code

- **Syntax highlighting** (also called pretty printing): keywords, names and text appear in different colours.
- **Auto-completion:** the IDE suggests names and keywords as you type.
- **Automatic indentation and bracket matching** keep the structure clear.
- **Expanding and collapsing** blocks of code lets you focus on one part.

Features for finding errors

- **Error highlighting:** syntax errors are underlined as you type, before the program runs.
- **Breakpoints** pause the program at a chosen line.
- **Single stepping** runs the program one line at a time.
- A **variable watch** window shows each variable's value as it changes — like a trace table filled in for you.

A built-in **run-time environment** lets you run the program inside the IDE and see its output straight away, without switching between separate tools.

Pause & Try

Describe three features of an IDE that help a programmer to find and fix errors.

CHAPTER 1 · PRACTICE

Practice

Practice · Chapter 1

1. The following bytes represent binary integers in two's complement form.
   a. State the equivalent denary values of 0100 1111 and 1001 1010. [2]
   b. Write the integer −53 in two's complement form. [1]
   c. Write the maximum possible range of numbers using an 8-bit two's complement number. Give your answers in denary. [2]

2. Binary-coded decimal (BCD)
   a. Write the denary integer 798 in BCD format. [1]
   b. Write the denary number represented by the BCD number 1001 0111 0111 0110. [1]
   c. Give one use of the binary-coded decimal system. [1]

3. A software developer uses a microphone and a sound-editing app to collect sounds for a new game.
   a. State what is meant by sampling resolution. [1]
   b. Describe how sampling resolution affects how accurately the digitised sound is stored. [2]
   c. Explain the term image resolution. [1]
   d. The developer uses 16-colour bitmap images. State the number of bits required to encode one pixel. [1]
   e. One image is 16 384 pixels wide and 512 pixels high and is saved as a 256-colour bitmap. Calculate the size of the file in gibibytes. [3]

4. Write these six stages of the Von Neumann fetch–execute cycle in the correct order: the instruction is copied from the MDR and placed in the CIR; the instruction is executed; the instruction is decoded; the address contained in the PC is copied to the MAR; the value in the PC is incremented by 1; the instruction is copied from the memory location in the MAR and placed in the MDR. [6]

5. File History and Time Machine are back-up utilities offered by two different operating systems. Explain why it is important to back up files on a computer. [3]

6. Assemblers, compilers and interpreters are all used to translate programs. Discuss the different roles played by each translator. [6]

CHAPTER 1 · PRACTICE

Practice (continued)

Practice · Chapter 1 (continued)

7. Number systems and binary arithmetic
   a. Convert the byte 1110 0101 into hexadecimal. [1]
   b. Convert the hexadecimal number 3B into denary. [1]
   c. Add the 8-bit binary numbers 0010 1101 and 0011 0110, showing your working. [2]
   d. Explain what is meant by overflow, using an 8-bit example. [2]

8. Text, images and sound
   a. The ASCII code for 'A' is 65. Write the 8-bit binary code for 'E'. [1]
   b. Describe two differences between a bitmap image and a vector graphic. [4]
   c. Calculate the size, in megabytes, of a 3-minute mono recording sampled at 22 050 Hz with a sampling resolution of 8 bits. [3]
   d. Use run-length encoding to compress this row of pixels: WWWWBBBWWWWWWBB. [2]
   e. Explain why a music-streaming service uses lossy compression, but a bank must compress its records losslessly. [3]

9. The processor and its connections
   a. Describe the roles of the MAR and the MDR in the fetch–execute cycle. [2]
   b. Identify the bus that is unidirectional, and explain why it only needs to carry signals in one direction. [2]
   c. Describe what happens when the CPU receives an interrupt. [4]
   d. State one use for each of these connections: USB, HDMI and VGA. [3]

10. Operating systems and utilities
    a. Describe two tasks carried out by an operating system's memory management. [2]
    b. Explain why defragmentation software can speed up a hard disk, and why it is not needed on an SSD. [3]
    c. Name two other utility programs and describe what each one does. [4]

11. Translators and assembly language
    a. State the relationship between assembly language and machine code. [1]
    b. Describe the two passes of a two-pass assembler, including the role of the symbol table. [4]
    c. Using the teaching instruction set, trace this program and state its output: 100 LOAD #6 · 101 SUB #2 · 102 OUT · 103 JUMPZ 105 · 104 JUMP 101 · 105 HALT. [4]
    d. Describe three features of an IDE that help a programmer to write code. [3]

MORNING

BEHAVIORISM & BEHAVIORIST AI

2

CHAPTER 2 · BEFORE YOU BEGIN

Learning goals

This chapter begins with how animals and people learn from their surroundings, then follows that idea into machines: robots that react directly to what their sensors detect, and software agents that learn from rewards and penalties. On the way you will meet the mathematics behind this kind of learning — Q-learning and Markov decision processes — and see how each idea grows out of the one before it.

Learning goals

You will learn about:

- What behaviourism is and its main principles
- How behaviour is influenced by external stimuli and environmental interactions
- How observable responses to stimuli explain learning and behavioural changes
- What behaviour-based robotics is and how robots use sensory input to respond to their surroundings
- What Behaviourist AI is and how it relates to the principles of behaviourism
- How AI can develop intelligent behaviour through sensory input and interaction with the environment rather than relying on complex internal models of the world
- What Reinforcement Learning (RL) is and how agents learn through trial and error using rewards and penalties
- The basic components of Reinforcement Learning, including agents, environments, states, actions and rewards
- The common algorithms and techniques used in Reinforcement Learning
- What Markov Decision Processes (MDPs) are and their role in modelling decision-making under uncertainty
- How Reinforcement Learning and MDPs enable AI systems to learn, make decisions and adapt their behaviour to different environments

One idea, four steps

2.1 Behaviourism — animals learn from their surroundings → **2.2 Behaviourist AI** — robots react to their surroundings →

2.3 Reinforcement learning — agents learn from rewards → **2.4 MDPs** — decisions under uncertainty

Each step builds on the one before. The thread that ties them together: behaviour is shaped by interaction with the environment — first in living things, then in machines.

CHAPTER 2 · BEFORE YOU BEGIN

Key terms

Key terms

Behaviourism: A theory of learning that focuses on observable behaviour and how it is influenced by external stimuli and environmental interactions.

Stimulus: An external event or change in the environment that triggers a response from an organism or agent.

Response: An observable reaction or behaviour produced in response to a stimulus.

Stimulus–response (S–R): The relationship between an external stimulus and the observable response it produces.

Environmental interaction: The process through which an organism or agent senses and responds to changes in its surroundings.

Classical conditioning: Learning by association: a neutral stimulus that is repeatedly paired with a reflex trigger comes to produce the reflex on its own (Pavlov).

Operant conditioning: Learning from consequences: a voluntary behaviour becomes more or less frequent depending on what follows it (Skinner).

Reinforcement: A consequence that makes the behaviour it follows more likely to happen again.

Punishment: A consequence that makes the behaviour it follows less likely to happen again.

Behaviourist AI: An approach to artificial intelligence that develops intelligent behaviour through sensory input and environmental interaction rather than relying primarily on complex internal representations of the world.

Behaviour-based robotics: An approach to robotics in which robots use sensory input and simple behavioural rules to interact with and respond to their environment.

Symbolic AI (GOFAI): Early "good old-fashioned" AI that builds a detailed internal model of the world and plans before it acts.

Sensor: A device that detects information from the environment, such as touch, light, distance or temperature.

Sensory input: Information collected from the environment through sensors, allowing an AI system or robot to detect and respond to external conditions.

Actuator: A device, such as a motor, that turns a control signal into physical movement or action.

Reactive control: Control by fixed stimulus–response rules that link sensor readings directly to actions and do not change with experience.

Subsumption architecture: Layered robot control without a central map, in which a higher-priority behaviour can suppress the ones below it.

Feedback loop: A process where an AI system uses the result of its actions to adjust its future behaviour.

Adaptive behaviour: The ability of an AI system to adjust its responses based on feedback or changing conditions.

Reinforcement learning (RL): A machine learning approach in which an agent learns to make decisions through trial and error by receiving rewards or penalties.

Trial and error: A learning process in which an agent tries different actions, evaluates their outcomes and adjusts its behaviour based on feedback.

CHAPTER 2 · BEFORE YOU BEGIN

Key terms

Key terms (continued)

Agent: The learner or decision-maker in reinforcement learning that interacts with an environment.

Environment: The external setting in which an agent operates, takes actions and receives feedback.

State: A representation of the current situation or condition of the environment.

Action: A decision or movement made by an agent that affects the environment.

Reward: Feedback received by an agent that indicates how desirable the outcome of an action is.

Penalty: Negative feedback or a negative reward that discourages undesirable actions or outcomes.

Return: The total reward an agent collects from a given moment until the end of an episode.

Episode: One complete attempt at a task, from the start until a terminal state is reached.

Terminal state: A state in which an episode ends, such as reaching the goal or falling into a trap.

Policy: A strategy or rule that determines which action an agent should take in a given state.

Value function: A function that estimates the expected future rewards an agent can obtain from a particular state or state–action pair.

Q-learning: A model-free reinforcement learning algorithm that learns the expected value of taking specific actions in particular states.

Q-value, Q(s, a): The estimated return of taking action a in state s and acting well afterwards; Q-values are stored in a Q-table.

Model-free: Describes learning that needs no model of how the environment works; the agent learns purely from experience.

Learning rate (α): A number between 0 and 1 that controls how much a single experience changes a Q-value.

Exploration: The process of trying different actions to discover potentially better strategies or rewards.

Exploitation: The process of choosing actions based on existing knowledge to maximise expected rewards.

Epsilon-greedy (ε-greedy): A strategy that explores with a small probability ε and otherwise exploits.

Markov decision process (MDP): A mathematical framework for modelling decision-making in situations where outcomes are partly uncertain and depend on the current state and chosen action.

Markov property: The future depends only on the current state and action, not on how the agent got there.

Transition probability: The probability that an action will move an agent from one state to another in an environment.

Discount factor (γ): A value that determines how much an agent prioritises future rewards compared with immediate rewards.

2.1 FOUNDATIONS OF BEHAVIORISM

Learning from the environment

Behaviourism is a school of thought in psychology that grew up in the early 20th century, shaped by pioneers such as **Ivan Pavlov**, **John B. Watson** and **B. F. Skinner**. Before it, psychologists mostly studied the mind by *introspection*, asking people to describe their own thoughts and feelings — reports that nobody else can check. The behaviourists argued that a real science must study only what can be **observed and measured**: what an organism does, and the events in its environment that come before and after.

The central principle of behaviourism

Behaviour is learned, modified and maintained through interaction with the environment and through the consequences of that behaviour.

- **Learned:** a new behaviour appears through experience — a puppy learns to sit when told.
- **Modified:** a behaviour changes when the environment changes — you take a new route to school when roadworks slow down the old one.
- **Maintained:** a behaviour continues while it keeps paying off and fades when it stops — the puppy keeps sitting while treats keep coming.

Every idea in this chapter, from Pavlov's dogs to learning robots, is an example of this principle.

Living things constantly adjust what they do to their surroundings: where and how these wading birds feed depends on what their environment offers.

The brain as a "black box"

Behaviourists did not try to guess what happens inside the head. They treated the brain as a **black box**: you cannot see inside it, but you can measure what goes in — the **stimulus** — and what comes out — the **response**.

Stimulus → **Black box** → **Response**

Instead of "the dog is happy", a behaviourist would record "the dog wagged its tail 40 times in a minute" — something anyone can observe and check.

Pause & Try

Give one example of a behaviour of your own that was **learned**, one that was **modified** and one that is **maintained**. For each, name the part of your environment or the consequence that shaped it.

2.1 FOUNDATIONS OF BEHAVIORISM

Stimulus and response

At the core of behaviourism is a direct link between a **stimulus** and a **response**, known as the **stimulus–response (S–R) association**. A **stimulus (S)** is any detectable change or event in the environment: a bright light, a bell ringing, a touch sensor being pushed. A **response (R)** is the measurable action or reaction it produces: blinking, moving away, turning left.

Spotting S–R pairs

Stimulus (S)	Response (R)	Inborn or learned?
a bright light shines in your eyes	your pupils shrink	inborn reflex
something flies towards your face	you blink	inborn reflex
the end-of-lesson bell rings	students pack their bags	learned
your phone buzzes	you check the screen	learned
a robot vacuum's bumper is pressed	it reverses and turns	built in by its designer

Notice the last row: the S–R idea describes machines as well as living things. That is why it later became a starting point for artificial intelligence.

Inborn links and learned links

Some S–R links are **reflexes** you were born with: nobody teaches you to blink. Many others are **learned**: no baby is born knowing that a bell means the end of a lesson. Through continuous interaction with their environment, living things keep building and adjusting these learned links — this is how they survive and thrive in a changing world.

Psychologists found two main ways in which new S–R links are learned. In **classical conditioning**, an organism learns that one *stimulus* predicts another. In **operant conditioning**, it learns from the *consequences* of its own actions. The next two pages explain each one.

Pause & Try

A self-checkout machine beeps when you scan an item, and you place the item in the bagging area. Identify the stimulus and the response, and decide whether this S–R link is inborn or learned. Explain your answer.

2.1 FOUNDATIONS OF BEHAVIORISM

Classical conditioning

Around 1900, the Russian scientist **Ivan Pavlov**, while studying digestion in dogs, noticed that they began to salivate *before* any food reached them — at the sight or sound of the assistant who fed them. Pavlov tested this by sounding a bell or a metronome just before feeding, and so discovered **classical conditioning**: *learning through association between stimuli.*

Pavlov's experiment in three stages

1 · Before conditioning
food → salivation
sound → no salivation

→

2 · During conditioning
sound, then food,
repeated many times

→

3 · After conditioning
sound alone
→ salivation

The five parts, in plain words

Term	Meaning	In Pavlov's experiment
Unconditioned stimulus (UCS)	triggers a response without any learning	food
Unconditioned response (UCR)	the natural, automatic reaction to the UCS	salivating at food
Neutral stimulus (NS)	causes no response at first	the sound
Conditioned stimulus (CS)	the neutral stimulus once it has been learned	the sound
Conditioned response (CR)	the learned reaction to the CS	salivating at the sound

What an "elicited" response is

In classical conditioning the response is **elicited**: the stimulus *draws it out* of the animal automatically, like a knee-jerk. The dog does not decide to salivate and earns nothing for doing it — salivation is an **involuntary reflex** that has become attached to a new trigger. The link is maintained only while the association holds: if the sound is played many times without food, the conditioned response gradually fades. This is called **extinction**.

Pause & Try

At the dentist, the whine of the drill always comes just before a painful moment, and you tense up. After a few visits, the sound of a drill alone makes you tense. Identify the UCS, UCR, NS, CS and CR.

2.1 FOUNDATIONS OF BEHAVIORISM

Operant conditioning

Classical conditioning explains reflexes, but most of what you do is **voluntary**: you choose to raise your hand, open an app or practise a skill. From the 1930s, **B. F. Skinner** studied how voluntary behaviour is shaped by what happens *after* it. He called this **operant conditioning**, because the animal *operates* on its environment. In operant conditioning, *learning occurs through the consequences of voluntary behaviour.*

Skinner's experiment

A hungry rat is placed in a box with a lever. At first it presses the lever only by accident while exploring — but every press releases a food pellet.

Explores → **Presses the lever** by accident → **A pellet drops** pleasant result → **Presses more** behaviour strengthened

The rat's lever-pressing has been **shaped by its consequences**. The behaviour comes first, freely chosen by the animal; the consequence follows and changes how often it happens in future.

Reinforcement and punishment

A consequence that makes a behaviour **more likely** is **reinforcement**; one that makes it **less likely** is **punishment**. Each can work by *adding* something (**positive**) or *taking something away* (**negative**) — here, positive and negative mean added and removed, not good and bad.

	Add something	Take something away
Behaviour increases	positive reinforcement: a dog sits and gets a treat	negative reinforcement: you fasten your seat belt and the warning beep stops
Behaviour decreases	positive punishment: you touch a hot pan and get burnt	negative punishment: a teenager loses phone time for coming home late

Trainers use operant conditioning to build complex behaviour step by step, a technique called **shaping**: a dolphin is rewarded first for swimming near a hoop, then only for touching it, and finally only for jumping through it.

Pause & Try

A student is praised every time they hand in homework on time, and soon does so more often. Name the type of consequence, say whether handing in homework is voluntary or a reflex, and explain why this is operant rather than classical conditioning.

2.1 FOUNDATIONS OF BEHAVIORISM

From operant conditioning to reinforcement learning

Of the two kinds of conditioning, only operant conditioning involves **choosing actions and living with their consequences** — which makes it the direct ancestor of an important branch of AI.

Classical vs operant conditioning

	Classical (Pavlov)	Operant (Skinner)
How learning occurs	association between two stimuli	consequences of an action
Type of behaviour	involuntary reflex (elicited)	voluntary action (chosen)
Key event happens	before the response	after the response
Example	a dog salivates at a sound	a dog sits for a treat
AI parallel	pattern recognition: learning which signals predict others	reinforcement learning

Three steps from psychology to AI

Behaviourism the environment shapes behaviour → **Operant conditioning** consequences shape behaviour → **Reinforcement learning** numerical rewards shape an agent's choices

In reinforcement learning (RL), every part of Skinner's experiment has a machine version:

The rat the agent → **Lever press** an action → **Food pellet** a reward → **New habit** a policy

RL keeps the core idea — actions followed by good consequences become more likely — but builds it from **mathematics**: rewards are numbers, the goal is the largest total reward, and an **algorithm** updates the **policy**. The reward is only a training signal; the agent does not feel it.

Pause & Try

Explain why reinforcement learning is linked to operant conditioning rather than to classical conditioning. Use the words *action* and *consequence* in your answer.

2.2 BEHAVIORIST AI & BEHAVIOR-BASED ROBOTICS

Think first, act later

Early artificial intelligence research, from the 1950s to the 1980s, is often called **symbolic AI** or **GOFAI** — *Good Old-Fashioned AI*. Its researchers believed that to act intelligently, a robot first had to build a complete, detailed **internal model** of its surroundings — a 3D map of every wall, chair and doorway — and then reason about that model with symbols and logic.

Symbolic AI imagined intelligence as a "brain" that reasons about a detailed model of the world before the body moves.

Sense, model, plan, act

1 · Sense
gather camera and sensor data

↓

2 · Model
build a detailed 3D map

↓

3 · Plan
search the map for the best route

↓

4 · Act
only now, move

Why it struggled

Each stage must wait for the one before, so a symbolic robot spends most of its time calculating. **Shakey**, a famous robot built at the Stanford Research Institute between 1966 and 1972, often stood still for long periods while it planned its next move.

Worse, the world does not wait. If someone walks past or moves a chair while the robot is still calculating, its map is already out of date — and the robot may drive into something that is no longer where the map says. The real world is messy, changes constantly and can never be modelled completely.

In the mid-1980s, **Rodney Brooks**, a roboticist at MIT, proposed a radically different idea. It became known as **Behaviourist AI**, and it is the subject of the rest of this section.

Pause & Try

A symbolic robot takes 20 seconds to map a room and plan its route. Two seconds after it starts moving, a person pushes a chair into its path. Explain what goes wrong, and why.

2.2 BEHAVIORIST AI & BEHAVIOR-BASED ROBOTICS

What is Behaviourist AI?

Brooks argued that **"the world is its own best model"**: the real world is always up to date and contains every detail, so a robot should *look at it* whenever it needs to, rather than keep a copy in memory. Instead of one slow, central "brain", a robot needs many fast, direct links between its **sensors** and its **actuators**.

Definition

Behaviourist AI (also called behaviour-based AI) is an **approach** to designing intelligent systems in which intelligent behaviour **emerges from the interaction between sensors, control mechanisms and the environment**, rather than from reasoning about a detailed internal model of the world.

Behaviour-based robotics is this approach applied to robots: a robot is built from simple behaviours, each linking what it senses directly to what it does.

Situated

The robot exists in the real, physical world — not in an abstract simulation — and senses that world as it happens.

Embodied

The robot has a physical body — wheels, motors, arms — that experiences real physics: friction, bumps and slopes.

Related, but different disciplines

	Behaviourism	Behaviourist AI
Field	psychology	artificial intelligence and robotics
Aim	to explain how animals and people learn	to build machines that behave usefully
Subjects	animals and people	robots and software agents
Role of learning	the main thing it studies	optional: many such robots never learn

What they share: both explain behaviour through observable interaction with the environment, without relying on hidden inner states — mental states for psychologists, detailed world models for engineers.

Pause & Try

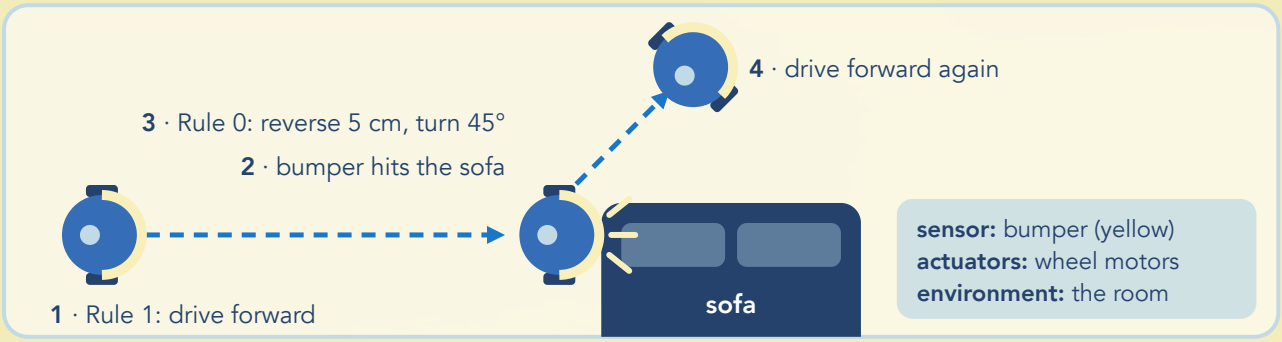
Define Behaviourist AI in your own words. Then give one way in which it is similar to behaviourism in psychology and one way in which it is different.

2.2 BEHAVIORIST AI & BEHAVIOR-BASED ROBOTICS

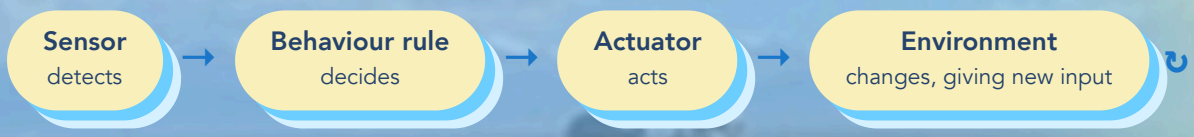
Sense, decide, act

Every behaviour-based robot runs the same simple loop, many times a second. A robot vacuum cleaner shows each part at work.

The vacuum cleaner analogy



A **symbolic** vacuum would map every chair leg and plan a route before moving — and a moved chair would break its plan. A **behaviourist** vacuum needs two rules. **Rule 0 (emergency):** if the bumper touches an object, stop, reverse 5 cm and turn 45°. **Rule 1 (default):** drive straight ahead.



What each part does

Part	Its job	In the robot vacuum
Sensor	detects information from the environment	bumper switch, cliff sensors
Behaviour rule	decides on an appropriate response	IF bumped THEN reverse and turn
Actuator	carries out the physical movement	the two wheel motors
Environment	changes, giving the sensors new input	walls, furniture, stairs

From two simple rules, intelligent-looking behaviour **emerges**: given time, the robot covers the whole floor with no map or master plan. This is called **emergent behaviour**.

Pause & Try

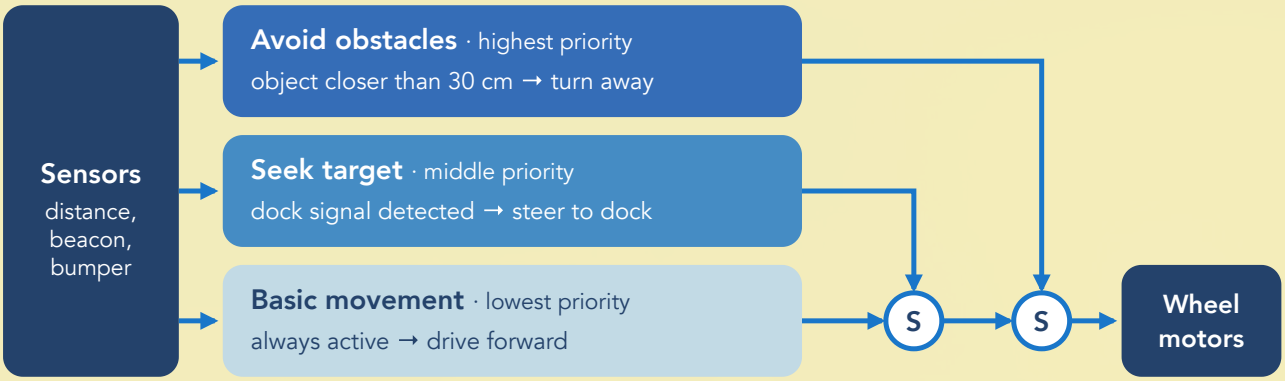
An automatic sliding door at a supermarket opens when someone walks towards it. Identify its sensor, behaviour rule, actuator and environment, and state its stimulus and response.

2.2 BEHAVIORIST AI & BEHAVIOR-BASED ROBOTICS

Layers of behaviour

Brooks' **subsumption architecture** (1986) organises a robot's behaviours in **layers**. Each layer is a simple behaviour linking sensors to motors on its own — there is no central map or planner. When a higher layer's trigger fires, it **suppresses** (takes over from) the layers below; when it goes quiet, they carry on.

Three layers in a home robot



S marks a suppression point: while the layer above is active, its command replaces the one arriving from the left. So avoiding obstacles always wins.

One trip to the charging dock

Time	Sensors report	Layer in control	Wheels
0 s	nothing nearby, no dock signal	basic movement	drive straight ahead
4 s	dock signal to the left	seek target	steer left, towards the dock
7 s	chair leg 20 cm ahead	avoid obstacles	turn away from the chair
9 s	path clear, dock signal again	seek target	steer towards the dock

Why is this more responsive than map-building AI? Each layer reacts to its sensors in milliseconds, with no map to rebuild or plan to recalculate — and if one layer fails, the others keep working.

Pause & Try

While the robot is steering towards its dock, a cat walks in front of it. Which layer takes control, which layer does it suppress, and what happens once the cat has passed?

2.2 BEHAVIORIST AI & BEHAVIOR-BASED ROBOTICS

Reactive control vs learning

Every robot so far uses **reactive control**: a designer writes fixed stimulus–response rules, and the robot follows them in exactly the same way every time. It reacts, but it never learns. The alternative is a robot that **learns from experience** which actions produce better outcomes. This difference — following rules versus learning — is the most important distinction in this chapter.

Reactive control

- Rules are **written by a designer** before the robot starts work.
- The same stimulus always produces the **same response**.
- Experience changes nothing: the thousandth bump is handled like the first.
- Fast, simple and predictable, but it **cannot improve** and fails in situations nobody foresaw.
- Like an **inborn reflex**.

Learned behaviour

- The robot discovers **which actions work best** from feedback.
- Its response to a stimulus can **change with experience**.
- Performance **improves over time** and can adapt to new situations.
- Learning takes time and data, and can be hard to predict while it happens.
- Like **operant conditioning** — in AI, **reinforcement learning**.

Two robot vacuums in the same narrow corner

Robot A (reactive) has one rule: when bumped, turn 45° to the right. In a narrow corner it bumps, turns, bumps the other wall, turns again — and can stay trapped for minutes, making the same choice every time.

Robot B (learning) receives a reward signal: +1 for each new patch of floor cleaned, −1 for each bump. Over many cleaning runs it learns that in this corner, turning left gets it out in one move. Robot A behaves identically on day 1 and day 100; Robot B is better on day 100 than on day 1.

The two are often combined: a learning robot usually keeps reactive safety rules, such as "stop if the bumper is pressed", that always take priority. The rest of this chapter is about the learning side: **reinforcement learning**.

Pause & Try

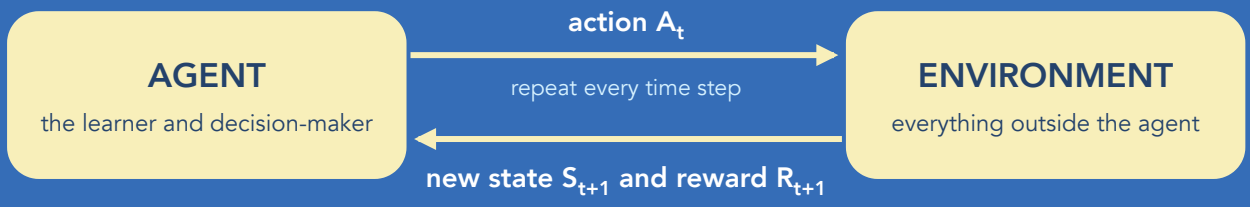
Classify each system as reactive control or learned behaviour, and explain why: a) a thermostat that switches the heating on below 19 °C; b) a chess program that wins more often after every game it plays; c) car headlights that switch on automatically when it gets dark; d) a delivery robot that, after weeks of trips, starts avoiding a corridor that is often crowded.

2.3 INTRODUCTION TO REINFORCEMENT LEARNING (RL)

The reinforcement learning loop

Reinforcement learning (RL) is the computational version of Skinner's operant conditioning. An **agent** learns to reach a goal by **trial and error**, through repeated interaction with an **environment**: it tries an action, sees what happens and gradually prefers actions that lead to more reward. Nobody tells it the correct move — it must discover it from the consequences of its own actions.

The interaction loop



At each **time step** t, the agent observes the state S_t and chooses an action A_t. The environment responds with a reward R_{t+1} and a new state S_{t+1}, and the loop begins again.

The parts of an RL system

Part	Meaning	A robot learning to reach its charger
Agent	the learner and decision-maker	the robot's control program
Environment	everything outside the agent	the room, its walls and the charger
State (S_t)	a snapshot of the situation at step t	the robot's position
Action (A_t)	a choice available in that state	forward, turn left, turn right
Reward (R_{t+1})	a number sent back after each action	+10 at the charger, −1 per move
Policy (π)	which action to take in each state	"in this spot, turn left"

The aim of RL is to find an **optimal policy**, written π^*: the strategy that collects the most reward in the long run. Section 2.4 shows that RL problems are usually described as **Markov decision processes**.

Pause & Try

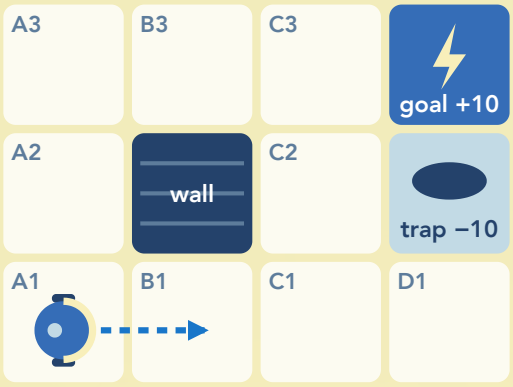
A program learns to play a car-racing video game. Identify the agent, the environment, one possible state, two actions and a sensible reward.

2.3 INTRODUCTION TO REINFORCEMENT LEARNING (RL)

One interaction, step by step

To see learning happen, RL is often studied in a **grid world**. Our agent is a small rover on a 4 × 3 grid. It starts on A1; B2 is a wall, D2 is a trap and D3 is the goal, a charging station. Entering the goal earns **+10** and entering the trap earns **−10** — both end the attempt. Every other move earns **−1**, a small penalty that encourages short routes; a move into the wall or off the grid leaves the rover where it is.

The grid world



The rover's first move: from A1 it chooses **right**.

One complete interaction

1 · Observe
the agent sees its state: it is on A1

2 · Choose
its policy picks an action: move right

3 · Respond
the environment moves the rover to B1

4 · Reward
the environment sends back −1 and the new state, B1

5 · Learn
the experience (A1, right, −1, B1) improves the agent's estimates and its policy

How experience changes behaviour

One attempt, from the start to the goal or the trap, is called an **episode**. In its first episode the rover knows nothing, so it wanders: suppose it makes six pointless moves and then falls into the trap, collecting 6 × (−1) + (−10) = **−16**. Then it starts again from A1. Episode after episode, the rewards tell it which moves pay off, and eventually it settles on a five-move route to the goal, collecting 4 × (−1) + 10 = **+6**. Its behaviour has been shaped by consequences, just like the behaviour of Skinner's rats.

Pause & Try

The rover is on C1 and chooses to move up. Write out the five steps of this interaction, including the reward and the new state.

2.3 INTRODUCTION TO REINFORCEMENT LEARNING (RL)

Reward, return, value and policy

Four RL words are easy to confuse, but in the grid world each has its own number. Each square shows the total reward the rover can expect to collect from there to the end of the episode by following the arrows.

Values and policy

Numbers are values; arrows show the policy. On A1, up and right are equally good.

Four different ideas

- **Reward** — the number for one move. The rover's first move from A1 earns **−1**, even though it is a good move.
- **Return** — the total reward from now until the episode ends. Following the arrows from A1: −1 −1 −1 −1 +10 = **+6**.
- **Value** — the return the agent *expects* from a state. V(A1) = 6 and V(C3) = 10: squares nearer the goal are worth more, although stepping onto them earns only −1.
- **Policy** — the rule for choosing actions; here, "move to the neighbouring square with the highest value".

A good agent does not chase the biggest reward for the *next* move; it follows the highest *value*. Every ordinary move costs −1, yet moving towards the goal is right, because the +10 at the end outweighs the small costs on the way. When outcomes are uncertain, a value is an **average** over many episodes — which is why the agent must learn it from experience.

Penalties are negative rewards

RL has no separate punishment signal: a **penalty** is simply a reward with a negative number. The −1 per move is a small penalty that discourages wandering; the −10 trap is a large penalty for a disastrous outcome. Penalties lower the values of the states and actions that lead to them, so the agent learns to avoid them. Take care: *negative reinforcement* in psychology means removing something unpleasant, which makes a behaviour more likely — it is not the same as a negative reward.

Pause & Try

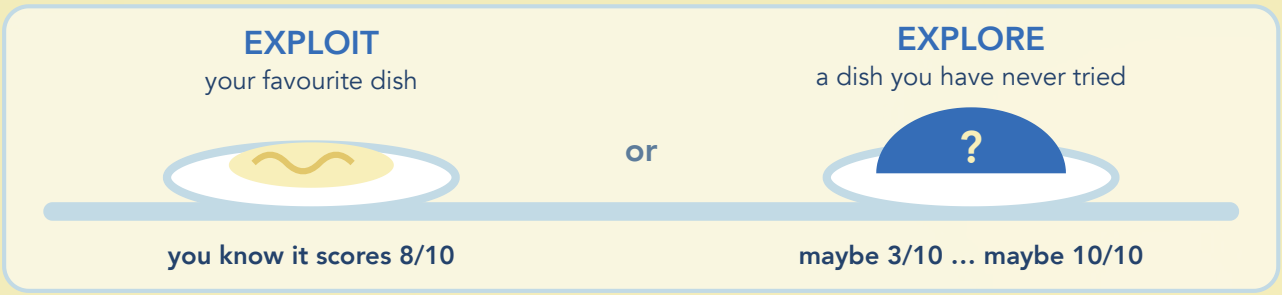
The rover starts on C1 and follows the arrows. State the reward for its first move, its return and the value of C1. Then explain why the rover on C2 should never move right.

2.3 INTRODUCTION TO REINFORCEMENT LEARNING (RL)

Explore or exploit?

To collect a high return, an agent must **exploit** what it already knows. But everything it knows comes from what it has tried — so it must also **explore**. Balancing the two is called the **exploration–exploitation trade-off**, and it is one of the central challenges in RL.

The restaurant analogy



Imagine visiting a restaurant you know well. **Exploitation** is ordering your favourite dish, because you know it is good. **Exploration** is trying a dish you have never had: it might disappoint you (a penalty), but you might discover something even better.

In RL terms, **exploration** means trying unfamiliar or apparently worse actions to **gather information** about the environment; **exploitation** means choosing the action with the **best current estimate**, to collect as much reward as possible now.

Why an agent must balance both

Early estimates are based on very little experience, so they can be wrong. An agent that **only exploits** sticks with the first action that worked and may never find a better one. An agent that **only explores** keeps gathering information but never uses it, so its rewards stay low.

Exploration is not just trying random things: every new experience **improves the agent's estimates**, and better estimates lead to better choices later. In the grid world, suppose the rover's first successful route took nine moves — a return of $8 \times (-1) + 10 = +2$. If it only exploited, it would repeat that route for ever. By exploring, it discovers the five-move route worth +6.

Pause & Try

A music app keeps playing songs by the first artist you liked. In RL terms, what is the app doing, and what might it be missing? Suggest how it could balance the two.

2.3 INTRODUCTION TO REINFORCEMENT LEARNING (RL)

Epsilon-greedy in action

A simple, popular way to balance the two is the **ε-greedy** (epsilon-greedy) strategy, where ε is a small probability such as 0.1. Before every decision the agent, in effect, rolls a die: with probability ε it **explores** by choosing an action at random; otherwise, with probability 1 − ε, it **exploits** by choosing the action with the highest **Q-value** — its current estimate of how good each action is.

One decision, worked through

An agent has three actions, and ε = 0.1.

Action	Q-value	Chosen when…
A	5	exploring
B	8	exploiting, or exploring
C	3	exploring

When it **exploits**, it picks **B**, the highest Q-value. When it **explores**, it picks A, B or C at random. Over 100 decisions it exploits about 90 times (always B) and explores about 10 times (3 or 4 of each), so B is chosen about 93 times and A and C about 3 times each.

Exploring improves the estimates

Why ever choose A or C? Suppose the agent estimates each action's value as the average reward it has received, and C's estimate of 3 came from a single unlucky try. Exploring C four more times gives rewards of 10, 9, 11 and 12. Its new estimate is (3 + 10 + 9 + 11 + 12) ÷ 5 = **9**, better than B's 8 — so the agent switches to C. Without exploration it would never have found out.

Many agents start with a large ε and **reduce it over time**, exploring less as their estimates become reliable.

Exploring safely: a real robot

Exploration has a real-world cost: a delivery robot learning its way around a hospital could, by acting at random, drive into a patient or towards a staircase. So engineers:

- train it first in a **simulation**, where mistakes cost nothing;
- let it explore only among **safe actions**, for example at walking speed;
- keep **reactive safety rules**, such as "stop if anything is within 50 cm", that override the learner — subsumption at work;
- keep a **human supervisor** who can stop it at any time.

No simulation is perfect, so the robot is still tested on site before it works alone.

Pause & Try

An agent has three actions with Q-values A = 2, B = 6 and C = 4, and uses ε = 0.2. Which action does it choose when it exploits? Out of 100 decisions, about how many times would you expect it to choose each action?

2.3 INTRODUCTION TO REINFORCEMENT LEARNING (RL)

Q-learning

So far the agent has simply *had* Q-values. **Q-learning**, introduced by Chris Watkins in 1989, is one of the most famous RL algorithms for learning them from experience. It is **model-free**: the agent does not need to know in advance how its world works — where each action leads or what it pays. It learns purely by doing. Model-free does not mean memory-free: the agent stores what it learns in a **Q-table**.

The Q-table

Each cell holds **Q(s, a)**: the agent's estimate of the return from taking action a in state s and then acting well. "Q" stands for the *quality* of that action. Here are three states of the grid world after training (rewards simply added up, with no discounting):

Action	A1	C2	C3
up	6	**9**	9
down	5	7	8
left	5	8	8
right	6	−10	**10**

Choosing from the table

To act, the agent looks up the column for its state and picks the action with the **highest Q-value** — that is its policy. On C2 it moves up (9) and never right, into the trap (−10). On C3 it moves right, into the goal (+10).

The highest Q-value in each column is also the **value** of that state: V(C2) = 9, as on the value grid.

From experience to better choices

Experience one step (s, a, r, s') → **Update** adjust one cell of the table → **Better estimates** the table grows more accurate → **Better choices** the policy picks the highest Q

At the start every Q-value is 0, so the agent's choices are random. Each step of experience updates **one cell** of the table. Over thousands of steps the numbers settle towards the values above, and the agent's behaviour improves with them. The next page shows exactly how one cell is updated.

Pause & Try

Using the Q-table, state the action the rover chooses on A1, C2 and C3 when it exploits. Explain why right on C2 has such a low Q-value, and why up on C3 is worth 9.

2.3 INTRODUCTION TO REINFORCEMENT LEARNING (RL)

The Q-learning update rule

After every step, Q-learning nudges **one** Q-value — the one for the state it was in and the action it took — towards a better estimate. It uses this update rule:

The update rule

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right]$$

The arrow ← means "is replaced by": the old value on the right is used to calculate the new value on the left.

1 · Target $r + \gamma \max_{a'} Q(s', a')$ → **2 · Error** target − $Q(s, a)$ → **3 · Update** move a fraction α of the way to the target

The **target** is what this one experience suggests the action is worth: the reward just received plus the discounted value of acting well from the next state. The **error** is how far the old estimate was from the target. If the error is positive, the Q-value rises; if it is negative, the Q-value falls.

What the symbols mean

- **s** — the current state
- **a** — the action the agent took in s
- **r** — the reward it received for that action
- **s′** — the next state it landed in
- **a′** — any *possible* action in s′. It is only considered, not taken: $\max_{a'} Q(s', a')$ is the best Q-value on offer in s′.

The two settings

- **α (alpha), learning rate**, 0 to 1: how much one experience changes the estimate. α = 0 means never learning; α = 1 means trusting only the newest experience.
- **γ (gamma), discount factor**, 0 to 1: how much future rewards count. Near 0 the agent is short-sighted — with γ = 0 the target is just r. Near 1 it is far-sighted.

Pause & Try

Two agents learn the same task, one with γ = 0.1 and one with γ = 0.95. Which is more likely to accept a small penalty now in order to reach a large reward later? Explain your answer using the target $r + \gamma \max_{a'} Q(s', a')$.

2.3 INTRODUCTION TO REINFORCEMENT LEARNING (RL)

A Q-value update, worked through

The update rule becomes clear once you put numbers into it. Every update follows the same three steps: work out the **target**, find the **error**, then move the old value a fraction **α** of the way towards the target.

What is a terminal state?

A **terminal state** is a state in which the episode ends: the rover reaches the charger or falls into the trap, or a game is won or lost. Nothing can happen after it, so there are no future rewards to add — the term $\gamma \max_{a'} Q(s', a')$ is taken as **0**, and the target is simply the final reward r.

Example 1: an ordinary step

The agent takes action a in state s and lands in s′, which is **not** terminal.

Quantity	Value
old Q(s, a)	2
α and γ	0.5 and 0.9
reward r	+1
best Q-value in s′	4
1 · target	1 + 0.9 × 4 = 4.6
2 · error	4.6 − 2 = 2.6
3 · new Q(s, a)	2 + 0.5 × 2.6 = **3.3**

Example 2: penalties into a terminal state

Each time the rover on C2 moves right, it falls into the trap: r = −10 and the episode ends, so the target is just −10. With α = 0.5 and a starting value of 0:

Update	New Q(C2, right)
1st	0 + 0.5 × (−10 − 0) = **−5**
2nd	−5 + 0.5 × (−10 + 5) = **−7.5**
3rd	−7.5 + 0.5 × (−10 + 7.5) = **−8.75**

Each penalty pulls the value further towards −10, so the action is chosen less and less — the Q-table's version of punishment.

Watch the tutorial: Basic reinforcement learning (Video 01)

https://github.com/user-github-me/cs-ai-project-01/blob/main/assets/tutorials/stage2/videos/BlueImpulse_Video01_Basic_Reinforcement_Learning.mp4

Pause & Try

Q(s, a) = 4, α = 0.2, γ = 0.9, r = −1, and the best Q-value in the next state is 10. Calculate the new Q(s, a). Then calculate it again assuming s′ is a terminal state, and explain why the two answers differ.

2.4 MARKOV DECISION PROCESSES (MDPS) MADE SIMPLE

Why we need MDPs

Real decisions rarely come one at a time. A robot makes a **sequence** of decisions, each changing the situation for the next — and its actions do not always turn out as intended: wheels slip, wind blows, people get in the way. A **Markov decision process (MDP)** is a mathematical framework for describing exactly this kind of problem: **sequential decision-making when actions can lead to different possible outcomes**. Written as an MDP, a problem can be reasoned about precisely and solved with algorithms.

In Snakes and Ladders, where your counter lands depends on the dice as well as on the board.

The board-game analogy

Think of an MDP as a board game such as Snakes and Ladders or a video-game maze:

- the square you stand on is your **state**;
- the move you choose, such as walking forward or jumping, is your **action**;
- a dice roll, or slipping on an icy square, means a move does not always land where you intended: **probability**;
- landing on a gold coin wins points and falling into a trap loses them: **reward** and **penalty**.

In real Snakes and Ladders the dice decide everything; an MDP adds **choices**, as if you could pick which die to throw.

How MDPs and RL fit together

The MDP **describes the problem**: its states, actions, possible outcomes and rewards. RL is a way of **solving** it — finding a good policy by trial and error, even when the agent does not know the probabilities and rewards in advance. That is why RL problems are almost always written as MDPs. Inside the framework each part has a clear role: the **environment** is described by the MDP and decides what happens after each action; the **agent** observes the state and makes the decisions; the **policy** is the agent's rule for which action to take in each state.

Pause & Try

Describe a game you know, such as Ludo, Monopoly or a maze video game, as an MDP. Name the states, two actions, where the uncertainty comes from and one reward or penalty.

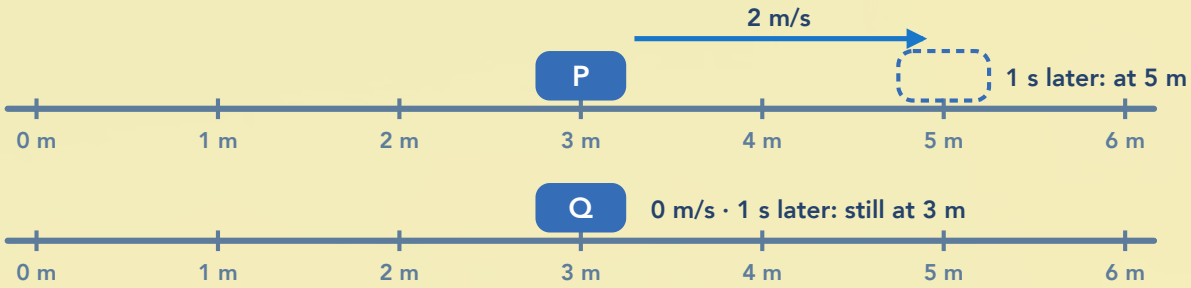
2.4 MARKOV DECISION PROCESSES (MDPS) MADE SIMPLE

The Markov property

An environment has the **Markov property** if what happens next depends only on the **current state** and the **action** taken now — not on how the agent got there. The present state holds all the information that matters, so the Markov property is often called **memoryless**. In chess, for example, you choose your next move from the current position of the pieces (and whose turn it is); whether a knight arrived on its square five turns ago or twenty makes no difference.

Position alone is not enough

Robots P and Q both stand at the 3 m mark of a track. If the state records only their **position**, they look identical — yet one second later they are in different places.



The same "state" leads to different futures, so a position-only state does **not** have the Markov property: to predict what happens next, you would need the history (where was each robot a second ago?). Add **speed** to the state — (3 m, 2 m/s) and (3 m, 0 m/s) — and the next position can be predicted from the present alone. The Markov property now holds.

Designing a good state

For the Markov property to hold, the state must contain **enough information to describe the situation**. For a delivery robot that might be its **position**, **speed**, **battery level** and whether it is **carrying a parcel**: the battery level decides whether it can reach the next stop, and the parcel decides where it should go next. Real sensors can be noisy or incomplete — a camera cannot see behind a box — so engineers often combine several readings over time to estimate a state that is as close to Markov as possible.

Pause & Try

A self-driving car's state records only which lane it is in. Explain why this state does not have the Markov property, and suggest two pieces of information to add to it.

2.4 MARKOV DECISION PROCESSES (MDPS) MADE SIMPLE

The five ingredients of an MDP

Every MDP is built from five ingredients, written together as the tuple **(S, A, P, R, γ)**. Here they are for the rover's grid world from 2.3 — but now the floor is **icy**, so the rover does not always go where it intends.

The icy grid world as an MDP

Ingredient	Meaning	In the icy grid world
S: states	every situation the agent can be in	the 11 squares A1 to D3 (the wall is not a state)
A: actions	every choice the agent can make	up, down, left, right
P(s' | s, a): transition probabilities	the chance that action a in state s leads to state s'	0.8 to go where intended; 0.1 to slip to each side
R(s, a): reward function	the number earned for taking action a in state s	+10 into the goal, −10 into the trap, −1 otherwise
γ: discount factor	how much future rewards count, from 0 to 1	γ = 0.9

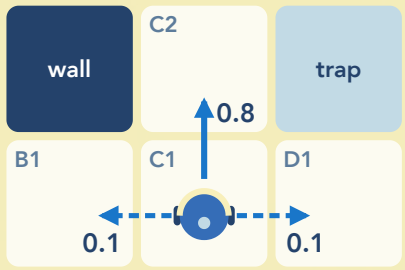
Reading P for one action

The rover is on C1 and chooses **up**:

- P(C2 | C1, up) = 0.8: it moves up as intended;
- P(B1 | C1, up) = 0.1: it slips to the left;
- P(D1 | C1, up) = 0.1: it slips to the right.

The three probabilities add up to 1, because the rover must end up somewhere. The **agent chooses the action**; the **environment, through P, chooses the outcome**.

One action, three outcomes



Reading γ. With γ = 0.9, each step into the future multiplies a reward by 0.9. A +10 received on the rover's next move counts as 10; one move later it counts as 0.9 × 10 = 9; one move after that, 0.9 × 0.9 × 10 = 8.1. Discounting makes the agent prefer reaching the goal sooner rather than later.

Pause & Try

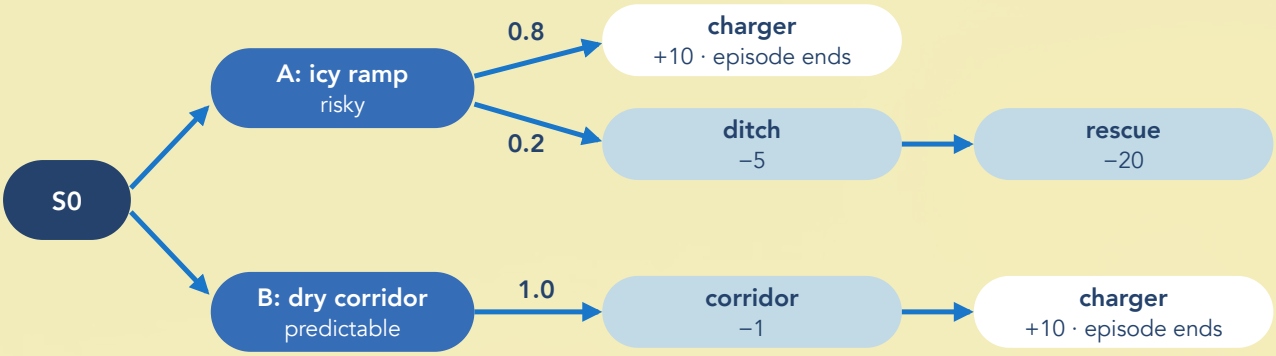
A delivery drone flies between rooftops on a windy day. Suggest one example of each ingredient — S, A, P, R and γ — for this problem.

2.4 MARKOV DECISION PROCESSES (MDPS) MADE SIMPLE

Risky or predictable?

Transition probabilities let an agent **compare actions whose outcomes are uncertain**. Picture the rover at a junction, S0, with two ways to reach its charger: a short ramp that is icy, or a dry corridor that takes one extra move.

Two actions from S0



	A: icy ramp (risky)	B: dry corridor (predictable)
Transitions, P	P(charger | S0, A) = 0.8; P(ditch | S0, A) = 0.2	P(corridor | S0, B) = 1.0
Expected immediate reward	0.8 × 10 + 0.2 × (−5) = **7**	1.0 × (−1) = **−1**
Expected total reward	0.8 × 10 + 0.2 × (−5 − 20) = **3**	−1 + 10 = **9**

Judged only by the **immediate** reward, the risky ramp looks much better: 7 against −1. Once the **future** is included — the rescue after a slip, the +10 after the corridor — the predictable corridor wins: 9 against 3. The best action depends on future rewards, not only on the next one, which is why RL agents learn *values*. (To keep the arithmetic simple, future rewards are not discounted here: $\gamma = 1$.)

Expected is not guaranteed

The expected immediate reward of the icy ramp is 7, yet no single attempt ever earns 7: each attempt earns either +10 or −5. Seven is the **long-run average**: over 100 attempts, about 80 earn +10 and about 20 earn −5, a total of 800 − 100 = 700, or 7 per attempt.

Pause & Try

An agent has a 70% probability of receiving +10 and a 30% probability of receiving −4. Calculate the expected immediate reward and explain why the actual reward may differ.

2.4 MARKOV DECISION PROCESSES (MDPS) MADE SIMPLE

Learning, then working

RL and MDPs let an AI system learn to make decisions. But what happens once the learning is done? Most systems have two separate phases.

Training

The agent explores, collects experience (s, a, r, s') and updates its Q-values or policy. Training often happens in a **simulation**, where mistakes are cheap, and may take millions of steps.

Deployment

The trained policy is put to work. Learning is usually switched off (no more exploring: $\varepsilon = 0$). The robot still **reacts** to new sensor readings, but its **policy stays the same**.

Moving to an unfamiliar environment

A rover trained to deliver parcels in Warehouse 1 is moved to Warehouse 2.

- **Does its policy still work?** Only as far as the new place resembles the old one. A Q-table for Warehouse 1's squares says nothing about Warehouse 2's layout: the **states (S)** have changed.
- **Does it need more training?** Usually — ideally building on what it already knows, often in a simulation of the new layout first.
- **Can it adapt while working?** With **online learning** it keeps updating its Q-values on the job, with a small ε and strict safety rules.
- **What if the reward assumptions no longer apply?** On slippery floors the **transition probabilities (P)** change, and a reward for speed alone would push the robot to race across them. Designers must change the **reward function (R)**, for example by adding a penalty for skidding.

Because an RL agent learns from interaction, it can **re-learn** when its world changes — and the MDP tells us exactly *what* changed: S, A, P or R.

Pause & Try

A robot trained to sort parcels on a dry, flat factory floor is moved to a factory with a sloping, wet floor. Which MDP ingredients have changed? Describe two ways the robot could adapt, and one risk of letting it learn while it works.

2.4 MARKOV DECISION PROCESSES (MDPS) MADE SIMPLE

Three ideas, not one

Behaviour-based robotics, reinforcement learning and neural networks often appear together, but they are **related ideas, not interchangeable labels**: each answers a different question.

What each idea is

Idea	What it is	The question it answers
Behaviour-based robotics	an approach to robot **control and architecture**	How should a robot's behaviours be organised and linked to sensors and motors?
Reinforcement learning	a **learning framework**	How can an agent learn from rewards which actions to take?
Neural network	a **computational model** of a function, such as a policy	How can a computer represent a complex link from inputs to outputs?

Why neural networks join in

The grid world's Q-table has just 44 cells (11 states × 4 actions). A robot that sees through a camera can be in millions of states — far too many for a table. A **neural network** can stand in for the table: it takes the state as input and outputs a Q-value for each action. This is **deep reinforcement learning**; in 2015, DeepMind's DQN agent used it to learn 49 Atari games from screen pixels.

One, two or all three

System	Behaviour-based	RL	Neural network
early robot vacuum: bump and turn	yes	no	no
grid-world rover with a Q-table	no	yes	no
phone face unlock	no	no	yes
walking robot: deep RL plus a safety stop	yes	yes	yes

Pause & Try

A warehouse robot uses a neural network to recognise parcels, Q-learning to choose routes and a bumper rule that stops it on contact. Match each part to one of the three ideas, and explain why.

CHAPTER 2 · PRACTICE

Practice

Practice · Chapter 2

1. Behaviourism rests on one central principle.
   a. State the central principle of behaviourism. [1]
   b. Using an example for each, explain how behaviour can be learned, modified and maintained. [3]

2. Differentiate between classical conditioning (Pavlov) and operant conditioning (Skinner) in terms of how learning occurs and how they apply to artificial intelligence. [4]

3. Define Behaviourist AI and explain how it relates to the principles of behaviourism. State one way in which it differs from behaviourism in psychology. [3]

4. Autonomous reflex-based agents interact directly with their environment without storing complex internal 3D world representations. Consider a smart home electronic thermostat or a simple robotic lawnmower. Clearly identify its:
   a. primary stimulus [1]
   b. associated response [1]
   c. operating environment [1]

5. Describe how Rodney Brooks' subsumption architecture organises robot behaviours into layered modules without using a centralised internal map. Explain why this model is more responsive than traditional map-building AI. [4]

6. Explain the difference between reactive control and learned behaviour in a robot. Give one example of each. [4]

7. A rover needs four moves to reach its charger. Every move earns −1, except the final move into the charger, which earns +10.
   a. State the reward for the rover's first move. [1]
   b. Calculate the return from the start if the rover takes this shortest route. [1]
   c. Explain why the value of the start square is positive even though the first move earns a negative reward. [2]

8. Markov decision processes (MDPs) formalise sequential decision-making under environment uncertainty.
   a. Define the roles of the agent, the environment and the policy (π) within an MDP framework. [3]
   b. Explain the difference between exploration (trying new actions) and exploitation (repeating known high-reward actions). Why must an RL agent maintain a balance during training? [4]

CHAPTER 2 · PRACTICE

Practice

Practice · Chapter 2 (continued)

9. An agent uses an ε-greedy strategy with ε = 0.1. In its current state, Q(s, left) = 4, Q(s, right) = 7 and Q(s, stay) = 1.
   a. State which action it chooses when it exploits, and describe what happens when it explores. [2]
   b. Explain why the agent's preferred action might change later in training. [2]

10. In Q-learning, updates are driven by feedback signals and discounted future estimates.
    a. Explain the purpose of the discount factor (γ) when calculating optimal state–action values. What occurs if γ is set to 0? [2]
    b. Explain how negative rewards (penalties) alter an agent's Q-table values over multiple learning iterations. [2]
    c. An agent has Q(s, a) = 5. It takes action a, receives a reward of +2 and lands in state s′, where the highest Q-value is 8. Using α = 0.5 and γ = 0.9, calculate the new value of Q(s, a). Show your working. [3]

11. A cleaning robot moves between the rooms of a house.
    a. Describe the five ingredients of an MDP (S, A, P, R, γ), with an example of each for this robot. [5]
    b. The robot's state records only which room it is in. Explain, with an example, why this state might not have the Markov property. [2]

12. Action X gives +6 with probability 0.5 and +2 with probability 0.5. Action Y gives +12 with probability 0.4 and −3 with probability 0.6.
    a. Calculate the expected immediate reward of each action. [2]
    b. Explain why the action with the lower expected immediate reward could still be the better choice. [2]

13. A robot trained in one warehouse is moved to a new warehouse with a different layout. Discuss whether its learned policy will still work and how it could adapt. [4]

14. Explain the difference between behaviour-based robotics, reinforcement learning and neural networks. Describe one system that combines at least two of them. [4]

MIDDAY

HARDWARE

3

CHAPTER 3 · BEFORE YOU BEGIN

Learning goals

Hardware is the physical side of computing: the devices that sense the world, the memory that holds data, the processors that do the work and the tiny logic circuits they are built from. In this chapter you will follow data from input devices and sensors, through memory and the CPU, to output devices and actuators, and then look inside the processor at pipelines, parallel processing and logic gates. The chapter ends by showing how all of this hardware makes artificial intelligence possible.

Learning goals

You will learn about:

- Differences between RAM, ROM, SRAM, DRAM, PROM and EPROM
- The use of RAM, ROM, SRAM and DRAM in a range of devices
- Monitoring and control systems
- Sensors and actuators
- The difference between RISC and CISC processors
- The importance of pipelining and registers in RISC processors
- SISD, SIMD, MISD, MIMD basic computer architecture
- Characteristics of massively parallel computers
- Interrupt handling on CISC and RISC computers
- Basics of logic circuits
- Truth tables in logic circuits
- Simplification of logic circuits and expressions using Boolean algebra

How this chapter is organised

3.1 Input, processing and output hardware → **3.2** Computer memory and storage →

3.3 Processing devices and CPU architecture → **3.4** Parallel processing and computer systems →

3.5 Logic circuits and Boolean algebra → **3.6** How hardware enables AI

CHAPTER 3 · BEFORE YOU BEGIN

Key terms

Key terms

Hardware: The physical parts of a computer system that you can see and touch.

Input device: Captures data or instructions and sends them into a computer.

Output device: Presents results as images, sound, print or movement.

Processing device: Carries out instructions and calculations, e.g. a CPU.

Screen resolution: Pixels across × pixels down.

Organic LED (OLED): A display whose pixels give out their own light; no backlight is needed.

Touchscreen: A display that also senses touch.

Capacitive touchscreen: A finger changes the electric field on a charged glass layer.

Resistive touchscreen: Pressure pushes a plastic layer onto glass, completing a circuit.

Virtual reality (VR) headset: Goggles with a screen for each eye and motion sensors.

Direct 3D printing: A print head moving in x, y and z builds layers of melted material.

Binder 3D printing: Spreads a layer of dry powder, then sprays a binding agent onto it.

Sensor: An input device that measures a physical quantity, e.g. temperature.

Actuator: An output device that turns a signal into physical action, e.g. movement.

Analogue-to-digital converter (ADC): Turns an analogue signal into binary numbers.

Digital-to-analogue converter (DAC): Turns digital values into an analogue signal.

Monitoring system: Observes and records conditions without changing them.

Control system: Uses sensor data to drive actuators that change the conditions.

Feedback: The result of an action is measured and used to decide the next action.

Embedded system: A computer built into a larger device to do one dedicated job.

Microcontroller: One chip with a processor, memory and input/output pins.

Primary storage: Memory the CPU accesses directly: RAM, ROM and cache.

Secondary storage: Non-volatile, long-term storage that the CPU cannot access directly.

Volatile memory: Loses its data without power.

Non-volatile memory: Keeps its data without power.

Random access memory (RAM): Volatile primary memory that can be read and written.

Read-only memory (ROM): Non-volatile primary memory that is read but not normally changed.

Static RAM (SRAM): Stores bits in flip-flops; needs no refreshing; used for cache.

Dynamic RAM (DRAM): Stores bits as charge on capacitors; must be constantly refreshed.

Refreshed: Recharged regularly to keep its data.

Cache memory: Small, fast SRAM near the processor, holding data it will need again.

Programmable ROM (PROM): ROM bought blank that can be programmed only once.

Erasable PROM (EPROM): ROM that can be erased with ultraviolet light and reprogrammed.

Electrically erasable PROM (EEPROM): ROM erased and rewritten electrically, inside the circuit.

Flash memory: EEPROM erased in blocks; used in SSDs, memory cards and USB sticks.

Hard disk drive (HDD): Magnetic storage with spinning platters and moving read-write heads.

Latency: The lag in a system, e.g. waiting for a disk sector to reach the read-write head.

Fragmented: Stored in non-consecutive sectors, e.g. after editing and deleting data.

Removable hard disk drive: A portable HDD plugged in by USB, often used for back-ups.

Solid-state drive (SSD): Storage with no moving parts that keeps data in flash memory.

CHAPTER 3 · BEFORE YOU BEGIN

Key terms

Key terms (continued)

Optical storage: CDs, DVDs and Blu-ray discs, read and written with laser light.

Dual layering: Two recording layers on one disc, nearly doubling its capacity.

Birefringence: A DVD reading problem: the laser light is refracted into two beams.

Central processing unit (CPU): The main processor; it fetches, decodes and executes instructions.

Control unit (CU): Decodes instructions and sends control signals to the other components.

Arithmetic logic unit (ALU): Carries out arithmetic and logic operations.

Register: A tiny, very fast storage location inside the CPU.

Program counter (PC): Holds the address of the next instruction to be fetched.

Memory address register (MAR): Holds the address of the memory location being read or written.

Memory data register (MDR): Holds data just read from memory, or about to be written to it.

Current instruction register (CIR): Holds the instruction being decoded and executed.

Accumulator (ACC): Holds the value being processed and the results from the ALU.

Address bus: Carries addresses from the CPU to memory and devices; one-way.

Data bus: Carries data and instructions between components; two-way.

Control bus: Carries signals such as read, write and interrupt requests; two-way.

CISC: Complex instruction set computer: many complex, variable-length instructions.

RISC: Reduced instruction set computer: few, simple, fixed-length instructions.

Pipelining: Overlapping the stages of several instructions to process them at the same time.

Hazard: Anything that stops the next instruction entering the pipeline on time.

Interrupt: A signal that asks the CPU to stop and deal with an event.

Interrupt service routine (ISR): The code the CPU runs to deal with an interrupt.

Parallel processing: Splitting a task so that different processors execute its parts at the same time.

SISD: Single instruction, single data: one processor, one data item at a time.

SIMD: Single instruction, multiple data: one instruction is applied to many data items at once.

MISD: Multiple instruction, single data: several processors work on the same data.

MIMD: Multiple instruction, multiple data: processors each run their own instructions on their own data.

Cluster: Several computers networked together to work as one resource.

Supercomputer: An extremely powerful computer with many processors, built for huge scientific calculations.

Massively parallel computer: Thousands of processors linked together to work as one machine.

Logic gate: A circuit that gives one binary output from one or more binary inputs.

Truth table: Shows the output for every possible combination of inputs.

Logic circuit: Logic gates connected so that the outputs of some become inputs of others.

Boolean algebra: The algebra of 1s and 0s, used to write and simplify logic expressions.

AI hardware: Processors, memory and accelerators that run AI tasks.

Graphics processing unit (GPU): A processor whose thousands of small cores work in parallel.

Video RAM (VRAM): Fast memory on a graphics card that holds the data the GPU is working on.

AI accelerator (NPU, TPU): A chip designed to speed up the maths of neural networks.

Memory bandwidth: How much data can move between memory and a processor each second.

Edge AI: Running an AI model on the device itself instead of in the cloud.

3.1 INPUT, PROCESSING AND OUTPUT HARDWARE

What is hardware?

Hardware is every physical part of a computer system: the parts you can see and touch, from the chips on a circuit board to the keyboard, the screen and the cables in between. **Software** is the set of programs that tells the hardware what to do. Each needs the other. A laptop without software is a box of silent electronics, and a program without hardware has nothing to run on.

Every computer, from a games console to the controller in a washing machine, uses its hardware for the same four jobs: it takes data **in**, **processes** it, **stores** it and sends results **out**.

Input
keyboard, microphone, sensor
→
Processing
the CPU follows the program
→
Output
screen, speaker, motor

The four jobs of hardware

- **Input devices** capture data from people or the world: keyboard, camera, temperature sensor.
- **Processing devices** carry out instructions and calculations: the CPU, a graphics processor (GPU), a microcontroller.
- **Storage devices** keep programs and data: RAM while you work, an SSD for the long term.
- **Output devices** present results or act on them: monitor, headphones, printer, motor.

One device, many jobs

A smartphone contains all four kinds:

- touchscreen: **input** and **output**
- camera, microphone, GPS: **input**
- processor chip: **processing**
- flash memory: **storage**
- speaker, vibration motor: **output**

Hardware is also either **internal**, inside the case (motherboard, processor, memory, power supply), or a **peripheral** connected from outside (mouse, printer, external drive) through ports such as USB and HDMI, which you met in Chapter 1.

The hardware also sets the limits of what a computer can do: the processor decides how fast it works, the memory how much it can work on at once, and the storage how much it can keep. Better hardware runs the same program faster, but no hardware can do anything useful without a program to follow.

Pause & Try

A smart speaker contains a microphone, a processor, flash memory, a loudspeaker and a ring of lights. Classify each part as input, processing, storage or output. Then explain why the speaker needs both hardware and software to answer a spoken question.

3.1 INPUT, PROCESSING AND OUTPUT HARDWARE

Input and Output

An **input device** turns something from the outside world (a key press, a movement, a sound, light or a printed code) into digital data that the processor can use. An **output device** does the opposite: it turns digital data back into something people can see, hear or feel, or into a physical action. Look at a typical desk set-up and you can spot both kinds. Touchscreens and sensors are input devices too; you will meet them over the next pages.

In: keyboard, mouse and microphone. Out: monitor, speakers and headphones.

What every input device does

Detect
a switch, light sensor or diaphragm reacts

↓

Convert
the reaction becomes an electrical signal

↓

Digitise and send
binary data travels to the computer, e.g. over USB

Common input devices

Device	Captures	How it becomes data
Keyboard	key presses	each key closes a switch; a chip sends that key's code
Mouse	movement, clicks	a light and a tiny camera track the desk surface to measure movement
Microphone	sound	a thin diaphragm vibrates, making a changing voltage that an ADC samples
Camera	light	millions of light-sensitive pixels each measure the light falling on them
Barcode reader	printed codes	dark bars reflect less light; the pattern is decoded into a number

Pause & Try

At a supermarket self-checkout you scan each item, place it on a weighing scale, tap the screen and pay with a contactless card. List the input devices you use and say what data each one captures.

3.1 INPUT, PROCESSING AND OUTPUT HARDWARE

Output devices

An **output device** converts the computer's digital results into light, sound, print or motion. The most common ones present information to people.

Screens

A screen is a grid of **pixels**; each pixel has red, green and blue parts that mix to make any colour. **Screen resolution** is the number of pixels across and down: a 1920 × 1080 (Full HD) screen has 1920 × 1080 = **2 073 600** pixels.

- In an **LCD** screen, each pixel is a liquid-crystal cell that blocks or lets through light from a white **backlight** behind it. A little light always leaks through, so black looks dark grey.
- In an **OLED** (organic LED) screen, each pixel **gives out its own light**. No backlight is needed, so the screen can be thinner, even flexible, and a black pixel is simply switched off: true black.

Speakers and headphones

Sound is stored as numbers, but a speaker needs a smoothly changing electric current:

Digital sound numbers → **DAC** to analogue → **Amplifier** stronger signal → **Speaker** coil moves a cone

The moving cone pushes the air back and forth, creating sound waves. **Headphones** work in the same way, with tiny speakers next to your ears so only you hear the sound.

Printers

- An **inkjet printer** fires tiny droplets of ink through nozzles onto the paper.
- A **laser printer** uses a laser to draw the page as a pattern of static charge on a drum; powdered **toner** sticks to the charged areas and is melted onto the paper.
- A **3D printer** builds a solid object layer by layer (next page).

Pause & Try

A 4K screen has a resolution of 3840 × 2160. Calculate how many pixels it has, and how many times more that is than a 1920 × 1080 screen. Then explain why an OLED screen can show a deeper black than an LCD.

3.1 INPUT, PROCESSING AND OUTPUT HARDWARE

Touchscreens, VR and 3D printing

Some devices do two jobs at once. A **touchscreen** is an output device (the display) and an input device (the touch sensor) in one. There are two main technologies.

Capacitive or resistive?

	Capacitive	Resistive
How it detects a touch	a glass layer holds an electric charge; your finger changes the electric field at that point	pressing pushes a flexible plastic layer onto a glass layer, so their coatings touch and complete a circuit
What can touch it	a bare finger or a special stylus; supports multi-touch	anything, even a gloved finger or a pen; usually one touch at a time
Clarity and durability	very clear, tough glass	less clear; the plastic scratches
Typical use	phones, tablets	factory controls, some medical equipment

VR headsets

A **virtual reality headset** has a small screen for each eye, showing slightly different images so the scene looks 3D. Motion sensors (a gyroscope and an accelerometer) track your head, and the picture changes as you look around. The headset is **output** (screens, sound) and **input** (head and hand movements) at the same time.

3D printers

A 3D printer builds a solid object **layer by layer** from a 3D design file.

- **Direct 3D printing:** a print head moves in the x, y and z directions, pushing melted material out through nozzles to build each layer.
- **Binder 3D printing:** two stages for every layer: spread a thin layer of dry powder, then spray a liquid **binding agent** where the object should be solid. At the end the loose powder is brushed away.

Pause & Try

A hospital wants touchscreens that nurses can use while wearing gloves. Which touchscreen technology should it choose? Give one advantage and one disadvantage of your choice.

3.1 INPUT, PROCESSING AND OUTPUT HARDWARE

Sensors

A **sensor** is an input device that measures a physical quantity, such as temperature, light, pressure or distance, and turns it into an electrical signal. Sensors are how a phone knows to dim its screen in the dark, or a car knows there is a wall behind it.

Example: an ultrasonic distance sensor

This sensor sends out a burst of sound at 40 kHz, too high for humans to hear, and times how long the echo takes to come back. The sound travels to the object **and back**, so distance = speed of sound × time ÷ 2.

Worked example: the echo returns after 5.8 ms. At 343 m/s, distance = 343 × 0.0058 ÷ 2 ≈ **0.99 m**.

T sends; R hears the echo.

Common sensors

Sensor	Example use
Temperature	central heating, greenhouses
Light	street lights, screen brightness
Pressure	burglar-alarm mats, tyres
Moisture	greenhouses, irrigation
Infrared (PIR): body heat	security lights, automatic doors

From sensor to binary: the ADC

Most sensors produce an **analogue** signal: a voltage that can take any value in a range. An **analogue-to-digital converter (ADC)** measures it many times a second and turns each measurement into a binary number. An 8-bit ADC splits 0–5 V into 256 levels (0 to 255), so 3.0 V becomes 3.0 ÷ 5 × 255 = **153**, or 1001 1001. Some sensors have a built-in ADC.

Pause & Try

A car's parking sensor hears the echo of its pulse 2.9 ms after sending it. Using 343 m/s for the speed of sound, calculate the distance to the wall. Then explain why an ADC is needed between a temperature sensor and a processor.

3.1 INPUT, PROCESSING AND OUTPUT HARDWARE

Actuators, monitoring and control

An **actuator** is an output device that turns an electrical signal into physical action. **Motors** spin fans, wheels and pumps; **solenoids** push or pull a rod to open a valve or a door lock; **relays** let a small signal switch a large current, for example to a heater. The processor's own signal is far too weak to move anything: a **digital-to-analogue converter (DAC)** turns its digital values into a varying voltage (for example to set a motor's speed), and a driver circuit or relay supplies the larger current.

Monitoring system

Collects sensor readings and **displays, records or warns**, but does not change the conditions itself.

- patient monitor in a hospital
- weather station
- burglar alarm

Control system

Uses sensor readings to **decide and act**, driving actuators that change the conditions.

- central heating
- greenhouse
- traffic lights

The feedback loop

Sensor reads 20 °C → **ADC** to binary → **Processor** checks rules → **Actuator** heater on → **Greenhouse** warms up ↻

Worked example: a greenhouse controller

The program follows two rules: **if** the temperature is below 22 °C, switch the heater on; **if** it is above 27 °C, open the roof window. The loop repeats every few seconds, and each new reading is **feedback**: it shows the result of the last action.

Reading	Heater	Roof window
20 °C	on	closed
24 °C	off	closed
29 °C	off	opened by a motor

Pause & Try

Outline a simple scenario for an automated environmental control system, such as a greenhouse or a smart thermostat. Specify the sensor used, the processing condition and the resulting actuator action.

3.1 INPUT, PROCESSING AND OUTPUT HARDWARE

Embedded systems

An **embedded system** is a computer built into a larger device to do one dedicated job. You use dozens every day without noticing them: in washing machines, microwave ovens, digital cameras, smart thermostats, fitness bands and a car's engine and brakes. Unlike a laptop, an embedded system runs a single program, often for years, and you cannot install new software on it.

Most embedded systems are built around a **microcontroller**: a single chip that contains a whole small computer.

Sensors
door, water level, heat
→
Microcontroller
CPU, RAM, ROM, I/O pins
→
Actuators
valves, heater, motor

Inside a washing machine

Fill
a valve opens until the water-level sensor reads full

↓

Heat
the heater stays on until the water reaches 40 °C

↓

Wash
the motor turns the drum back and forth

↓

Drain and spin
the pump empties the drum, then it spins fast

The control program lives in the chip's **ROM or flash**, so it is ready the moment the power comes on. A little **SRAM** holds the chosen programme, the timer and the latest sensor readings.

Benefits and drawbacks

Benefits

- small, light and cheap to mass-produce
- uses very little power
- starts instantly and responds in real time
- reliable, because it does one job

Drawbacks

- hard or impossible to upgrade
- limited memory and processing power
- a fault often means replacing the whole board
- "smart", connected devices can be hacked

Pause & Try

Explain why a microwave oven uses an embedded microcontroller instead of a general-purpose computer. Give two benefits and one drawback.

3.2 COMPUTER MEMORY AND STORAGE

Primary and secondary storage

A computer keeps two kinds of things: the program and data it is working on **right now**, and everything it might need **later**. No single technology is fast, big and cheap enough to do both jobs, so computers use two kinds of storage.

Primary storage

Primary storage is memory the CPU can access **directly**: RAM, ROM and cache. It is very fast but small and expensive per byte, and most of it (RAM and cache) is **volatile**: it loses its contents when the power is switched off.

Secondary storage

Secondary storage keeps programs and files when the power is off: it is **non-volatile**. Hard disks, SSDs, USB flash drives, memory cards and optical discs are all secondary storage. The CPU **cannot access it directly**. When you open a game, its program is first copied from the SSD into RAM, and only then can the CPU run it.

Primary and secondary storage compared

	Primary storage	Secondary storage
Examples	RAM, ROM, cache	HDD, SSD, flash drive, DVD
Volatile?	RAM and cache: yes; ROM: no	no
Access time	nanoseconds	microseconds to milliseconds
CPU access	direct	only after copying into RAM
Capacity and cost	gigabytes; expensive per GB	terabytes; cheap per GB

The memory hierarchy

Registers under 1 ns → **Cache** about 1–10 ns → **RAM** about 100 ns → **SSD** about 0.1 ms → **Hard disk** about 10 ms

The closer a memory is to the processor, the faster, smaller and more expensive it is. Reading from RAM (about 100 ns) is roughly **100 000 times** faster than waiting for a hard disk (about 10 ms).

Pause & Try

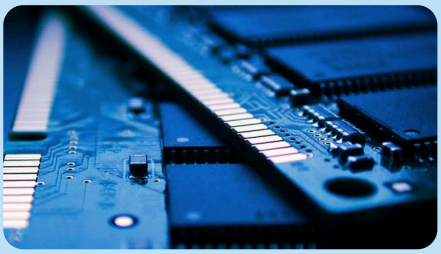
Differentiate between primary storage and secondary storage in terms of volatility, speed and CPU accessibility.

3.2 COMPUTER MEMORY AND STORAGE

RAM: SRAM and DRAM

What RAM does

Random access memory (RAM) is the computer's working memory: it holds the operating system, the open programs and their data. "Random access" means the CPU can go straight to any location in the same short time. RAM can be **read from and written to**, and it is **volatile**: switch off without saving and your work is gone. A typical laptop has 8–32 GB; if the programs need more, data has to be swapped out to secondary storage, which is far slower.



A RAM module.

Two ways to build RAM

Static RAM (SRAM)

Each bit is held by a **flip-flop**: a tiny circuit of about six transistors that stays in one of two states while it has power. Nothing leaks away, so SRAM **never needs refreshing**.

Dynamic RAM (DRAM)

Each bit is a charge on a tiny **capacitor**, switched by one transistor. The charge leaks away, so every cell must be **refreshed** (read and recharged) about every 64 ms.

SRAM and DRAM compared

	SRAM	DRAM
Stores a bit in	a flip-flop (about 6 transistors)	a capacitor and 1 transistor
Refreshing	not needed	constantly, about every 64 ms
Speed	very fast (a few ns)	slower (tens of ns)
Cost and density	expensive; fewer bits per chip	cheap; many bits per chip
Typical use	CPU cache	main memory

Pause & Try

Explain the structural difference between SRAM and DRAM in how they store data and in their need for refreshing. Which would you use for a CPU's cache, and why?

3.2 COMPUTER MEMORY AND STORAGE

ROM, PROM, EPROM and flash

What ROM does

Read-only memory (ROM) is primary memory that is **non-volatile** and **not normally changed** while the device runs. It stores instructions that must be ready the instant the power comes on, such as the start-up firmware (BIOS or UEFI) that checks the hardware and loads the operating system, or the whole program of a microwave oven.

From fixed to rewritable

Ordinary ROM is fixed for ever when the chip is made. Engineers wanted ROM they could program themselves, and later change, so a family of types developed, from fixed for ever to rewritable inside the circuit.

The ROM family

Type	How data is written	Can it be erased?	Typical use
ROM	built in during manufacture	no	toys, calculators
PROM	bought blank; a programmer burns tiny fuses, once	no: one time only	fixed firmware, small batches
EPROM	electrically, by a programmer	yes: by strong UV light through a quartz window, out of the circuit	developing and testing firmware
EEPROM	electrically	yes: electrically, byte by byte, in the circuit	saved settings
Flash	electrically	yes: electrically, in whole blocks	SSDs, USB sticks, BIOS chips

Flash: the ROM that became storage

Erasing a whole block at once keeps each flash cell small and simple, so flash is cheap and dense enough to be used as storage, from memory cards to SSDs. It also shows that ROM is not always read-only: a PC's start-up firmware sits in a flash chip, so the maker can send an update and you can "flash the BIOS". It is still called ROM because the computer only reads it in normal use.

Pause & Try

Contrast ROM, PROM and EPROM in terms of how data is written and erased.

3.2 COMPUTER MEMORY AND STORAGE

Memory in everyday devices

Almost every device uses the same building blocks in amounts that suit the job. Fast but expensive SRAM is used in small amounts where speed matters most; cheap, dense DRAM provides the large working memory; ROM and flash keep whatever must survive power-off.

Where each type of memory is used

Device	RAM is used for	ROM or flash holds
Laptop or PC	8–32 GB of **DRAM** for the OS and open programs; a few MB of **SRAM** cache in the CPU	**flash ROM**: the BIOS/UEFI start-up firmware
Smartphone	several GB of low-power **DRAM** for apps; **SRAM** cache in the processor	**flash**: the operating system, apps and photos
Washing machine or microwave	a few KB of **SRAM** in the microcontroller for the timer and readings	**ROM or flash**: the fixed control program
Games console	fast **DRAM** shared by the CPU and GPU	**flash**: firmware and games
Wi-Fi router	**DRAM** for data packets passing through	**flash**: firmware, settings

RAM, ROM, SRAM, DRAM, PROM and EPROM at a glance

Memory	Volatile?	Can the contents be changed?	Main use
RAM	yes	yes: read and write	working memory for programs in use
SRAM	yes	yes: no refreshing needed	CPU cache
DRAM	yes	yes: must be refreshed	main memory
ROM	no	no: fixed when made	permanent start-up and device programs
PROM	no	once only, after manufacture	fixed firmware in small batches
EPROM	no	yes: after erasing with UV light	developing and updating firmware

Pause & Try

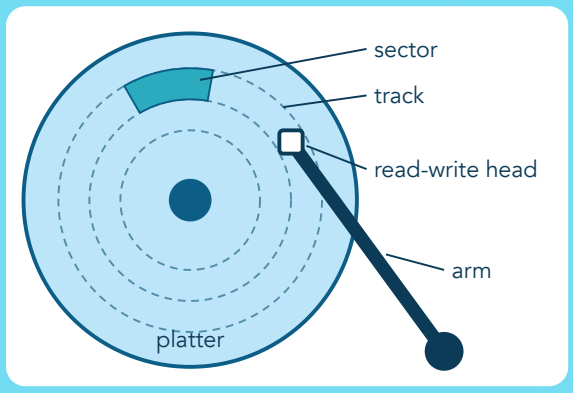
A smart thermostat contains a microcontroller. State which type of memory would store: a) its control program; b) the current temperature reading; c) the user's heating schedule, which must survive a power cut. Explain each choice.

3.2 COMPUTER MEMORY AND STORAGE

Magnetic storage: hard disk drives

A **hard disk drive (HDD)** stores data magnetically on **platters**: rigid discs coated with a magnetic material, spinning at a constant speed such as 7200 revolutions per minute (rpm). Each surface is divided into circular **tracks**, and each track into **sectors**. A **read-write head** on a moving arm floats just above the surface: to write, it magnetises tiny areas in one of two directions (0 or 1); to read, it senses those directions.

Inside a hard disk



Why a hard disk has to wait

To read a sector, the drive must:

1. move the head to the right track (the **seek time**);
2. wait for the sector to spin round under the head (the **rotational latency**);
3. read the data as it passes.

Worked example: at 7200 rpm the platter turns 7200 ÷ 60 = 120 times a second, so one turn takes 1 ÷ 120 s ≈ 8.3 ms. On average the sector is half a turn away, so the latency is about **4.2 ms**: roughly 40 000 times longer than a RAM access.

Fragmentation

When files are edited and deleted, free space is left in scattered gaps, and a new file may be split across **non-consecutive sectors**: it becomes **fragmented**. The head must then jump between tracks to read it, adding seek time. A defragmentation utility (Chapter 1) moves the pieces back together.

Strengths and weaknesses

Hard disks are cheap per terabyte and come in huge capacities, so they suit servers and back-ups. A **removable hard disk** plugs in by USB when it is needed, which makes it a popular back-up device. But the moving parts make HDDs slower, noisier and more power-hungry than SSDs, and a drop can damage the head or platters.

Pause & Try

A hard disk spins at 5400 rpm. Calculate the time for one revolution and the average rotational latency. Explain why a fragmented file takes longer to read.

3.2 COMPUTER MEMORY AND STORAGE

Solid-state and optical storage

Solid-state drives

A **solid-state drive (SSD)** stores data in **flash memory** and has **no moving parts**. Each cell is a transistor that can trap electrons in an insulated "floating gate"; cells with and without trapped electrons stand for 0 and 1, and stay that way without power. With nothing to move or spin, an SSD finds data in about 0.1 ms, roughly 100 times faster than a hard disk. USB sticks and memory cards use the same technology.

SSD strengths

- very fast start-up and file loading
- silent, light and survives being dropped
- uses less power, so batteries last longer

SSD weaknesses

- costs more per terabyte than an HDD
- each block can only be erased a few thousand times, so the drive spreads its writes evenly

Optical storage: CDs, DVDs and Blu-ray discs

An optical disc stores data along a spiral track of microscopic **pits** and flat **lands**. A laser shines on the spinning disc, and the changes in reflected light are read as 1s and 0s.

Dual layering: a second recording layer, which the laser can focus on instead, nearly doubles the capacity. **Birefringence** is a DVD reading problem: the plastic splits the laser into two beams, causing read errors. Blu-ray keeps its data just 0.1 mm below the surface to reduce it.

Data sits on one long spiral track.

A shorter laser wavelength makes a smaller spot, so the pits can be smaller and closer together:

Disc	Laser	Capacity per layer
CD	infrared, 780 nm	700 MB
DVD	red, 650 nm	4.7 GB
Blu-ray	blue-violet, 405 nm	25 GB

Pause & Try

Give an example of a secondary storage technology for each of the three main categories: magnetic, optical and solid-state (flash). For each one, describe a situation where it would be the best choice.

3.3 PROCESSING DEVICES AND CPU ARCHITECTURE

Inside the CPU

A **processing device** carries out instructions and calculations. The most important one is the **central processing unit (CPU)**, the chip that runs every program on a computer. A modern CPU is a slab of silicon smaller than a postage stamp containing billions of transistors, sealed in a package that plugs into the motherboard.

A desktop CPU locked into its socket on the motherboard.

The main parts of a CPU

- **Control unit (CU):** decodes instructions and sends control signals to the other parts.
- **Arithmetic logic unit (ALU):** does arithmetic (add, subtract) and logic (AND, OR, compare).
- **Registers:** tiny, ultra-fast storage inside the CPU.
- **Cache:** fast SRAM on the chip for data needed again.
- **Clock:** regular pulses that keep every step in time.

How fast is fast?

Clock speed is measured in hertz (pulses per second). A 3 GHz CPU receives 3 000 000 000 pulses every second, so one clock cycle lasts $1 \div (3 \times 10^9)$ s ≈ **0.33 nanoseconds**; in that time light travels only about 10 cm. Most CPUs also have several **cores**: complete processing units on one chip, each able to work on a different instruction at the same moment (section 3.4).

Other processing devices

- A **GPU** (graphics processing unit) has thousands of small cores for graphics and AI (section 3.6).
- A **microcontroller** is a tiny, cheap processor with its own memory, inside embedded systems.
- An **NPU** (neural processing unit) in phones and laptops is built to run AI models efficiently.

Pause & Try

Name three main parts inside a CPU and describe the job of each. A CPU runs at 2.5 GHz: how long does one clock cycle last?

3.3 PROCESSING DEVICES AND CPU ARCHITECTURE

Registers

A **register** is a tiny storage location inside the CPU. It holds one **word** (e.g. 64 bits) and can be read or written within a clock cycle, far faster than RAM. **General-purpose registers** hold whatever data a program is using; **special-purpose registers** each have a fixed job in the fetch–execute cycle (Chapter 1).

The special-purpose registers

Register	What it holds
PC: program counter	the address of the **next** instruction to fetch
MAR: memory address register	the address of the memory location being read from or written to
MDR: memory data register	data or an instruction just read from memory, or about to be written to it
CIR: current instruction register	the instruction being decoded and executed
ACC: accumulator	the value being worked on, and the results from the ALU

Also: the **status register** holds flags such as "the last result was negative"; the **index register (IX)** holds a number added to an address to step through a list.

Worked example: one instruction, step by step

Address 20 holds the instruction **ADD 51** ("add the number at address 51 to the accumulator"); address 51 holds **7**. At the start PC = 20 and ACC = 5. The arrow ← means "is copied into".

Step	Register transfer	Result
1	MAR ← PC	MAR = 20
2	MDR ← contents of address 20	MDR = ADD 51
3	PC ← PC + 1	PC = 21, ready for the next instruction
4	CIR ← MDR, then the CU decodes it	CIR = ADD 51
5	MAR ← 51, then MDR ← contents of address 51	MDR = 7
6	ACC ← ACC + MDR (the ALU adds)	ACC = 5 + 7 = 12

Pause & Try

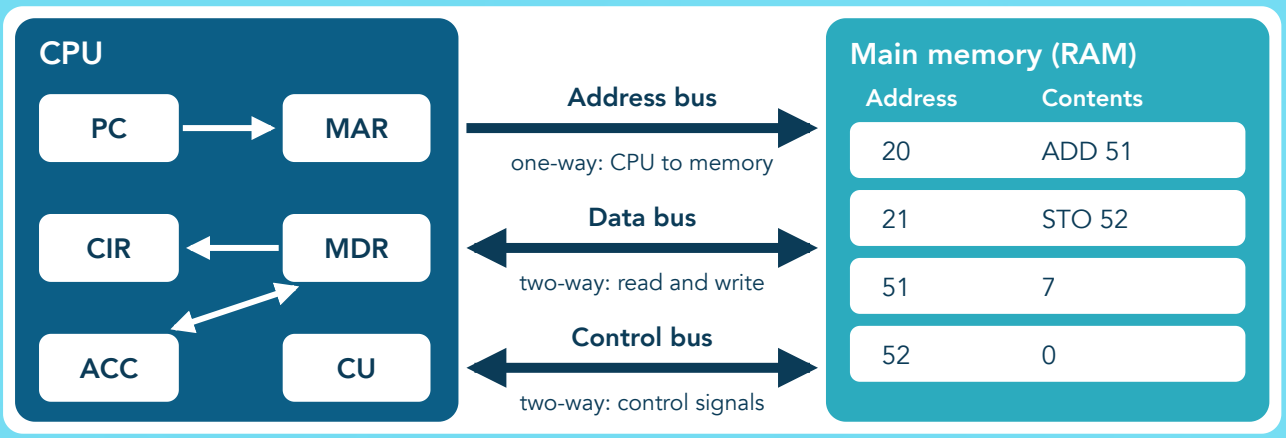
State the function of the program counter (PC), the accumulator (ACC) and the memory data register (MDR) during the execution of an instruction.

3.3 PROCESSING DEVICES AND CPU ARCHITECTURE

Buses and their directions

The CPU, memory and input/output (I/O) devices are linked by **buses**: sets of parallel wires, each wire carrying one bit at a time. A 64-bit data bus has 64 wires, so it can move 64 bits (8 bytes) at once. There are three **system buses**, and the direction in which each one carries signals matters.

The three buses between the CPU and memory



The white arrows inside the CPU are the register transfers from the previous page: PC → MAR, MDR → CIR and MDR ↔ ACC. The same three buses also link the CPU to the input and output devices.

Address bus

Carries the **address** of the location the CPU wants. Only the CPU sends addresses, so it is **unidirectional**: from the MAR to memory and I/O.

Data bus

Carries the **data and instructions**. Data travels into the CPU when reading and out when writing, so it is **bidirectional**.

Control bus

Carries **signals** such as "memory read", "memory write" and clock pulses, and interrupt requests back to the CPU, so it is **bidirectional**.

Bus width matters. An address bus with n wires can carry 2^n different addresses. A 32-bit address bus gives 2^{32} = 4 294 967 296 addresses; with one byte at each address, that is 4 GiB. That is why old 32-bit computers could use at most 4 GiB of RAM.

Pause & Try

Explain why the address bus is unidirectional but the data bus is bidirectional. How many memory locations can a 24-bit address bus address?

3.4 PARALLEL PROCESSING AND COMPUTER SYSTEMS

RISC and CISC processors

Every processor understands a fixed list of machine-code instructions: its **instruction set**. A **CISC** (complex instruction set computer) offers many powerful instructions, so programs need fewer of them; a **RISC** (reduced instruction set computer) offers fewer, simpler instructions that each do one small step quickly.

Worked example: multiply two numbers stored in memory

Task: multiply the values at addresses 200 and 201, and store the result back at address 200.

CISC, one instruction: `MULT 200, 201`. The hardware loads, multiplies and stores, taking several clock cycles.

RISC, four simple instructions: `LOAD R1, 200` · `LOAD R2, 201` · `MUL R1, R2` · `STORE R1, 200`. Each is designed to take about one clock cycle.

Comparing the two designs

Feature	CISC	RISC
Instructions	many, some very complex	fewer and simple
Instruction length	variable	fixed, e.g. 32 bits
Clock cycles per instruction	often several	usually one
Memory access	many instructions use memory	only LOAD and STORE
Registers	fewer	many general-purpose ones
Control unit	microprogrammed	hard-wired
Pipelining	harder	easier and more efficient
Program length	shorter: more work for the hardware	longer: more work for the compiler
Examples	Intel and AMD x86 chips in PCs	ARM chips in phones and Apple Macs

Neither design always wins. RISC chips are simpler and use less power, so almost every smartphone has one. Modern x86 chips blend the two, breaking complex instructions into simpler steps inside the chip.

Pause & Try

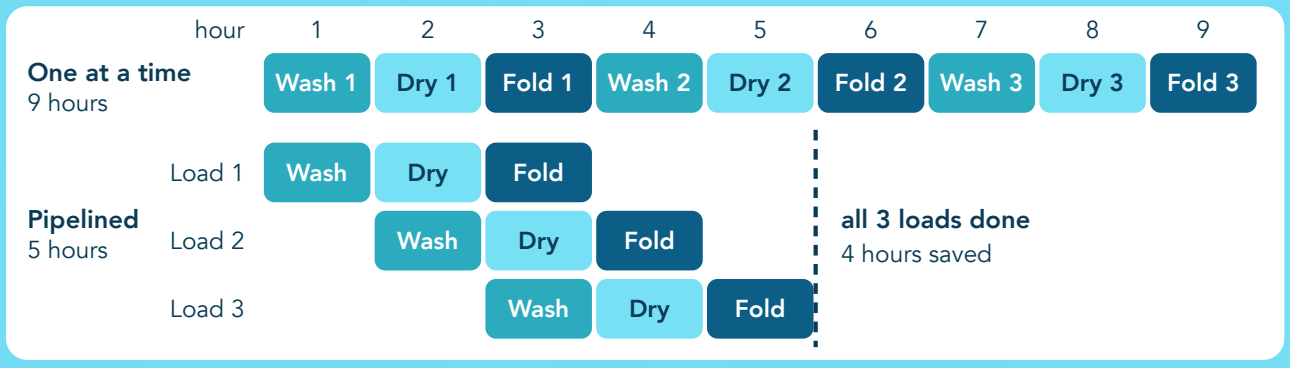
Define the core difference between RISC and CISC architectures regarding instruction size and decoding complexity.

3.4 PARALLEL PROCESSING AND COMPUTER SYSTEMS

Pipelining: the laundry analogy

Imagine you have three loads of laundry. Each load goes through three stages: **wash** (1 hour), **dry** (1 hour) and **fold** (1 hour). If you finish one load completely before starting the next, the washing machine stands idle while you dry and fold.

One load at a time, or a pipeline?



Without a pipeline: 3 loads × 3 hours = **9 hours**. **With a pipeline:** as soon as load 1 moves to the dryer, load 2 goes into the washer. Once the pipeline is full, one load is finished **every hour**, so the total is 3 + (3 − 1) = **5 hours**. Each load still takes 3 hours from start to finish (its **latency**), but more loads are finished per hour (the **throughput** rises).

The same idea inside a CPU

Executing an instruction also happens in stages, and each stage uses a different part of the processor:

Fetch get the instruction → **Decode** work out what it means → **Execute** the ALU does the work → **Write back** store the result

A **pipelined** processor fetches the next instruction while the current one is being decoded, just as the washer takes a new load while the dryer is busy. One load is one instruction, each machine is one stage, and each hour is one clock cycle. With k stages, n instructions take k + (n − 1) clock cycles instead of n × k.

Pause & Try

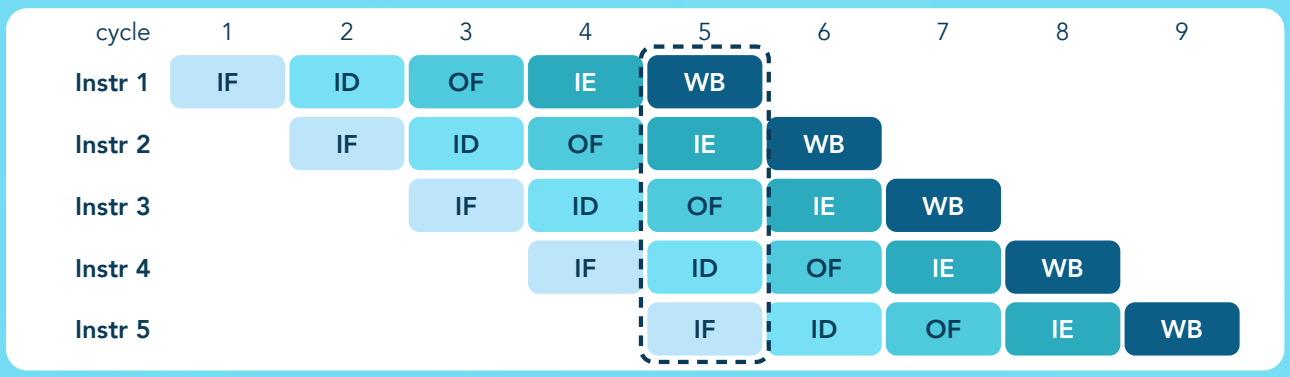
Using the laundry example, how long would 6 loads take with and without a pipeline? Explain the difference between latency and throughput.

3.4 PARALLEL PROCESSING AND COMPUTER SYSTEMS

Pipelining and registers in RISC

RISC processors were designed with pipelining in mind. A typical RISC pipeline has five stages, each taking **one clock cycle**: instruction fetch (**IF**), instruction decode (**ID**), operand fetch (**OF**), instruction execute (**IE**) and write back (**WB**).

Five instructions in a five-stage pipeline



In cycle 5 (dashed) all five stages are busy. Without pipelining, 5 instructions take 5 × 5 = 25 cycles; with it, 5 + (5 − 1) = **9 cycles**. For 100 instructions: 104 cycles instead of 500.

Why RISC suits pipelining

- **Fixed length:** the next instruction always starts in a known place.
- **Simple instructions:** each stage takes one cycle, so the pipeline moves in step.
- **Load/store:** only LOAD and STORE use slow memory.

Why registers matter

A RISC processor has many general-purpose registers (32 is common). Values stay **inside the CPU**, where any stage can use them in one cycle. Fewer trips to memory mean fewer delays, so the pipeline stays full.

When the pipeline stalls

A **hazard** stops the next instruction entering the pipeline on time: a **data hazard** (it needs a result not yet written back), a **control hazard** (after a branch, the instructions already fetched are wrong and are thrown away) or a **structural hazard** (two stages need the same hardware at once).

Pause & Try

Explain the role of pipelining in executing instructions concurrently within a RISC processor, and name one hazard that can stall the pipeline.

3.4 PARALLEL PROCESSING AND COMPUTER SYSTEMS

Interrupt handling on CISC and RISC

An **interrupt** is a signal from hardware (a key press, a printer that has run out of paper, a timer) or from software (an error such as dividing by zero) that asks the CPU to stop what it is doing and deal with an event. The CPU checks for interrupts at the end of every fetch–execute cycle.

What happens when an interrupt arrives

1 · **Finish the current instruction, then check for interrupt signals**

↓

2 · **Check the priority: a lower-priority interrupt has to wait**

↓

3 · **Save the context: push the PC and other registers onto the stack**

↓

4 · **Load the address of the interrupt service routine (ISR) into the PC**

↓

5 · **Run the ISR to deal with the event**

↓

6 · **Restore the saved registers from the stack and carry on**

On a CISC processor

One complex instruction can take **many clock cycles**, and the CPU normally lets it finish before responding, so the wait can be longer. A few very long instructions, such as copying a whole block of memory, can be paused part-way; the CPU must then save extra information so that it can restart them later.

On a RISC processor

Instructions are short, so the CPU can respond quickly. But the pipeline may hold up to five partly finished instructions. The usual solution is to **discard** them, except the one in the write-back stage, service the interrupt, then **restart** from the next instruction. Less often, the contents of all five stages are saved in registers so no work is lost.

Some RISC processors keep a spare set of registers just for interrupt routines, so less has to be saved.

Pause & Try

What is a hardware interrupt, and what happens, step by step, when the CPU receives one? Explain why a pipelined RISC processor needs an extra step.

3.4 PARALLEL PROCESSING AND COMPUTER SYSTEMS

SISD, SIMD, MISD and MIMD

Parallel processing means splitting work so that different processors carry out parts of it **at the same time**. In 1966 Michael Flynn sorted computer architectures by two questions: how many **instruction streams** run at once, and how many **data streams** are processed at once?

Flynn's four classes

	Single instruction stream	Multiple instruction streams
Single data stream	SISD	MISD
Multiple data streams	SIMD	MIMD

SISD

Single instruction, single data. One processor carries out one instruction on one data item at a time: the classic Von Neumann computer, such as an early single-core PC.

SIMD

Single instruction, multiple data. Many processing units carry out the **same** instruction on different data at the same moment. Ideal for images, sound and AI: GPUs and the vector units in modern CPUs work this way.

MISD

Multiple instruction, single data. Several processors carry out different instructions on the same data. It is rare: used where results must be cross-checked, e.g. some fault-tolerant flight-control computers.

MIMD

Multiple instruction, multiple data. Several processors each run their own instructions on their own data. Multi-core CPUs, clusters and supercomputers are MIMD.

Worked example. Brightening a 1 000 000-pixel photo (add 20 to every pixel) takes an SISD processor one million ADD instructions; a SIMD unit that adds to 16 pixels at once needs 1 000 000 ÷ 16 = **62 500**.

Pause & Try

Describe the difference between SIMD and MIMD, giving an example of each. To which class does a quad-core laptop running four different programs belong?

3.4 PARALLEL PROCESSING AND COMPUTER SYSTEMS

Massively parallel computers

Some problems are too big for any single processor: forecasting the weather for a whole continent, simulating how a new medicine works in the body, or training a large AI model. They are tackled by splitting the work across many processors, from a few dozen to millions.

Clusters

A **cluster** is a number of ordinary computers (**nodes**) linked by a fast network so that they work as one resource. Each node is a complete computer, often with SIMD processors. Clusters are cheap to build and easy to expand: just add more nodes.

Supercomputers

A **supercomputer** is an extremely powerful system built for huge scientific and engineering calculations. The fastest ones contain millions of processor cores and carry out more than 10^{18} calculations every second.

Characteristics of massively parallel computers

A **massively parallel computer** links thousands of processors together so that they effectively form one machine. Its main characteristics:

- **thousands of processors** work on parts of the same problem at the same time (MIMD);
- each processor usually has **its own memory** and works on its own share of the data;
- the processors are joined by a **high-speed interconnect** and coordinate by **passing messages** to each other;
- the software must **split the task** into parts that can run independently, then combine the results.

The limits of parallel processing

Adding processors only speeds up the part of a job that can be split. **Worked example:** a job takes 100 s on one processor; 90 s of it can be shared out, but 10 s must run in sequence. With 10 processors it takes 10 + 90 ÷ 10 = **19 s**: about 5 times faster, not 10 times. Even with a million processors it can never take less than 10 s. Passing messages between processors also costs time, so massively parallel computers suit problems that split into many independent pieces.

Pause & Try

Describe two characteristics of massively parallel computers. A job takes 200 s on one processor, and 50 s of it cannot be split. How long would it take on 5 processors?

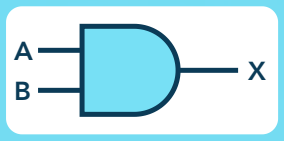
3.5 LOGIC CIRCUITS AND BOOLEAN ALGEBRA

Logic gates: AND, OR and NOT

Deep inside every CPU, the ALU and the control unit are built from **logic gates**. A logic gate is a tiny electronic circuit, made from a few transistors, that takes one or more binary inputs and produces one binary output. A high voltage stands for **1** (true) and a low voltage for **0** (false).

To describe exactly what a gate does, we use a **truth table**: it lists **every** possible combination of inputs and the output for each one. With n inputs there are 2^n combinations, so a gate with two inputs needs 4 rows.

AND



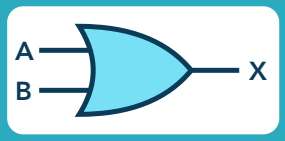
The output is 1 **only if both** inputs are 1.

X = A AND B, written **A.B**

Example: a car starts only if the key is turned AND the brake is pressed.

A	B	X
0	0	0
0	1	0
1	0	0
1	1	1

OR



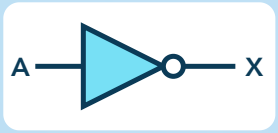
The output is 1 if **at least one** input is 1.

X = A OR B, written **A + B**

Example: a car's interior light comes on if the driver's door OR a passenger door is open.

A	B	X
0	0	0
0	1	1
1	0	1
1	1	1

NOT



The output is the **opposite** of the input. It has only one input.

X = NOT A, written $\overline{A}$

Example: a night light is on when it is NOT light outside.

A	X
0	1
1	0

Reading the symbols: inputs come in on the left and the output leaves on the right. AND has a flat back and a round front; OR has a curved back and a pointed front; NOT is a triangle with a small circle. That small circle always means "invert the output".

Pause & Try

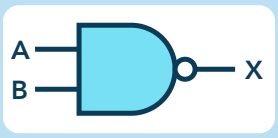
Draw or describe the logic gate symbols and write out the truth tables for the three basic operations: AND, OR and NOT.

3.5 LOGIC CIRCUITS AND BOOLEAN ALGEBRA

NAND, NOR and XOR

The next three gates are built by combining the first three. Remember the small circle on an output: it always means "followed by NOT".

NAND (NOT AND)

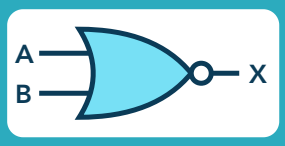


An AND gate followed by NOT: the output is 0 **only if both** inputs are 1.

$X = \overline{A.B}$

A	B	X
0	0	1
0	1	1
1	0	1
1	1	0

NOR (NOT OR)

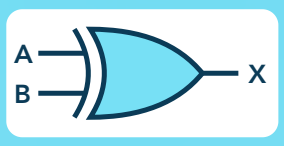


An OR gate followed by NOT: the output is 1 **only if both** inputs are 0.

$X = \overline{A + B}$

A	B	X
0	0	1
0	1	0
1	0	0
1	1	0

XOR (exclusive OR)



The output is 1 if the inputs are **different**.

$X = A \oplus B$

Example: a stair light with a switch at the top and the bottom: flicking either switch changes the light.

A	B	X
0	0	0
0	1	1
1	0	1
1	1	0

Why these gates matter

NAND and NOR are called **universal gates**: any logic circuit can be built from NAND gates alone, or from NOR gates alone. For example, joining both inputs of a NAND gate turns it into a NOT gate. Chip makers can therefore build complex circuits by repeating one simple design millions of times.

XOR is how a computer adds. When the ALU adds two bits, the **sum** bit is A XOR B and the **carry** bit is A AND B: 1 + 1 = 10 in binary (sum 0, carry 1).

Pause & Try

Write the truth tables for NAND and NOR from memory. Then use a truth table to show that a NAND gate with both of its inputs joined together behaves like a NOT gate.

3.5 LOGIC CIRCUITS AND BOOLEAN ALGEBRA

Logic circuits

A **logic circuit** joins several gates so that the output of one becomes an input of the next. You can design one in three steps: describe the problem in words, write it as a **logic expression**, then draw the **circuit**.

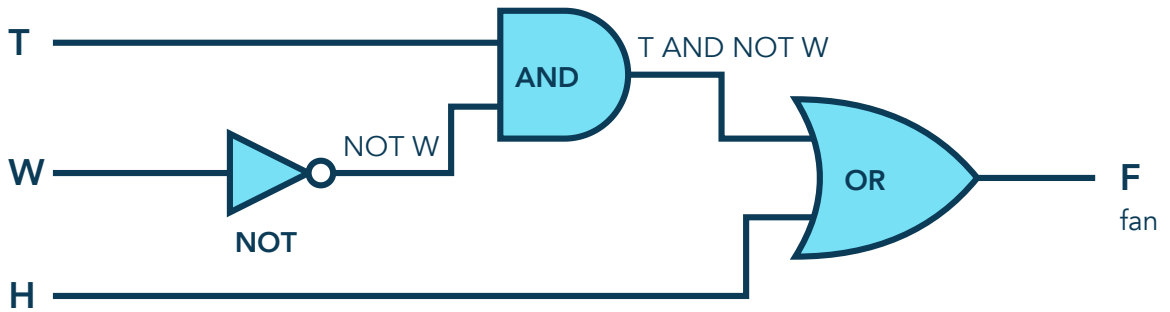
From a problem to a circuit

The problem: a greenhouse fan F must switch on when it is hot (T = 1) and the window is **not** open (W = 0), or whenever the humidity is high (H = 1).

Step 1: name the inputs and the output. Inputs: T (too hot), W (window open), H (humidity high). Output: F (fan on).

Step 2: turn the words into an expression. "and" becomes AND, "or" becomes OR, "not" becomes NOT: F = (T AND NOT W) OR H, written $F = T.\overline{W} + H$

Step 3: draw one gate for each operation, working from the inputs towards the output:



Tracing one case through the circuit

It is hot (T = 1), the window is shut (W = 0) and the air is dry (H = 0):

NOT gate NOT 0 = 1 → **AND gate** 1 AND 1 = 1 → **OR gate** 1 OR 0 = 1 → **Fan** on

Pause & Try

Draw the logic circuit for X = (A AND B) OR (NOT C). Then trace the output when A = 1, B = 0 and C = 0.

3.5 LOGIC CIRCUITS AND BOOLEAN ALGEBRA

Building a truth table

A truth table shows how a circuit behaves for **every** possible input. Let's build one for the greenhouse fan circuit, F = T.$\overline{W}$ + H, one column at a time.

Step by step

Step 1: count the rows. There are 3 inputs, so there are 2^3 = **8** combinations.

Step 2: fill in the inputs. Count in binary from 000 to 111. The H column alternates 0, 1; the W column goes in pairs; the T column is four 0s then four 1s. This way no combination is missed or repeated.

Step 3: add a column for each gate, in the order the signal passes through them. First the NOT gate: flip every value in the W column to get $\overline{W}$.

Step 4: the AND gate. T.$\overline{W}$ is 1 only in the rows where T = 1 **and** $\overline{W}$ = 1.

Step 5: the OR gate. F is 1 if T.$\overline{W}$ is 1 **or** H is 1.

Step 6: read and check the result. The fan is on in 5 of the 8 cases. Check one row against the words: in row 5 (hot, window shut, dry air) F = 1, just as the problem says.

Truth table for F = T.$\overline{W}$ + H

T W H	$\overline{W}$	T.$\overline{W}$	F
0 0 0	1	0	0
0 0 1	1	0	1
0 1 0	0	0	0
0 1 1	0	0	1
1 0 0	1	1	1
1 0 1	1	1	1
1 1 0	0	0	0
1 1 1	0	0	1

Read each row from left to right: the inputs first, then each gate's output in turn, until you reach F.

Three mistakes to avoid

- **Missing rows:** n inputs always need 2^n rows; counting in binary makes sure none is missed.
- **Treating OR as addition:** in logic, 1 + 1 = 1, not 2.
- **Wrong order:** work from the inputs to the output, brackets and NOT gates first.

Pause & Try

Build a full truth table for X = (A AND B) OR (NOT C), with a column for each gate. In how many of the 8 rows is X = 1?

3.5 LOGIC CIRCUITS AND BOOLEAN ALGEBRA

Boolean algebra

Boolean algebra is the algebra of 1s and 0s. It lets you write any logic circuit as an expression and then **simplify** it: a simpler expression needs fewer gates, so the circuit is cheaper, faster and uses less power. **Notation:** A.B means A AND B; A + B means A OR B; $\overline{A}$ means NOT A. Careful: + means OR, not addition, so 1 + 1 = 1.

Laws you can use

Law	AND form	OR form
Identity	A.1 = A	A + 0 = A
Null	A.0 = 0	A + 1 = 1
Idempotent	A.A = A	A + A = A
Inverse	$A.\overline{A} = 0$	$A + \overline{A} = 1$
Absorption	A.(A + B) = A	A + A.B = A
Distributive	A.(B + C) = A.B + A.C	A + B.C = (A + B).(A + C)
De Morgan	$\overline{A.B} = \overline{A} + \overline{B}$	$\overline{A + B} = \overline{A}.\overline{B}$

Also: NOT NOT A = A (double negation), and A.B = B.A, A + B = B + A (the order does not matter).

Worked example: simplify $X = A.B + A.\overline{B}$

Distributive law, take out the common factor A: $X = A.(B + \overline{B})$

↓

Inverse law, $B + \overline{B} = 1$: $X = A.1$

↓

Identity law, $A.1 = A$: $X = A$

Check with a truth table: for A B = 00, 01, 10 and 11 the original expression gives X = 0, 0, 1, 1: exactly the values of A. The original circuit needed two AND gates, a NOT gate and an OR gate; the simplified one is just a wire from A.

Pause & Try

Simplify $Z = A.B.C + A.B.\overline{C}$ using Boolean algebra, naming the law used at each step. Then check your answer with a truth table.

3.6 HOW HARDWARE ENABLES AI

Software needs hardware

An AI model is maths that has to run somewhere

An AI model, such as one that recognises cats in photos, is a program plus millions or billions of numbers called **weights**, learned during training. Using the model means multiplying the input data by those weights, layer after layer, and every one of those calculations must be carried out by a processor using data held in memory. The hardware decides how big a model can be and how fast it answers.

Who does what

The hardware behind an AI model

Hardware	Its role in AI
CPU	runs the program, prepares the data and makes decisions: varied, step-by-step tasks
GPU	thousands of small cores carry out the same maths on lots of data at once (SIMD style)
RAM	holds the program and the data set while they are in use
VRAM	the GPU's own fast memory: holds the model's weights and the data being processed
SSD	stores data sets and trained models, and loads them quickly into memory

Matrix maths

A network layer multiplies its inputs by a grid of weights called a **matrix**, then adds up the results.

A tiny layer, worked out

Two inputs, 2 and 3, feed two outputs. Output 1 has the weights 1 and 2: $2 \times 1 + 3 \times 2 = \mathbf{8}$. Output 2 has the weights 3 and −1: $2 \times 3 + 3 \times (-1) = \mathbf{3}$. No multiplication waits for another, so all four can be done at the same moment.

A real layer with 1000 inputs and 1000 outputs needs $1000 \times 1000 = \mathbf{1\ 000\ 000}$ multiplications for every input example. Because they are independent, a GPU can carry out thousands of them at once, which is why GPUs train neural networks many times faster than CPUs.

Pause & Try

Explain why a GPU is better suited than a CPU to training a neural network. Use the words "matrix", "parallel" and "cores" in your answer.

3.6 HOW HARDWARE ENABLES AI

Memory, sensors and limits

Memory bandwidth

Capacity answers "does the model fit?"; **memory bandwidth** answers "how fast can the data move?". A language model with 7 billion weights, each stored in 2 bytes, needs $7 \times 10^9 \times 2$ = **14 GB** of VRAM. To produce each word (or part of a word) it must read every weight once: at 1 TB/s that takes 14 GB ÷ 1000 GB/s = **14 ms**, so it can produce at most about 70 words a second, however fast its cores are. Often the processor is waiting for memory, not the other way round.

Sensors and edge AI

A robot or a phone senses the world through cameras, microphones, LiDAR and motion sensors. Running the model **on the device itself** is called **edge AI**: on many phones, face unlock runs on the phone's own neural processing unit. Edge AI responds quickly, works without a network and keeps data private, but the model must fit the device's small memory, battery and cooling. Every step from sensor to action takes time, and a delay anywhere slows the reaction:

Camera takes a frame → **Memory** holds it → **NPU** runs the model → **CPU** picks an action → **Motor** moves

Limits and accelerators

More powerful hardware lets a model be bigger or faster, but it does not create the learning algorithm. Every chip is limited by heat, power use, memory and cost, so engineers build **accelerators** just for AI maths: Google's **TPUs** in data centres and **NPUs** in phones and laptops. Training a large model may need thousands of GPUs for weeks, but running the trained model (**inference**) can then happen on much smaller hardware.

Watch the tutorial: How hardware enables AI (Video 02)
https://github.com/user-github-me/cs-ai-project-01/blob/main/assets/tutorials/v2/BlueImpulse_v2_Video02_How_Hardware_Enables_AI.mp4

Pause & Try

Explain how memory capacity, memory bandwidth, power and heat limit running a large AI model on a small device such as a drone.

CHAPTER 3 · PRACTICE

Practice

Practice · Chapter 3

1. Classify the following hardware components as input, output or storage devices, noting dual roles where they apply:
   a. touchscreen display (consider both perspectives) [2]
   b. solid-state drive (SSD) [1]
   c. barcode reader [1]

2. Output devices
   a. Explain why an OLED screen does not need a backlight. [1]
   b. Describe how a speaker produces sound from digital data, naming the component that converts the data into an analogue signal. [3]

3. Outline a simple scenario for an automated environmental control system, for example an automated greenhouse or a smart thermostat, specifying:
   a. the sensor used [1]
   b. the processing threshold condition [1]
   c. the resulting actuator action [1]

4. A washing machine is controlled by an embedded microcontroller.
   a. State what is meant by an embedded system. [1]
   b. Give two benefits and one drawback of using an embedded microcontroller in a washing machine. [3]

5. Storage classification
   a. Differentiate between primary storage and secondary storage in terms of volatility, access speed and direct CPU accessibility. [3]
   b. Give one example of a secondary storage technology for each of the three main categories: magnetic, optical and solid-state (flash). [3]

6. Memory technologies
   a. Explain the structural difference between SRAM (static RAM) and DRAM (dynamic RAM) regarding how data bits are stored and their need for periodic refreshing. [4]
   b. Contrast ROM (read-only memory) with PROM (programmable ROM) and EPROM (erasable programmable ROM) in terms of how data is written and erased. [4]
   c. A smartphone contains DRAM, SRAM and flash memory. State what each type of memory is used for in the phone. [3]

CHAPTER 3 · PRACTICE

Practice

Practice · Chapter 3 (continued)

7. Registers and buses
   a. State the function of the following registers during the instruction cycle: program counter (PC), accumulator (ACC) and memory data register (MDR). [3]
   b. Explain why the address bus is unidirectional. [1]
   c. Calculate how many memory locations can be addressed using a 20-bit address bus. [1]

8. Processor design
   a. Define the core difference between RISC (reduced instruction set computer) and CISC (complex instruction set computer) architectures regarding instruction size and decoding complexity. [3]
   b. Explain the role of pipelining in executing instructions concurrently within a RISC processor, and name one hazard that can stall the pipeline. [3]

9. What is a hardware interrupt, and what step-by-step sequence occurs at the machine-code level when an interrupt signal is received by the CPU? [4]

10. Parallel processing
    a. Describe the difference between SIMD and MIMD architectures, giving an example of each. [4]
    b. Describe two characteristics of massively parallel computers. [2]

11. Draw or describe the logic gate symbols and write out the full truth tables for the three basic logic operations: AND, OR and NOT. [6]

12. A logic circuit has three inputs, A, B and C. Its output X is 1 when A is 1 and B is 0, or when B and C are both 1.
    a. Write the Boolean expression for X. [1]
    b. Draw the logic circuit. [2]
    c. Complete the truth table for the circuit. [4]

13. Simplify $Q = A.B.C + A.B.\overline{C} + A.\overline{B}$ using Boolean algebra, naming the law used at each step. [3]

14. Explain why a GPU with a large amount of VRAM is used to train neural networks, and give one reason why a phone might run an AI model on its own NPU instead of sending the data to the cloud. [4]

15. Calculations
    a. An ultrasonic sensor's echo returns after 11.6 ms. Calculate the distance to the object (speed of sound 343 m/s). [2]
    b. A hard disk spins at 10 000 rpm. Calculate its average rotational latency. [2]
    c. A five-stage pipeline runs 20 instructions. Calculate the clock cycles needed with and without pipelining. [2]
    d. A 60 s job has 12 s that cannot be split. Calculate how long it takes on 4 processors. [2]

AFTERNOON

NETWORKING

4

CHAPTER 4 · BEFORE YOU BEGIN

Learning goals

Every time you send a message, stream a video or open a web page, data makes a journey: across the cables and radio links of a local network, through the routers of the Internet, packaged and addressed by protocols at every step. This chapter follows that journey, then shows how networks keep data secure, private and correct — from passwords and firewalls to encryption, digital certificates and quantum key distribution.

Learning goals

You will learn about:

- The characteristics of a WAN and a LAN
- Client-server and peer-to-peer models in networking
- The differences between thick client and thin client
- Public and private cloud computing
- Hardware required to support a LAN
- Function of routers
- How data collisions are detected and avoided
- Differences between the Internet and the World Wide Web
- Hardware needed to support the Internet
- IP addresses
- The use of the uniform resource locator (URL) to locate a resource on the World Wide Web
- The role of the Domain Name Service (DNS)
- The need for protocols during communication
- The implementation of protocols such as a stack
- TCP/IP protocols including the 4 layers (Application, Transport, Internet and Link)
- HTTP, POP3, FTP, IMAP, etc.
- The terms security, privacy, integrity of data
- Security to protect computer systems such as user accounts, passwords, digital signatures, firewalls, antivirus and anti-spyware software and encryption
- Security threats such as viruses and spyware, hacking, phishing, and pharming
- Methods to reduce security risks such as encryption and access rights
- Use of validation to protect data integrity
- Use of verification during data entry and data transfer to reduce or eliminate errors
- Circuit switching
- Benefits and drawbacks of packet switching
- Function of a router in packet switching
- Use of packet switching to pass messages along a network
- How encryption works, including the use of public and private keys, plaintext and ciphertext, symmetric key cryptography and asymmetric key cryptography
- How keys can be used to send verified messages
- How data is encrypted using symmetric and asymmetric cryptography
- Quantum cryptography and QKD
- SSL and TLS
- Examples of where SSL and TLS would be used
- How digital certificates are acquired
- How digital certificates are used to produce digital signatures

CHAPTER 4 · BEFORE YOU BEGIN

Key terms

Key terms · Networks, models and the cloud

Node: Any device connected to a network — a computer, printer, storage device or smartphone.

Host: A computer or device that can communicate with other computers or devices on a network.

LAN: Local area network — covers a small area such as one building or site, and is owned and managed by one organisation.

WLAN: Wireless LAN — a LAN in which devices connect by radio waves (Wi-Fi) instead of cables.

WAN: Wide area network — covers a very large area such as a country or the whole world, usually over links leased from telecom companies.

MAN: Metropolitan area network — larger than a LAN but smaller than a WAN, e.g. several buildings in one city or a university campus.

PAN: Personal area network — a network centred around one person or their workspace.

WPAN: Wireless PAN — connects devices in very close proximity, such as a laptop, smartphone, tablet and printer in a user's home.

ARPAnet: Advanced Research Projects Agency Network — the US network of 1969 that pioneered packet switching and grew into the Internet.

Topology: The layout of the links between the nodes of a network, e.g. bus, star or mesh.

Client-server: A network that uses separate dedicated servers and specific client workstations; all client computers are connected to the servers.

Client: A computer that requests a service, such as a file or a web page, from a server.

Server: A computer that provides a service — files, email, web pages, printing — to the clients on a network.

File server: A server where central files and other data are stored; users can access them when logged on to the network.

Peer-to-peer (P2P): A network with no central server: each node keeps its own data and can share files with all the other nodes.

Peer: A client that is part of a peer-to-peer network or file-sharing community.

BitTorrent: A protocol used in P2P networks to share a file between many peers at once, in small pieces.

Pieces: The small parts a file is split into for peer-to-peer file sharing.

Metadata: Data that describes other data, e.g. a file's name, size and list of pieces.

Tracker: A central server that stores details of all the computers in a swarm, but not the shared files themselves.

Swarm: The connected peers (clients) that share one torrent.

Seed: A peer that has downloaded a file (or pieces of it) and then makes it available to other peers in the swarm.

Leech: A peer with negative feedback from swarm members, usually because it downloads far more than it uploads.

Lurker: A user who downloads files but does not supply any new content to the community.

Thin client: A device or program that needs a network connection to work and depends on a more powerful computer (a server) for processing and storage.

Thick client: A device or program that can work both offline and online, because it processes and stores data itself.

Cloud storage: Storing data on off-site servers that are run by a provider and reached over the Internet.

Public cloud: Cloud services owned by a provider and shared by many customers, e.g. Google Drive or Amazon Web Services.

Private cloud: Cloud infrastructure used by one organisation only, hosted on its own site or by a provider.

Data redundancy: The same data stored on several servers, so it is still available during maintenance, repair or a failure.

CHAPTER 4 · BEFORE YOU BEGIN

Key terms

Key terms · LAN hardware, media and the Internet

NIC: Network interface card — hardware that connects a device to a network; it has a unique MAC address set at the factory.

MAC address: The unique 48-bit hardware address of a NIC, written as six pairs of hexadecimal digits, e.g. 00:1A:2B:3C:4D:5E.

Hub: Hardware that connects devices into a LAN and sends every incoming packet to all the devices on it.

Switch: Hardware that connects devices into a LAN and sends each incoming packet only to its destination device.

Router: A device that routes packets between different networks, e.g. joins LANs to form a WAN or connects a home LAN to the Internet.

Routing table: A router's list of the networks it knows, the next hop towards each one and the cost of that route.

(W)AP: (Wireless) access point — allows a device to access a LAN without a wired connection.

Repeater: A device that boosts a weakened signal on wired or wireless networks so that it can travel further.

Repeating hub: A network device that is a hybrid of a hub and a repeater.

Bridge: A device that connects LANs which use the same protocols.

Gateway: A device that connects LANs (or other networks) which use different protocols, translating between them.

Modem: Modulator-demodulator — converts digital data to analogue signals to send down a telephone wire, and analogue signals back to digital.

Softmodem: Software modem — a modem that does most of its work in software and uses minimal hardware.

Twisted pair cable: A cable in which the two wires of each circuit are twisted together to reduce interference; several pairs make up one cable.

Wi-Fi: Wireless connectivity that uses radio waves and microwaves; it implements the IEEE 802.11 protocols.

Bluetooth: Short-range wireless connectivity that uses radio waves in the 2.45 GHz frequency band.

Spread spectrum technology: Wideband radio frequency transmission with a range of 30 to 50 metres.

Spread spectrum frequency hopping: A device picks one of 79 radio channels at random and, if it is in use, hops to another; range up to 100 metres.

Data collision: Two or more nodes sending on a shared medium at the same time, so that their signals overlap and are corrupted.

CSMA/CD: Carrier sense multiple access with collision detection — the wired method: listen, send, detect a collision, jam, back off and retry.

CSMA/CA: Carrier sense multiple access with collision avoidance — the wireless method: wait for a clear channel, send and expect an acknowledgement.

Internet: The worldwide network of interconnected networks — the cables, routers and servers that carry data.

World Wide Web (WWW): The collection of linked web pages and resources that are accessed over the Internet using HTTP.

ISP: Internet service provider — a company that connects homes and organisations to the Internet.

IP address: A number that identifies a device on a network so that packets can be routed to it.

IPv4 / IPv6: 32-bit addresses written as four denary numbers (e.g. 192.168.1.20) / 128-bit addresses written as eight groups of hex digits (e.g. 2001:db8::1).

URL: Uniform resource locator — the full address of a resource on the web, e.g. https://www.example.com/notes.html.

DNS: Domain Name Service (also called the Domain Name System) — the worldwide system of servers that translates domain names such as www.example.com into IP addresses.

CHAPTER 4 · BEFORE YOU BEGIN

Key terms

Key terms · Protocols, switching and threats

Protocol: A set of rules governing communication across a network; the rules are agreed by both sender and recipient.

Protocol stack: A set of protocols arranged in layers, where each layer does one job and serves the layer above it.

TCP/IP: The four-layer protocol stack used on the Internet: application, transport, internet and link layers.

TCP: Transmission control protocol — splits data into numbered segments, checks that they all arrive and puts them back in order.

Host-to-host: A protocol used by TCP when communicating between two devices.

Segment: A unit of data (packet) associated with transport-layer protocols such as TCP.

Packet: Data split into a small group of bits for transmission; it carries the sender's address, the recipient's address and the data.

Encapsulation: Each layer wrapping the data from the layer above inside its own header (and sometimes a trailer).

HTTP: Hypertext transfer protocol — the rules for requesting and sending web pages; HTTPS is its encrypted, secure form.

FTP: File transfer protocol — the rules for uploading and downloading files between a client and a server.

SMTP: Simple mail transfer protocol — the push protocol used to send emails to a mail server and between mail servers.

POP3: Post office protocol 3 — a pull protocol that downloads emails to one device and usually deletes them from the server.

IMAP: Internet message access protocol — a pull protocol that keeps emails on the server and syncs them across all your devices.

MIME: Multipurpose Internet mail extension — lets an email carry attachments such as images, sound and video as well as text.

Push protocol: Used when sending emails: the client opens a connection to the server, keeps it active and uploads new emails to it.

Pull protocol: Used when receiving emails: the client connects to the server periodically to download (pull) new emails.

Binary file: A file that does not contain text only, such as an image or a program; machine-readable but not human-readable.

Circuit switching: A dedicated path is set up between sender and receiver for the whole communication, then released.

Packet switching: A message is split into packets that travel independently, possibly by different routes, and are reassembled at the destination.

Data security: Protecting data and computer systems from unauthorised access, loss, damage or corruption.

Data privacy: The right of individuals to control how their personal data is collected, used and shared.

Data integrity: Data remaining accurate, complete and unchanged during entry, storage and transfer.

Malware: Malicious software, such as viruses and spyware, written to damage a computer or steal data.

Virus: A program that copies itself into other files and can delete or corrupt data, fill up storage or crash a computer.

Spyware: Software that secretly records a user's activity, such as key presses, and sends it to a third party.

Hacking: Gaining unauthorised access to a computer system, often to steal, change or delete data.

Phishing: Fake emails or messages that look genuine, sent to trick people into revealing personal details on a fake website.

Pharming: Malicious code on a computer or DNS server that redirects users to a fake website, even when they type the correct address.

CHAPTER 4 · BEFORE YOU BEGIN

Key terms

Key terms · Protection and cryptography

Firewall: Hardware or software that checks traffic going in and out against a set of rules and blocks anything not allowed.

Antivirus software: Software that scans files for viruses and quarantines or deletes infected ones.

Anti-spyware software: Software that detects and removes spyware and blocks keyloggers.

Access rights: Permissions that control what each user can do with a file or folder, e.g. read, write or delete.

Validation: Automatic checks that input data is sensible and follows set rules (e.g. range, format, length) before it is accepted.

Verification: Checking that data has been entered or transferred exactly as the original, e.g. by double entry or a parity check.

Eavesdropper: A person who intercepts data being transmitted.

Encryption: Using an algorithm and a key to turn plaintext into ciphertext, so that it is meaningless to anyone without the key.

Decryption: Using a key to turn ciphertext back into the original plaintext.

Plaintext: The original text, document or message before it is put through an encryption algorithm.

Ciphertext: The product when plaintext is put through an encryption algorithm.

Symmetric encryption: Encryption in which the same secret key is used to encrypt and decrypt messages.

Key distribution problem: The risk, in symmetric encryption, that the secret key is intercepted by an eavesdropper or hacker while it is being sent to the recipient.

Asymmetric encryption: Encryption that uses a pair of keys: a public key known to everyone and a private key known only to its owner.

Public key: An encryption/decryption key known to all users; it encrypts messages for its owner and verifies the owner's signatures.

Private key: An encryption/decryption key known only to a single user or computer; it decrypts messages and creates digital signatures.

Block cipher: The encryption of a number of contiguous bits in one go rather than one bit at a time.

Stream cipher: The encryption of bits in sequence as they arrive at the encryption algorithm.

Block chaining: Each plaintext block is XORed with the previous block of ciphertext before it is encrypted, so identical plaintext blocks do not give identical ciphertext.

Hash: A short, fixed-length value calculated from a message; any change to the message changes the hash.

Digital signature: A hash of a message encrypted with the sender's private key; it proves who sent the message and that it was not changed.

Digital certificate: An electronic document, signed by a certificate authority, that links a person or website to their public key.

Certificate authority (CA): A trusted organisation that checks identities and issues digital certificates.

SSL / TLS: Secure Sockets Layer and its successor, Transport Layer Security — protocols that encrypt and authenticate data sent between applications, e.g. HTTPS.

Quantum cryptography: Using the properties of photons (particles of light) to protect data, relying on the laws of physics instead of hard maths.

QKD: Quantum key distribution — sending a secret key as polarised photons; any eavesdropping disturbs the photons and is detected.

4.1 NETWORK ARCHITECTURE & TOPOLOGIES

Networks by size

A **network** is two or more devices connected so that they can exchange data and share resources: files, printers, software and an Internet connection. Each connected device is called a **node**. Networks are first described by the **area they cover**; the two you will compare in detail are the **LAN** and the **WAN**.

A PAN is like two people whispering on the same bench: close, private and quick.

From a whisper to the post

Picture four ways of passing on a message. A **PAN** is a whisper between two people on a bench. A **LAN** is the intercom inside one apartment block: privately wired and very fast. A **MAN** is a city's bus network. A **WAN** is the international post, using roads, planes and ships owned by many countries.

PAN a whisper → **LAN** a building intercom → **MAN** a city bus network → **WAN** the international post

Four sizes of network

Network	Area covered	Example
PAN / WPAN	Around one person (a WPAN is wireless)	Phone, smartwatch and earbuds linked by Bluetooth
LAN / WLAN	One building or site (a WLAN is wireless)	A school, an office, a home network
MAN	A town, a city or a large campus	University buildings spread across a city
WAN	Countries and continents	A bank's branches nationwide; the Internet

Pause & Try

Classify each network as a PAN, LAN, MAN or WAN: a) a laptop linked to a wireless keyboard and mouse; b) the computers in one school; c) a city council linking its 12 offices across town; d) an airline linking airports on four continents.

4.1 NETWORK ARCHITECTURE & TOPOLOGIES

Local area networks (LAN)

Most of the networks you use every day are LANs: the network at home, at school or in an office. Four characteristics (ownership, speed, reliability and transmission media) set a LAN apart from a WAN.

What makes a network a LAN?

A **local area network (LAN)** connects devices inside a small area: one home, one office, one school building or a single site.

- **Ownership:** every cable, switch and access point is bought, owned and managed by one person or organisation, such as the school's IT team. Nobody else's equipment is involved.
- **Speed:** data travels only a few metres, or a few hundred, over the owner's own equipment, so a LAN is very fast: 100 Mbps to 10 Gbps over cables (1 Gbps is typical) and several hundred Mbps over Wi-Fi.
- **Reliability:** short links pick up little interference, so few errors occur, and when something breaks the owner can fix it directly.
- **Transmission media:** twisted-pair Ethernet cables to each desk, fibre-optic cables between floors or buildings, and Wi-Fi radio for laptops and phones. A LAN whose devices connect by radio is a **WLAN** (wireless LAN).

A LAN's network cabinet: cables from every room meet at switches owned by the school.

The speed in numbers

Copying a 1 GB video (8 000 megabits) to a classmate:

- over a 1 Gbps LAN: 8 000 ÷ 1 000 = **8 seconds**
- over a 50 Mbps Internet link: 8 000 ÷ 50 = **160 seconds**

Same file, twenty times longer: the second copy leaves the LAN and shares slower, longer links owned by other companies.

Pause & Try

A school copies a 2 GB video over its 1 Gbps LAN, then sends the same file to a partner school over a 40 Mbps Internet link. a) Estimate both transfer times. b) Give two reasons why the LAN copy is faster.

4.1 NETWORK ARCHITECTURE & TOPOLOGIES

Wide area networks (WAN)

A **wide area network (WAN)** connects devices, or whole LANs, that are far apart: in different towns, countries or continents. A supermarket chain that links the tills in 300 stores to its head office runs a WAN, and the **Internet** is the largest WAN of all.

What makes a network a WAN?

- **Ownership:** no single organisation owns all of a WAN. Companies lease (rent) capacity from telecom providers: fibre-optic and undersea cables, telephone lines, microwave, satellite and 4G/5G links.
- **Speed:** backbone links carry many gigabits per second, but each user's share is usually smaller than on a LAN, and data takes longer to arrive because it crosses thousands of kilometres and many routers.
- **Reliability:** long distances, more equipment and many different owners mean more things can fail, so WANs are built with routers and alternative routes that keep data moving when one link breaks.
- **Transmission media:** mostly fibre-optic cable, plus copper telephone lines, radio and satellite where cables cannot reach.

LAN and WAN side by side

	LAN	WAN
Area	One building or site	Towns, countries, the world
Ownership	One organisation owns it all	Links leased from telecom companies
Speed	100 Mbps – 10 Gbps per device	Smaller share per user; longer delays
Reliability	High; the owner fixes faults	More points of failure; backup routes
Main media	Twisted pair, Wi-Fi, some fibre	Fibre, undersea cable, satellite, mobile

Because nobody owns all of a WAN, your data crosses equipment that belongs to other companies. That is one reason why encryption (4.9) matters so much once data leaves the LAN.

Pause & Try

A bank links 200 branches across a country; each branch has 30 computers in one building. Identify the LANs and the WAN in this system, then compare them on ownership, speed and reliability.

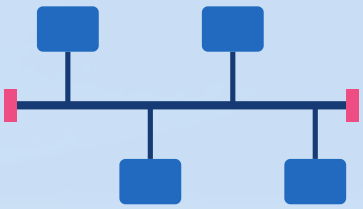
4.1 NETWORK ARCHITECTURE & TOPOLOGIES

Network topologies

A network's **topology** is its shape: the pattern of links that joins the nodes together. The topology decides how much cable is needed, how easily the network can grow and, most importantly, what happens when one link breaks. Most networks are built from three basic shapes.

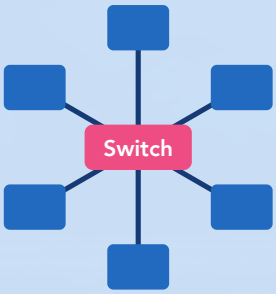
Three basic shapes

Bus



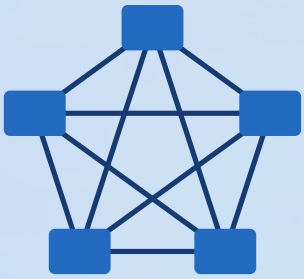
one shared cable

Star



a cable each to a switch

Mesh



several routes between nodes

Bus

All nodes share **one cable** (the backbone), with a terminator at each end. It is cheap and needs little cable, but only one node can send at a time, two senders cause a **collision** (see 4.4), and a break in the backbone stops the whole network.

Star

Each node has **its own cable** to a central switch. A broken cable affects only one node, new nodes are easy to add, and the switch sends data only where it needs to go. If the switch fails, though, the whole star stops. Most LANs today are stars.

Mesh

Nodes have **several links** to each other, so data can take another route when a link fails. This makes a mesh very reliable but expensive to build. The routers of the Internet form a partial mesh.

Real networks are often **hybrids**: a school might wire each classroom as a star and join the classroom switches with a fibre-optic backbone.

Pause & Try

A school wants a network in which one broken cable cannot stop everyone working, at a reasonable cost. Which topology would you recommend? Give one reason for your choice and one weakness it still has.

4.1 NETWORK ARCHITECTURE & TOPOLOGIES

Client-server networks

Networks are also described by **how they organise their services**. In the **client-server** model, one or more powerful computers called **servers** provide services, and the other computers, the **clients**, request them. Every client connects to the servers, and the servers decide who may do what.

How a request is served

Client sends a request → **Server** checks the user's rights → **Server** does the work → **Client** receives the response

Common servers: a **file server** stores users' files centrally; a **print server** queues print jobs; an **email server** stores and forwards mail; a **web server** sends web pages.

Client-server in real life

- **Your school network:** you can log in at any computer and still see your own documents, because they live on the file server, not on the PC in front of you.
- **Online banking:** the app on your phone is the client; your account data stays on the bank's servers.
- **Streaming and online games:** video and game servers send content to millions of clients at once.

Cost

High. Servers, server software, backup systems and trained IT staff all cost money. In return, one team can manage hundreds of clients, and the network grows easily by adding clients or upgrading servers.

Security

Central control. Passwords, access rights, antivirus updates and backups are managed in one place. But the server is a **single point of failure** and a tempting target: if it goes down, clients cannot work unless there is a backup server.

Pause & Try

Your school's file server crashes on the morning of an exam. Explain what happens to students using the client-server network, and suggest how the school could reduce this risk in future.

4.1 NETWORK ARCHITECTURE & TOPOLOGIES

Peer-to-peer networks

In a **peer-to-peer (P2P)** network there is no central server. Every node, called a **peer**, keeps its own data and acts as **both a client and a server**: it can request files from other peers and supply files to them.

P2P in real life

- **A small home or office:** three laptops share each other's folders and one printer, with no server.
- **BitTorrent file sharing:** a large file is downloaded from many peers at once.
- **Cryptocurrency networks** such as Bitcoin: thousands of peers each keep a copy of the record of payments.

How BitTorrent shares a file

The file is split into small **pieces**. A **tracker** server keeps metadata, such as which computers are in the **swarm** (the peers sharing this torrent), but not the file itself. Each peer downloads pieces from several others while uploading the pieces it already has. A **seed** has the whole file and keeps uploading it; a **leech** downloads far more than it uploads and gets negative feedback; a **lurker** downloads but never adds new content.

Client-server or peer-to-peer?

	Client-server	Peer-to-peer
Where data is stored	Centrally, on servers	On each peer
Cost	Higher: servers and IT staff	Low: no server needed
Security	Managed centrally by IT staff	Each user secures their own device
Backups	One central backup	Each peer backs up separately
If one computer fails	A server failure stops everyone	Only that peer's files are unavailable
Best for	Schools, banks, large firms	Homes, small offices, file sharing

Pause & Try

An architecture firm with 40 staff works on large 3D rendering files and needs daily off-site backups and strict user permissions. Should it use a client-server or a peer-to-peer network? Justify your recommendation with three reasons.

4.2 CLIENT COMPUTATIONS & CLOUD TECHNOLOGIES

Thin and thick clients

When a computer joins a network, its work can happen in two places: on the device itself or on a server. A **thin client** depends on a server; a **thick client** can do the work itself. The difference shows most clearly in **where data is processed** and **where it is stored**.

Thin client

A low-cost device, or a program such as a web app, that sends your input to a **server**, which runs the software and sends back the results.

- **Processing:** on the server.
- **Storage:** little or none; programs and your files live on the server.
- **Offline:** cannot do its main job.
- **Examples:** library terminals; a word processor in a web browser.
- **+** cheap; updated once, centrally. **–** useless without a connection; heavy server load.

Thick client

A full computer with its own processor, memory and **local storage**, which runs software itself.

- **Processing:** on its own CPU, GPU and RAM.
- **Storage:** its own SSD or hard disk holds the operating system, programs and files.
- **Offline:** keeps working.
- **Examples:** a gaming PC; a laptop with installed office software.
- **+** fast; works offline; less network traffic. **–** costly per user; software updated on every machine.

The game analogy

A thin client is like **cloud gaming**: your device sends key presses and shows video, while the game runs on a server far away. A thick client is like a game **installed on your own gaming PC**: your graphics card does all the work.

Thin client (your key press) → **Server far away** (runs the game) → **Your screen** (shows the video)

Thick client (your key press) → **Your own GPU** (runs the game) → **Your screen** (shows the result)

Pause & Try

Differentiate between a thick client and a thin client based on where data processing and storage take place. Then explain what happens to a thin-client user's work when the server fails.

4.2 CLIENT COMPUTATIONS & CLOUD TECHNOLOGIES

Public and private clouds

Cloud computing means using servers in a provider's data centre, reached over the Internet, to store files (**cloud storage**), run software or process data instead of owning all the hardware yourself. Providers build in **data redundancy**: every file is copied to several servers, often in different places, so one hardware failure does not lose it.

Public cloud

Owned and run by a provider and **shared by many customers**, who rent as much storage and computing power as they need.

- **Examples:** a school storing coursework in Google Drive or Microsoft OneDrive; Netflix running its streaming service on Amazon Web Services.
- **Cost:** low start-up cost; pay only for what you use.
- **Control:** limited. The provider chooses the hardware, and data may be stored in another country.

Private cloud

Cloud infrastructure **used by one organisation only**, in its own data centre or hosted for it elsewhere.

- **Examples:** a bank keeping customer accounts on its own cloud; a hospital storing patient scans; a government agency handling confidential records.
- **Cost:** high. The organisation buys the servers and employs the staff.
- **Control:** full control of hardware, location and security settings, which helps meet strict data laws.

The trade-offs

	Public cloud	Private cloud
Cost	Low upfront; pay as you go	High upfront, plus staff
Security	Provider secures shared hardware	You secure it; data is isolated
Control	Little say over location and setup	Full control
Growth	Almost unlimited, within minutes	Limited by your own hardware

Any cloud storage lets you reach your files from any device and acts as an off-site backup, but it needs an Internet connection, has ongoing fees and relies on the provider's security.

Pause & Try

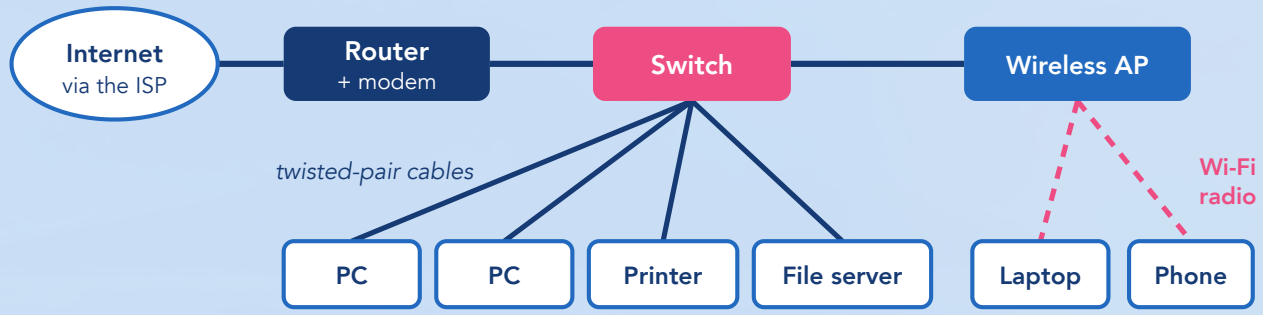
A hospital wants to store patient scans, and a start-up expects its new app to become suddenly popular. Recommend a public or a private cloud for each and justify your choices using cost, security and control.

4.3 HARDWARE REQUIRED TO SUPPORT A LAN

The hardware in a LAN

To build a LAN you need a way for each device to send and receive signals, a medium to carry them, a central device that joins the nodes together and, usually, a way out to the Internet. Here is a small school LAN, part by part.

A small LAN



Every device has a NIC (wired or wireless) with its own MAC address.

What each part does

Hardware	Its job in the LAN
NIC (network interface card)	Built into every device; turns data into signals and back. Each NIC has a unique **MAC address** set at the factory.
Cables	Twisted-pair Ethernet cables connect each wired device to the switch.
Switch (or hub)	The centre of the star: joins all the wired nodes and passes data between them.
Wireless access point	Lets laptops, tablets and phones join the LAN by Wi-Fi radio instead of a cable.
Router (with a **modem**)	Connects the LAN to other networks; the modem carries signals over the line to the Internet service provider (ISP).
Servers (optional)	A file or print server provides shared services in a client-server LAN.

At home, the router, modem, switch and access point usually come combined in **one box**.

Pause & Try

List the hardware needed to set up a LAN for a classroom with 25 desktop PCs, 5 tablets and one shared printer, all with Internet access. State the job of each item.

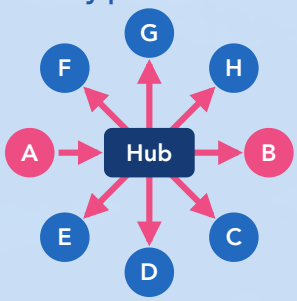
4.3 HARDWARE REQUIRED TO SUPPORT A LAN

Hubs, switches and more

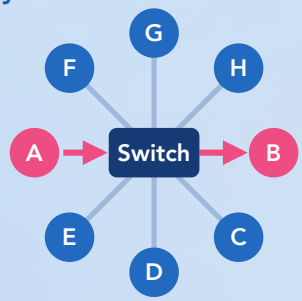
Hubs and switches both join the nodes of a LAN, but a **hub** copies every incoming packet to all of its ports, while a **switch** reads the destination **MAC address** and sends the packet only to the right port.

Worked example: one file, two devices

Hub: copies to every port



Switch: only to B



Node A sends a 10 MB file to node B, first on an **8-port hub**, then on an **8-port switch**.

- **Hub:** the 10 MB is copied to ports 2 to 8. Nodes C to H each receive 10 MB they did not need (60 MB wasted) and discard it. Meanwhile no other node can send.
- **Switch:** it looks up B's MAC address in its **MAC address table** and forwards the frames to B's port only, so C can talk to D at the same moment. The switch **learns** the table by noting the source address of every frame and the port it came in on.

Other connecting devices

- **Repeater:** regenerates a weak signal so it can travel further, e.g. a Wi-Fi extender. A **repeating hub** is a hub that also boosts the signals it copies.
- **Bridge:** joins two LANs that use the same protocols, e.g. the LANs of two classrooms.
- **Gateway:** joins networks that use different protocols, translating between them, e.g. a smart-home hub linking Zigbee light bulbs to Wi-Fi.
- **Modem:** converts digital data into analogue signals for a telephone line, and back again. A **softmodem** does this mainly in software, with minimal hardware.

Pause & Try

Node A sends a 6 MB file to node D on a 6-port hub. How much data reaches nodes that did not need it? Explain how a switch would change this, and why that makes the LAN faster for everyone.

4.3 HARDWARE REQUIRED TO SUPPORT A LAN

How routers choose a route

A switch moves data **within** one LAN. A **router** moves data **between** networks: from your home LAN to your Internet provider, or from one company site to another. Every packet carries a destination **IP address** (see 4.5), and the router passes it one step, or **hop**, closer to it.

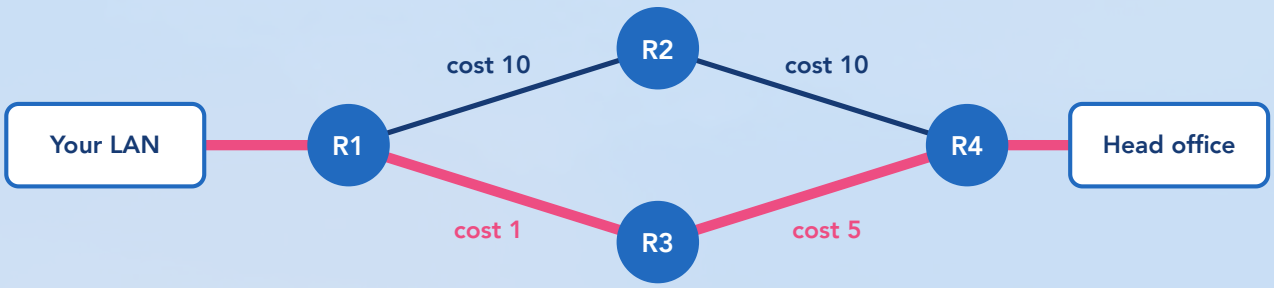
The routing table

Each router keeps a **routing table**: the networks it knows about, the **next hop** towards each one, and a **cost**, a score where lower is better (fewer hops or faster links).

Destination network	Next hop	Cost
192.168.1.x (this LAN)	Port 1: deliver directly	0
10.20.0.x (head office)	Router R3	6
Anything else (default route)	The ISP's router	–

For every packet, the router reads the destination address, finds the matching line and sends the packet to that next hop. A packet for 10.20.0.7 goes to R3; a packet for a web server the router has never heard of follows the **default route** to the ISP.

How the router decides



Routers regularly swap information with their neighbours about the networks they can reach and what it costs. From this, R1 works out that the route through R2 costs 10 + 10 = **20**, while the route through R3 costs 1 + 5 = **6**, so it sends head-office packets to R3. If the R3–R4 link fails, the routers tell each other, recalculate and switch to the R2 route within seconds. Routes **change** as the network changes.

Pause & Try

In the map above, the R1–R3 link becomes congested and its cost rises from 1 to 12. Which route will R1 now choose, and what is its total cost? Explain how R1 finds out about the change.

4.4 TRANSMISSION MEDIA & COLLISION HANDLING

Transmission media

The **transmission medium** is whatever carries the signal between nodes: copper wire, glass fibre or radio waves. Each medium has a different speed, range and cost, and a different resistance to **interference**: outside electrical noise that corrupts signals.

Cables

- **Twisted-pair cable:** pairs of copper wires twisted together to cancel interference from nearby cables. Cat5e and Cat6 carry 1–10 Gbps over up to 100 m: the standard for Ethernet LANs.
- **Coaxial cable:** a copper core inside a metal shield; used for cable TV and broadband.
- **Fibre-optic cable:** pulses of light in thin glass strands. Very fast over many kilometres and immune to electrical interference, but costly to install; used between buildings and for the Internet backbone.

Wireless

- **Wi-Fi** (IEEE 802.11) uses radio waves and microwaves to connect devices to a WLAN. It uses **spread spectrum technology**, spreading the signal over a wide band of frequencies, with a range of about 30–50 m.
- **Bluetooth** uses radio waves in the 2.45 GHz band for short-range WPANs, e.g. earbuds.
- **Spread spectrum frequency hopping** (used by Bluetooth): the device picks one of **79 channels** at random and hops to another if it is busy, avoiding interference; range up to 100 m.

Wired or wireless?

	Wired (cable)	Wireless (radio)
Speed	Faster and steady	Usually slower; falls with distance
Interference	Low (none for fibre)	Higher: walls, other devices
Security	An intruder must plug in	Signals leak outside: encrypt them
Mobility	Fixed positions	Move around freely
Cost	Cabling is costly to install	New devices join easily

Pause & Try

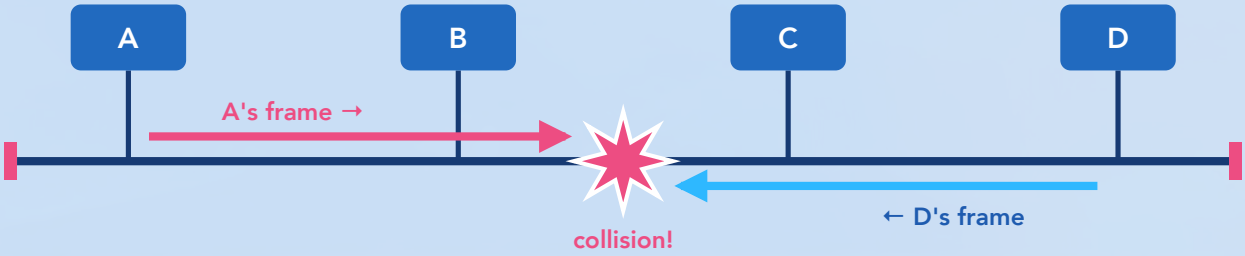
Recommend a transmission medium for each case and justify it: a) linking two school buildings 400 m apart; b) giving café customers Internet access on their phones; c) connecting wireless earbuds to a phone.

4.4 TRANSMISSION MEDIA & COLLISION HANDLING

Data collisions

On a **shared medium** (one cable used by several nodes, as in a bus or a hub, or the radio channel of a Wi-Fi network) only one node can send at a time. If two nodes send at the same moment, their signals overlap and corrupt each other: a **data collision**. Both frames are destroyed and must be sent again.

How a collision happens



1. Nodes A and D both listen and hear a **quiet** cable, so both start sending.
2. Signals take time to travel, so neither has yet heard the other's frame.
3. The signals meet and add together: B and C receive a **garbled** mixture that nobody can read.

The dark-room party

Imagine a dinner party in a **dark room**: you cannot see who is about to speak, so you can only listen. If two guests start talking at exactly the same moment, nobody understands either of them.

CSMA/CD is like both guests stopping, waiting a random few seconds and trying again. **CSMA/CA** is like raising your hand and waiting for the host's signal before you speak.

Two people talking at once: neither message gets through.

Networks need rules to **detect** collisions (cables) or **avoid** them (radio). On a switched Ethernet LAN each cable is a private link to the switch, so collisions cannot happen there.

Pause & Try

Two nodes on a shared cable both sense that it is idle and start sending at the same moment. Describe what happens to their signals, and explain why they must not simply resend straight away.

4.4 TRANSMISSION MEDIA & COLLISION HANDLING

Detecting collisions: CSMA/CD

CSMA/CD is used on wired Ethernet when several nodes share one cable or a hub. Its name lists its three ideas: **carrier sense** (listen before you send), **multiple access** (many nodes share the cable) and **collision detection** (notice when two signals clash, and recover).

CSMA/CD step by step

1 · Listen
is the cable idle? If not, wait

↓

2 · Send
start transmitting the frame

↓

3 · Monitor
keep checking the voltage on the cable

↓

4 · Collision?
voltage too high: stop, send a jam signal

↓

5 · Back off
wait a random time, then return to step 1

What each step is for

Listening first prevents most collisions: a node never interrupts a frame it can already hear.

Monitoring works because two overlapping signals add together, so the voltage on the cable rises above what one sender alone would produce.

The **jam signal** is a short burst that makes sure every node notices the collision, not just the two senders.

Backing off for a **random** time matters: if both nodes waited the same fixed time, they would simply collide again. After 16 failed attempts the frame is abandoned.

Why random waits work, in numbers

After a collision, A and B each pick a random wait of 0 or 1 time slots. They collide again only if they pick the **same** number: 2 chances in 4, so **1/2**. After a second collision each chooses from 0–3 slots, and the chance of another clash drops to 4 in 16 = **1/4**. Every further collision doubles the range, so one node soon wins the cable and the other waits its turn.

Pause & Try

After a third collision, nodes A and B each choose a random wait from 0–7 slots. What is the chance that they collide again? Explain why the range of waits keeps doubling, and why a jam signal is needed.

4.4 TRANSMISSION MEDIA & COLLISION HANDLING

Avoiding collisions: CSMA/CA

Wi-Fi devices also share one medium, the radio channel, but they cannot detect collisions as cables can. A radio cannot hear other signals while it is transmitting, and two devices on opposite sides of an access point may be too far apart to hear each other at all: the **hidden node** problem. So Wi-Fi uses **CSMA/CA**: it tries to **avoid** collisions, and relies on acknowledgements to find out whether a frame got through.

CSMA/CA step by step

1 · Listen
is the channel clear? If not, wait

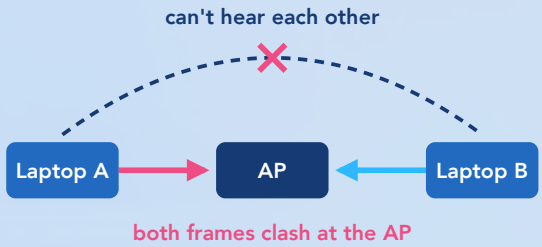
2 · Count down
wait a random back-off time

3 · Reserve (optional)
send RTS; the access point replies CTS

4 · Send
transmit the frame

5 · ACK?
no acknowledgement: assume a collision, back off and retry

The hidden node problem



A and B each hear a clear channel and send. The **CTS** from the access point, which both laptops can hear, tells everyone else to stay quiet until the frame is done.

Sharing a medium also slows a network down. **Bandwidth** is how much data a link can carry per second; **latency** is how long data takes to arrive. Collisions, back-offs and resends add latency and shrink each device's share of the bandwidth: two of the network challenges explored in the video.

Watch the tutorial: Network challenges (Video 03)
https://github.com/user-github-me/cs-ai-project-01/blob/main/assets/tutorials/stage2/videos/BlueImpulse_Video03_Network_Challenges.mp4

Pause & Try

How do network systems handle data collisions when several devices try to transmit at the same time over a shared medium? Answer for a wired network using CSMA/CD and for Wi-Fi using CSMA/CA.

4.5 INTERNET INFRASTRUCTURE, URLS & DNS

The Internet and the Web

People often use "the Internet" and "the Web" as if they meant the same thing, but they are different. The **Internet** is the infrastructure: a worldwide network of networks. The **World Wide Web** is one service that runs on top of it, just as email, video calls and online games do.

Internet or World Wide Web?

	Internet	World Wide Web (WWW)
What it is	A global network of networks	A collection of linked web pages
Made of	Cables, routers, servers	Pages, images, videos, hyperlinks
Works with	TCP/IP protocols, packets	Browsers, URLs, HTTP/HTTPS
Began	ARPAnet, 1969	Tim Berners-Lee, 1989
Carries	The Web, email, calls, games	Web pages only

Hardware that supports the Internet

Laptop → **Router + modem** → **ISP** → **Backbone** → **Web server**

- The laptop's **NIC** (wired or Wi-Fi) sends the data to the router.
- A **modem** converts digital data into signals that travel along the telephone or cable line to your ISP, and converts incoming signals back.
- **Routers** at every junction forward packets towards their destination, hop by hop.
- **Internet service providers (ISPs)** connect homes and organisations to the Internet and to each other through the high-speed **backbone**: fibre-optic and undersea cables crossing countries and oceans.
- **Servers** in data centres store web pages, emails and videos, plus the DNS records that turn names into addresses.

Pause & Try

State two differences between the Internet and the World Wide Web. Then name, in order, the hardware a request passes through on its way from your laptop to a web server.

4.5 INTERNET INFRASTRUCTURE, URLS & DNS

IP addresses: IPv4 and IPv6

Every device on the Internet needs an **IP address**: a number that identifies it, so that routers can deliver packets to it. Two versions are in use today.

IPv4: 32 bits

Four **denary numbers** from 0 to 255, separated by dots; each number is one byte.

`192.168.1.20` in binary is:

`11000000.10101000.00000001.00010100`

32 bits give $2^{32} \approx$ **4.3 billion** addresses, fewer than the number of devices online today.

IPv6: 128 bits

Eight groups of four **hexadecimal digits**, separated by colons; each group is 16 bits.

`2001:0db8:0000:0000:0000:ff00:0042:8329`

128 bits give $2^{128} \approx$ **3.4×10^{38}** addresses: enough for every device for the foreseeable future.

Shortening an IPv6 address

1. Drop the **leading zeros** in each group: 0db8 → db8, 0042 → 42, 0000 → 0.
2. Replace **one** run of consecutive zero groups with `::` (once only, or it would be ambiguous).

`2001:0db8:0000:0000:0000:ff00:0042:8329`
→ `2001:db8:0:0:0:ff00:42:8329` (rule 1)
→ `2001:db8::ff00:42:8329` (rule 2)

Practical differences

- **Running out:** the central pool of free IPv4 addresses ran out in 2011. Homes and schools cope by giving devices **private addresses** (e.g. 192.168.x.x) that share one **public address** through the router. With IPv6, every device can have its own public address.
- **Set-up:** an IPv6 device can configure its own address automatically when it joins a network.
- **Efficiency:** IPv6 packet headers have a simpler, fixed layout, so routers can process them quickly.
- **Today:** both versions run side by side, and most devices and websites support both.

Pause & Try

a) Write the IPv4 address 10.0.255.7 in binary. b) Shorten the IPv6 address 2001:0db8:0000:0000:0000:0000:0000:0001. c) Give one practical reason why networks are moving to IPv6.

4.5 INTERNET INFRASTRUCTURE, URLS & DNS

Reading a URL

A **uniform resource locator (URL)** is the full address of one resource, such as a page, an image or a video, on the World Wide Web. Each part of a URL answers a different question for the browser: **how** to fetch the resource, from **which server**, and **where** on that server it is.

One URL, six parts

```
https://www.example.com:443/courses/computer-science/notes.html#section2
```

Part	What it tells the browser
`https://`	**Protocol:** use HTTP Secure (HTTP protected by encryption)
`www.example.com`	**Domain name:** which server; DNS turns it into an IP address
`:443`	**Port:** the service on the server (443 = HTTPS, usually left out)
`/courses/computer-science/`	**Path:** which folder on the server
`notes.html`	**Resource:** which file to send back
`#section2`	**Fragment:** which part of the page to show; stays in the browser

Inside the domain name

Read it from **right to left**, from general to specific:

- `.com` : the **top-level domain** (TLD), like .org or .uk
- `example` : the organisation's registered name
- `www` : one particular host (server) it runs

Extra information

Some URLs end with a **query** after a question mark, e.g. `?q=networks`, which passes extra information (here a search term) to a program on the server.

Protocol use HTTPS → **DNS lookup** name → IP address → **Request** .../notes.html → **Display** jump to #section2

Pause & Try

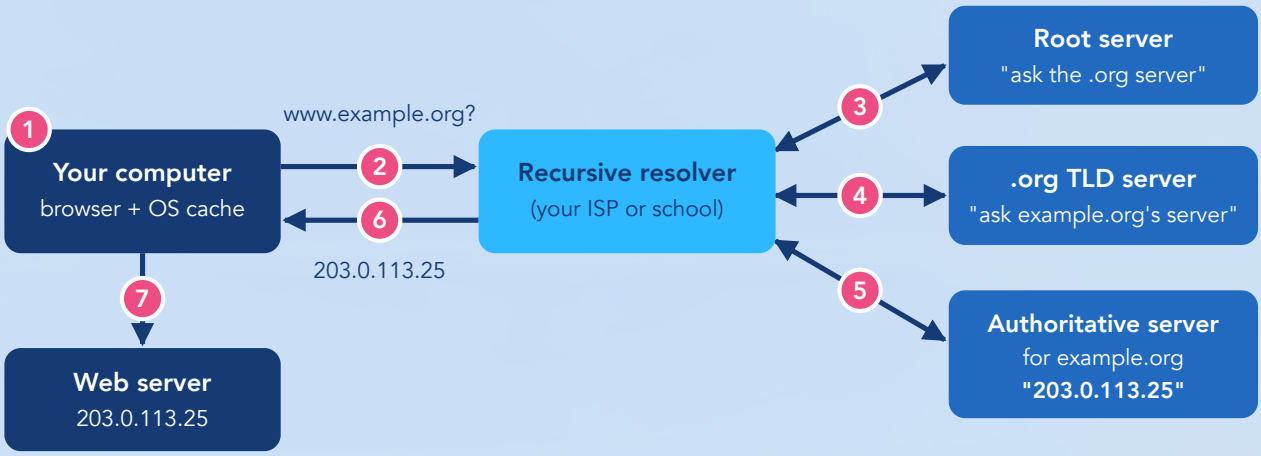
Break the URL https://shop.example.org/books/networking.html#reviews into its parts. For each part, explain how it helps the browser locate and display the resource.

4.5 INTERNET INFRASTRUCTURE, URLS & DNS

DNS: finding a website

Computers find each other by **IP address**, but people remember names such as www.example.org. The **Domain Name Service (DNS)** is a worldwide system of servers that translates domain names into IP addresses. No single server knows every name: the work is shared out in a hierarchy, and answers are **cached** (remembered for a while) to save time.

Following one lookup: www.example.org



Step by step

1. You type www.example.org. The browser and operating system first check their own **cache**: if you visited recently, the IP address is already there and the lookup ends here.
2. If not, your computer asks a **recursive resolver**, a DNS server usually run by your ISP or school.
3. The resolver asks a **root server**, which does not know the answer but replies: "ask the **.org TLD server**".
4. The **TLD server** replies: "ask **example.org's authoritative server**".
5. The **authoritative server** holds the official record and returns the IP address: **203.0.113.25**.
6. The resolver **caches** the answer and passes it back to your computer, which caches it too.
7. Your browser connects to 203.0.113.25 and requests the page using HTTPS.

Pause & Try

Describe, step by step, how DNS finds the IP address of www.example.org the first time it is visited. Then explain why a second visit a few minutes later is faster.

4.6 PROTOCOLS & THE TCP/IP PROTOCOL STACK

Protocols and the TCP/IP stack

A **protocol** is a set of rules for communication that the sender and the receiver both agree to follow. Without protocols, a receiver would get a stream of bits with no way to tell where a message starts, who it is for or whether it arrived intact. Protocols fix the **format** of the data, how it is **addressed**, how **errors** are detected, the **speed** of transmission and what to do when data goes missing.

Why a stack of layers?

Networking is too big a job for one protocol, so it is split into **layers**, each with one job. A group of layered protocols is called a **protocol stack**. Each layer uses the services of the layer below and serves the layer above, so one layer can change without affecting the others: your browser works the same whether the link underneath is Wi-Fi or a cable, and devices from different manufacturers can work together.

The four layers of TCP/IP

Layer	Its job	Data unit	Protocols
Application	Gives applications network services: web pages, email, file transfer	Data (message)	HTTP, HTTPS, FTP, SMTP, POP3, IMAP, DNS
Transport	Splits data into numbered segments, adds port numbers (which application), checks that every segment arrives and puts them back in order	Segment	TCP, UDP
Internet	Adds the source and destination IP addresses and routes packets across networks	Packet	IP (IPv4, IPv6)
Link	Adds MAC addresses and sends the bits over the physical medium	Frame	Ethernet, Wi-Fi

TCP is a **host-to-host** protocol: it manages the conversation between the two end devices (hosts), while routers in between only look as far as the internet layer. **UDP** is a lighter transport protocol that skips TCP's delivery checks; it is faster, so live video calls and online games often use it.

Pause & Try

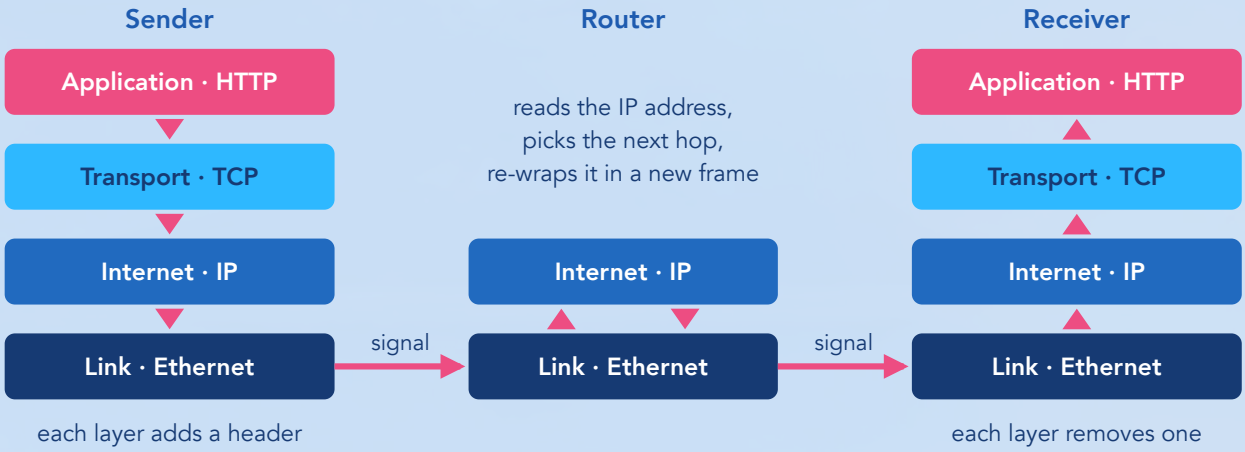
Outline the four layers of the TCP/IP stack in order, from top to bottom, giving the core responsibility of each. Then explain one benefit of organising protocols as a stack.

4.6 PROTOCOLS & THE TCP/IP PROTOCOL STACK

The layers in action

The layers never work alone. When you ask for a web page, your request travels **down** the layers of your computer, **across** the network through routers, and **up** the layers of the web server. Each layer only "talks" to the **same layer** on the other machine, using the header it adds.

One request, three machines



Following the request

1. **Application:** your browser creates an HTTP request, "GET /notes.html".
2. **Transport:** TCP splits it into segments, numbers them and adds the port number of the web service.
3. **Internet:** IP adds your IP address and the server's IP address, making a **packet**.
4. **Link:** Ethernet or Wi-Fi adds MAC addresses and an error check, making a **frame**, and sends it as signals.
5. **Router:** reads only as far as the internet layer. It checks the destination IP address, chooses the next hop and wraps the packet in a **new frame** for the next link.
6. **Server:** each layer checks and removes its own header, in reverse order, until the web server program receives exactly the request that was sent. The reply makes the same journey back.

Pause & Try

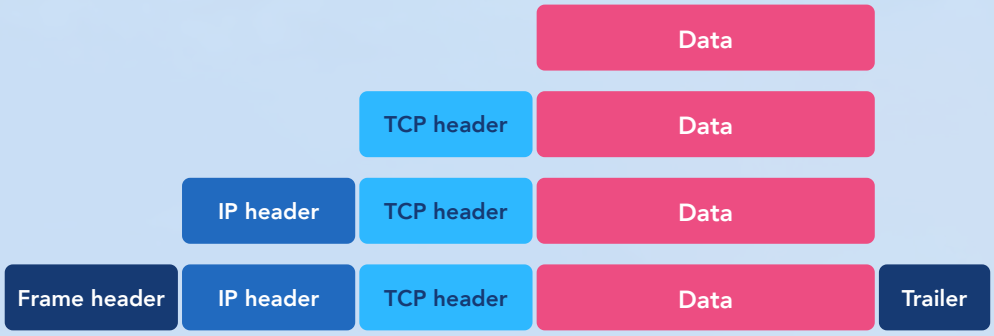
Your browser requests a web page. Describe what happens to the request at each layer of your computer, at a router on the way, and at the web server.

4.6 PROTOCOLS & THE TCP/IP PROTOCOL STACK

Encapsulation

Each layer treats everything it receives from the layer above as **data** and wraps it in its own **header** (the link layer also adds a **trailer**). This is **encapsulation**; the receiver unwraps the layers in reverse order.

Wrapping the data



TCP header: port numbers and the segment's sequence number. **IP header:** source and destination IP addresses. **Frame header:** source and destination MAC addresses. **Trailer:** an error check.

Worked example. A 5 000-byte image is sent over Ethernet, where a segment carries at most **1 460 bytes** of data. TCP needs 4 segments: 1 460 + 1 460 + 1 460 + 620 = 5 000. Each frame adds a TCP header (20 bytes), an IP header (20), an Ethernet header (14) and a trailer (4): 58 bytes. So 5 000 + 4 × 58 = **5 232 bytes** travel, about 4% of them headers.

The postal analogy

1. **Application:** you write a letter (the data).
2. **Transport:** a long letter is split into numbered pages, each in its own envelope (segments).
3. **Internet:** every envelope gets the destination and return addresses (packets).
4. **Link:** postal workers load the envelopes into a mail truck for one stretch of road (frames). At the next sorting office they move to another truck, just as a packet gets a new frame at each router.

Posting a letter: the envelope and its address wrap the message.

Pause & Try

A 3 000-byte web page is sent over Ethernet in segments of at most 1 460 bytes. How many segments are needed, and how much data does the last one carry? List the headers each frame carries, from the outside in.

4.6 PROTOCOLS & THE TCP/IP PROTOCOL STACK

Web and file protocols

The application layer holds the protocols you meet every day. Each one is a set of rules for one kind of job: fetching web pages, moving files or handling email.

HTTP and HTTPS: the web

Hypertext transfer protocol (HTTP) sets the rules for requesting and sending web pages. It works by **request and response**:

Browser sends a request → **Web server** finds the file → **Response** status code + the page → **Browser** displays it

```
Request: GET /courses/notes.html HTTP/1.1 · Host: www.example.com
Response: HTTP/1.1 200 OK · Content-Type: text/html · (then the page)
```

The browser then sends one more request for each image and style sheet on the page. If a file does not exist, the server replies **404 Not Found**. Submitting a form, such as a search or a login, sends your data to the server in a **POST** request. **HTTPS** is HTTP sent through an encrypted TLS connection (see 4.9): anyone who intercepts the packets sees only ciphertext. Every login, payment and form should use it.

FTP: moving files

File transfer protocol (FTP) is used to **upload and download files** between a client and a server: text files and **binary files** such as images, videos and programs. FTP opens one connection for commands and a separate one for the file data.

- A web designer uploads the new version of a website's files to the hosting server.
- A researcher downloads a 2 GB data set from a university archive.
- In a command-line terminal, `put report.pdf` uploads a file and `get data.zip` downloads one.

Plain FTP sends passwords without encryption, so secure versions such as **SFTP** and **FTPS** are used today.

Pause & Try

Briefly state the purpose of HTTP and of FTP, giving a practical example of each. Which protocol would you use to upload a 2 GB file to a remote website storage folder from a command-line terminal?

4.6 PROTOCOLS & THE TCP/IP PROTOCOL STACK

Email protocols

Sending an email uses **two kinds** of protocol. **SMTP** pushes the message from the sender towards the recipient's mail server; **POP3** or **IMAP** lets the recipient pull it out of their mailbox.

Follow one email

Aisha (aisha@school.example) sends a photo to Ben (ben@mail.example).

Aisha her email app → **Her mail server** SMTP → **Ben's mail server** SMTP; stores it → **Ben's devices** IMAP or POP3

SMTP: the push protocol

Simple mail transfer protocol (SMTP) is used for **sending** email. It is a **push protocol**: the sender opens a connection to a server and pushes (uploads) the message to it.

1. Aisha's email app pushes the message to her school's outgoing mail server using SMTP.
2. That server uses DNS to look up which mail server handles addresses ending in mail.example.
3. It relays the message to Ben's mail server, server to server, again using SMTP.

SMTP only delivers: it cannot fetch mail out of a mailbox. It was designed for plain text, so attachments are packaged with **MIME** (multipurpose Internet mail extensions), which lets one email carry images, sound, video and other binary files alongside the text.

POP3: download it

Post office protocol 3 is a **pull protocol**: the client connects to the server, downloads new emails to **one device** and, by default, deletes them from the server. It suits reading offline on a single computer, but Ben's phone would never see what his laptop downloaded.

IMAP: keep it on the server

Internet message access protocol is also a **pull protocol**, but the emails **stay on the server**. Every device shows the same inbox, folders and read or unread status, kept in sync. That is why most people use IMAP today.

Pause & Try

Which protocol is used in each case? a) Unread emails fetched on a laptop stay visible in the inbox on a smartphone. b) A PDF is attached to an email and sent. c) Explain why SMTP is called a push protocol.

4.7 NETWORK SECURITY, PRIVACY & DATA INTEGRITY

Security, privacy and integrity

Once devices are connected, other people can reach their data, and not all of them mean well. Protecting data means three different things; here they are, using a hospital's patient records as the example.

Data security

Protecting data and systems from **unauthorised access**, theft, accidental loss, deliberate damage or corruption.

Example: only logged-in hospital staff can open the patient database, and it is backed up every night.

Data privacy

The right of people to **control how their personal data is collected, used and shared**. Laws such as the EU's GDPR protect it.

Example: a nurse may see a record only while treating that patient, and the hospital may not sell it to an insurer.

Data integrity

Keeping data **accurate, complete and unchanged** during entry, storage and transfer.

Example: a patient's recorded allergy must be exactly what the doctor entered; one wrong digit in a dose could do harm.

Security guards control who gets in. User accounts, passwords and firewalls do the same job for data.

How they fit together

Security is the set of tools; **privacy** and **integrity** are two of the things those tools protect. A system can be secure but still misuse personal data (a privacy failure), or keep data private but full of typing mistakes (an integrity failure).

The rest of 4.7 looks at the **threats** to data, the **protection** against them, and the checks, **validation** and **verification**, that keep data correct.

Pause & Try

Differentiate between data security, data privacy and data integrity. Give an example of a failure of each in a school's student-records system.

4.7 NETWORK SECURITY, PRIVACY & DATA INTEGRITY

Security threats

A **threat** is anything that could damage, steal or expose data. Some threats are programs, known as **malware** (malicious software); others trick people, or attack a network directly.

Five threats to know

Threat	How it works	How to reduce the risk
Virus	A program that copies itself into other files and runs when they are opened; it can delete or corrupt data, fill up storage or crash the computer	Antivirus software; never open unknown attachments or run untrusted downloads
Spyware	Secretly records what you do, e.g. a **keylogger** records your key presses, and sends it to a third party	Anti-spyware software; on-screen keyboards or drop-down menus for passwords
Hacking	Gaining unauthorised access to a system, e.g. by guessing weak passwords or exploiting software bugs	Firewalls, strong passwords, software updates, encryption
Phishing	Fake emails or messages that look as if they come from a real bank or school, with a link to a fake website that asks for your details	Check the sender and the link before clicking; spam filters; staff training
Pharming	Malicious code on your computer, or a tampered DNS server, sends you to a fake website even when you type the correct address	Antivirus; check for the padlock and the site's certificate; keep software updated

Phishing or pharming?

Both lead you to a fake website that steals your details. The difference is **how you get there**. **Phishing** needs you to act: to click a link in a message. **Pharming** needs no click: the redirection happens behind the scenes, so even typing the correct URL takes you to the fake site.

Spot the attack

You receive an email "from your bank": *"Unusual activity on your account. Confirm your password within 24 hours."* The link goes to bank-secure-login.example. Warning signs: the urgency, the request for a password (banks never ask for one) and a web address that is not the bank's real domain. This is **phishing**.

Pause & Try

Explain the difference between phishing and pharming, and suggest one way to protect against each. Why can antivirus software alone not protect you from phishing?

4.7 NETWORK SECURITY, PRIVACY & DATA INTEGRITY

Accounts and access rights

The first line of defence: only the right people get in, and each of them can do only what they need.

User accounts and passwords

A **user account** gives each person a username and password. The system uses it to decide what that person may do, gives them their own storage space and logs their activity, so it is clear which account did what. When someone leaves, their account is disabled.

A **strong password** is long (12 or more characters), mixes upper and lower case letters, digits and symbols, avoids words and personal details, and is different for every site. **Two-factor authentication** adds a second check, such as a code sent to your phone.

Why length matters, in numbers

An attacker's computer can test about **10 billion** passwords per second.

Password	Possible combinations	Time to try them all
8 lower-case letters	$26^8 \approx 2.1 \times 10^{11}$	about 21 seconds
12 characters, any of 94 symbols	$94^{12} \approx 4.8 \times 10^{23}$	about 1.5 million years

Access rights

Access rights (permissions) set what each account may do with a file or folder: **read**, **write**, **delete** or **no access**. Users get only the rights they need, so a stolen student password cannot change marks.

Folder	Student	Teacher	Network manager
Own home folder	Read, write	Read, write	Full control
Class resources	Read only	Read, write	Full control
Exam marks	No access	Read, write (own classes)	Full control

Pause & Try

A password uses 6 lower-case letters. How many combinations are possible, and how long would 10 billion guesses per second take to try them all? Suggest two ways to make it stronger.

4.7 NETWORK SECURITY, PRIVACY & DATA INTEGRITY

Firewalls and anti-malware

Accounts control who can log in. The tools on this page guard everything else: the network connection, the files on the disk and the messages that travel between computers.

Firewall

Hardware or software that sits **between a computer (or a whole LAN) and the Internet** and examines every packet going in or out. It compares each packet's IP addresses, port numbers and protocol with a set of **rules** and blocks anything not allowed: for example, incoming connections that nobody asked for, or traffic to known malicious websites. It also keeps a log of blocked attempts so that attacks can be investigated.

Antivirus software

Scans files when they are downloaded or opened, and on a regular schedule, comparing them with a database of known virus **signatures**. It also watches for virus-like behaviour. Infected files are **quarantined** (isolated) or deleted. It must be updated often to recognise new viruses.

Anti-spyware software

Detects and removes spyware and blocks keyloggers from recording what you type. Some banking sites add on-screen keyboards or drop-down menus for passwords, so that there are no key presses to record.

Encryption

Scrambles data so that anyone who intercepts or steals it sees only meaningless **ciphertext**. It protects Wi-Fi traffic, web sessions (HTTPS) and the disk of a lost laptop. Encryption does not stop data being stolen; it makes stolen data useless without the key (see 4.9).

Digital signatures

Attached to an email or a document, a digital signature proves **who sent it** and that **nobody changed it** on the way. A fake email claiming to be from your bank would fail the signature check (see 4.9).

Pause & Try

Which security mechanism best solves each problem? a) Stopping hackers from reading captured Wi-Fi traffic. b) Proving that an email contract really came from a supplier and was not changed. c) Stopping employees from opening malicious websites on office computers.

4.7 NETWORK SECURITY, PRIVACY & DATA INTEGRITY

Validation and verification

Integrity can be damaged without any attacker: someone mistypes a date, or electrical noise flips a bit as data crosses a network. Two kinds of check catch these errors.

Validation: is the data sensible?

Validation is an automatic check, made as data is entered, that it follows set rules.

Check	Rule	Rejects
Range	Age must be 11–19	42
Length	Phone number has 11 digits	0791234
Type	Age must be a whole number	fifteen
Format	Date as dd/mm/yyyy	2024-02-12
Presence	Surname must not be blank	(empty)

Validation cannot prove that data is **correct**: a student aged 15 who types 16 still passes the range check. *Worked example:* 31/02/2024 passes the format check but fails a range (logic) check, because February never has 31 days.

Verification: was it copied correctly?

Verification checks that data entered or transferred **matches the original**.

- **During entry: double entry** means typing the data twice so the computer can compare the two (e.g. a new password); a **visual check** means a person compares the screen with the source document. Neither can catch the same mistake made twice.
- **During transfer:** a **parity check** adds one extra bit so that every byte has an even number of 1s (even parity). The receiver counts the 1s again: an odd count means an error. A **checksum** works the same way with a total calculated from a whole block of data.

Worked example: the data 1011001 has four 1s, so the parity bit is 0, giving **0**1011001. If noise flips one bit and 01011**1**01 arrives, its five 1s give an odd count: the error is detected and the byte is resent. If **two** bits flip, the count is even again and the error is missed.

Pause & Try

a) Add an even parity bit to the 7-bit data 1100101. b) The byte arrives as 01110101: has an error been detected? c) Contrast validation with verification, giving one example check of each.

4.8 SWITCHING MECHANISMS: CIRCUIT VS. PACKET SWITCHING

Circuit switching

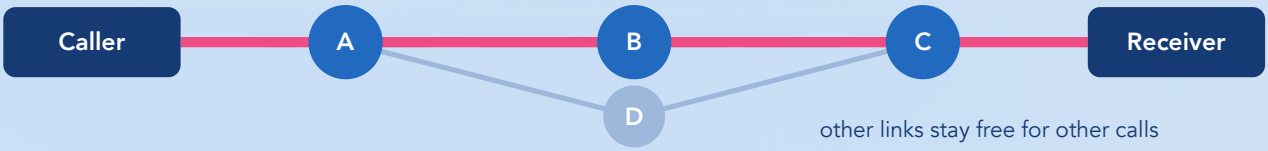
To cross a large network, data passes through many switching points. There are two ways to organise the journey: reserve a whole path in advance (**circuit switching**), or send the data in independent pieces that each find their own way (**packet switching**).

How circuit switching works

A **dedicated path** is set up between the sender and the receiver **before** any data is sent. The path, and all of its capacity, belongs to that one conversation until it ends.

1 · Set up reserve a path → **2 · Transfer** data follows it, in order → **3 · Tear down** release the path

one circuit through exchanges A, B and C, reserved for the whole call



The classic example is a **landline telephone call**. Dialling sets up a circuit through the telephone exchanges; the circuit stays open while you talk, even during silences; hanging up releases it.

Benefits

- The **whole bandwidth** of the path is reserved, so the data rate never drops.
- Delay is **low and constant**, and data arrives **in order**: ideal for live voice and video.
- No sequence numbers or reassembly are needed.

Drawbacks

- Capacity is **wasted** whenever nobody is speaking or sending.
- The path must be **set up first**, which delays the start.
- If any link on the path fails, the connection **drops**.
- When every circuit is busy, new calls are blocked; dedicated lines are expensive.

Pause & Try

a) Explain what is meant by circuit switching. b) Video conferencing is one application in which circuit switching is often preferred. Explain why.

4.8 SWITCHING MECHANISMS: CIRCUIT VS. PACKET SWITCHING

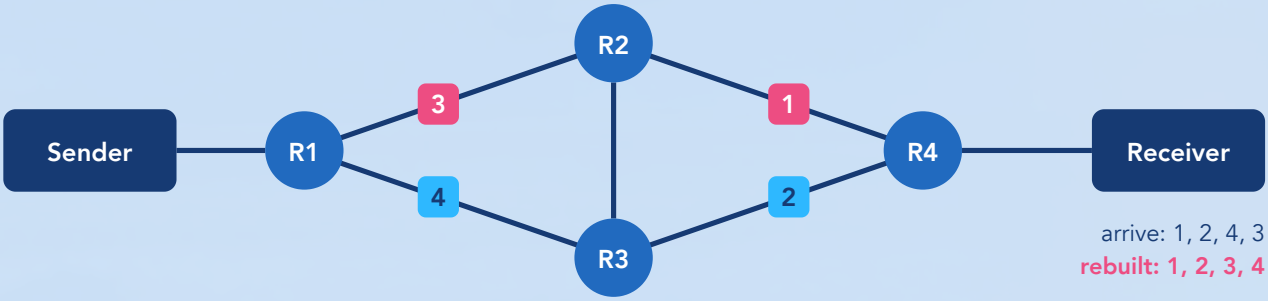
Packet switching

In **packet switching**, a message is split into small blocks called **packets**. Each packet travels through the network **independently**, perhaps by a different route, and the receiver puts the message back together. The Internet works this way.

Inside a packet

Header	Payload	Trailer
Source and destination IP addresses; sequence number; protocol; hop limit	A slice of the message, e.g. 1 500 bytes of a photo	An error check (such as a CRC) and an end-of-packet marker

A photo crosses the network



arrive: 1, 2, 4, 3
rebuilt: 1, 2, 3, 4

A 6 000-byte photo is split into **4 packets** of 1 500 bytes, numbered 1–4. At every **router**, the destination address in the header is read, the routing table is checked and the packet is sent to the best next hop **at that moment**: packets 1 and 3 travel via R2, while 2 and 4 go via R3. They arrive in the order 1, 2, 4, 3, and the receiver uses the **sequence numbers** to rebuild the photo. A lost or damaged packet is simply requested again. Each router also lowers the **hop limit** by 1 and discards any packet that reaches 0, so no packet can circle the network for ever.

Pause & Try

Describe what a router does with each packet it receives. Then explain how the receiver rebuilds a message whose packets arrive in the wrong order, or not at all.

4.8 SWITCHING MECHANISMS: CIRCUIT VS. PACKET SWITCHING

Packet or circuit?

Packet switching carries almost all Internet traffic because, for most data, its benefits outweigh its drawbacks. Here they are, explained one by one, followed by a side-by-side comparison with circuit switching.

Benefits of packet switching

- **Efficient:** links are shared. While you pause, other people's packets use the line, so no capacity is wasted on silences.
- **Robust:** if a link or router fails, the next packets are simply routed around it, and the message still arrives.
- **No set-up delay:** sending starts immediately, as there is no path to reserve first.
- **Cheaper recovery:** only lost or damaged packets are sent again, not the whole message.
- **Cheaper to run:** one shared network carries everyone's data, so nobody pays for a dedicated line.

Drawbacks of packet switching

- **Reassembly:** packets can arrive out of order, so the receiver must hold and reorder them using their sequence numbers.
- **Lost packets:** a busy router may drop packets, which must be noticed and resent, adding delay.
- **Variable delay (jitter):** queues at busy routers make packets arrive at uneven intervals, so live calls can stutter unless the receiver buffers them.
- **Overhead:** every packet carries a header: extra data to send and process.
- **No guarantee:** no user is promised a fixed share of the bandwidth.

Circuit or packet switching?

	Circuit switching	Packet switching
Path	One dedicated path for the whole session	Each packet routed separately
Use of capacity	Low: reserved even when idle	High: links shared by many users
Delay	Low and constant	Varies with traffic (jitter)
If a link fails	The connection drops	Packets are re-routed

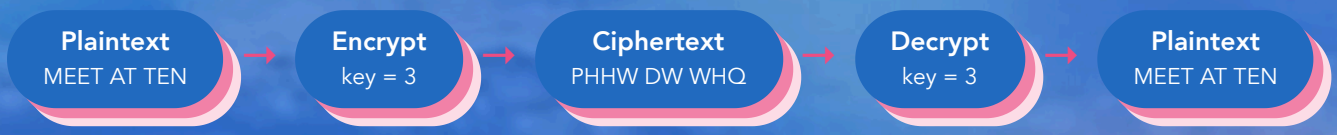
Pause & Try

List two major benefits and one drawback of packet switching, explaining why each one happens. Which switching method would you choose for a live video call across a busy network, and why?

4.9 CRYPTOGRAPHY, QUANTUM SECURITY & DIGITAL CERTIFICATES

How encryption works

Data crossing a network can be intercepted by an **eavesdropper**. Encryption does not stop the interception; it makes what is intercepted **useless**. An **encryption algorithm** uses a **key** to turn readable **plaintext** into unreadable **ciphertext**. **Decryption** uses a key to turn the ciphertext back into plaintext.



Worked example: a Caesar cipher

The algorithm: **shift every letter forward** through the alphabet, wrapping round from Z to A. The key: **how many places**. With key 3, each letter in the top row is replaced by the letter below it:

```
Plain    A B C D E F G H I J K L M N O P Q R S T U V W X Y Z
Cipher   D E F G H I J K L M N O P Q R S T U V W X Y Z A B C
```

So M → P, E → H, T → W, A → D and N → Q: MEET AT TEN becomes **PHHW DW WHQ**. To decrypt, the receiver shifts every letter **back** 3 places. Anyone who intercepts PHHW DW WHQ without knowing the key sees nonsense.

Why real keys are long

A Caesar cipher has only 25 useful keys, so an eavesdropper can simply try them all in seconds: a **brute-force attack**. Modern algorithms such as AES use keys of 128 or 256 bits. A 128-bit key has $2^{128} \approx 3.4 \times 10^{38}$ possible values: even at a trillion (10^{12}) guesses per second, trying them all would take about 10^{19} years, far longer than the age of the universe.

The algorithm itself is usually public; the secret is the **key**. That raises the real question of cryptography: how do the sender and receiver both get the key, without an eavesdropper getting it too? There are two answers: **symmetric** and **asymmetric** encryption.

Pause & Try

a) Encrypt the word CODE with a Caesar shift of 5. b) Decrypt KHOOR, which was encrypted with a shift of 3. c) Explain why a cipher with only 25 possible keys is not secure.

4.9 CRYPTOGRAPHY, QUANTUM SECURITY & DIGITAL CERTIFICATES

Symmetric encryption

In **symmetric encryption**, the **same secret key** is used to encrypt and to decrypt. It is fast and efficient, so it is used for nearly all bulk data: Wi-Fi traffic, encrypted disks and the main part of every HTTPS session.

Encrypting bits with XOR

Computers encrypt bits, not letters. A simple symmetric method combines each bit of the plaintext with a bit of the key using **XOR**, which gives 1 when the two bits differ and 0 when they match.

Step	Bits
Plaintext: the letter H	0100 1000
Key	1011 0011
Ciphertext = plaintext XOR key	1111 1011
Ciphertext XOR the **same** key	0100 1000 = H again

XOR with the same key twice gives back the original, so one key both locks and unlocks the data.

Stream and block ciphers

A **stream cipher** encrypts bits **one after another** as they arrive, like the XOR example: good for live audio and video. A **block cipher** encrypts a **fixed-size block**, e.g. 128 bits, in one go. If identical plaintext blocks gave identical ciphertext, patterns would show, so **block chaining** XORs each plaintext block with the **previous ciphertext block** before encrypting it.

The key distribution problem

Both sides need the same key. If the sender emails the key or sends it across the network, an **eavesdropper** can intercept it and read every message from then on. Meeting in person to swap keys is impossible with millions of websites. This **key distribution problem** is what asymmetric encryption solves.

Pause & Try

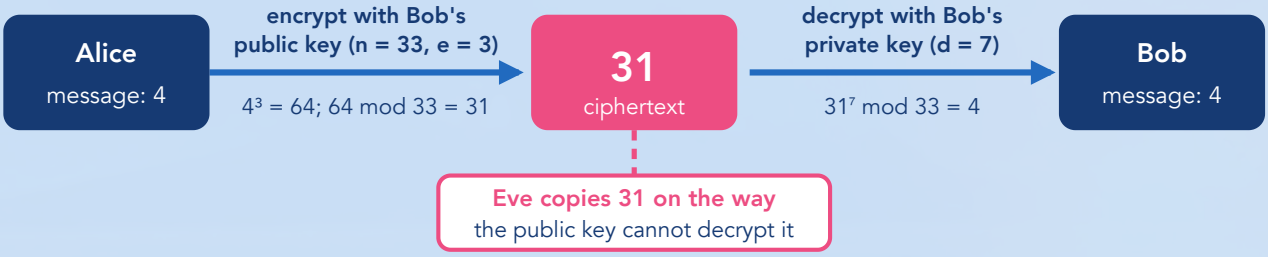
a) Encrypt the byte 0100 0001 (the letter A) with the key 1100 1100 using XOR, then decrypt your answer with the same key. b) Explain the key distribution problem.

4.9 CRYPTOGRAPHY, QUANTUM SECURITY & DIGITAL CERTIFICATES

Asymmetric encryption

Asymmetric encryption uses a **pair** of keys created together: a **public key**, which is shared with everyone, and a **private key**, which its owner never reveals. The keys are mathematically linked: what one key encrypts, **only the other** can decrypt, and the private key cannot be worked out from the public one.

Sending Bob a secret message, with numbers



Bob creates the key pair, keeps the private key secret and publishes the public key, e.g. on his website or in his certificate. Alice encrypts with **Bob's public key**; only **Bob's private key** can decrypt. Real keys are hundreds of digits long; this toy pair uses **mod**, the remainder after dividing (64 mod 33 = 31). To find d, Eve would have to split n into its prime factors (33 = 3 × 11): easy here, but for a real n with over 600 digits no ordinary computer could do it in any useful time.

Symmetric or asymmetric?

	Symmetric	Asymmetric
Keys	One shared secret key	Public key + private key
Key distribution	The key must be sent secretly	The public key can be sent openly
Speed	Fast: good for large amounts of data	Much slower
Typical use	Encrypting the data itself	Sharing a symmetric key; digital signatures

In practice the two work together: asymmetric encryption shares a **symmetric session key** safely, and that fast key then encrypts the data.

Pause & Try

Using Bob's toy public key (n = 33, e = 3), encrypt the number 2. Then use his private key (d = 7) to decrypt your answer and check that you get 2 back. Why can Eve not decrypt it with the public key?

4.9 CRYPTOGRAPHY, QUANTUM SECURITY & DIGITAL CERTIFICATES

Digital signatures

Encryption keeps a message **secret**. A **digital signature** answers two different questions: **who sent** the message, and has it been **changed** on the way? It uses a key pair **in reverse**: the sender signs with their **private** key, and anyone can check the signature with the sender's **public** key.

First, a **hash function** turns a message of any length into a short, fixed-length value called a **hash**. The same message always gives the same hash, but changing even one character gives a completely different hash, and the message cannot be rebuilt from its hash.

Alice signs

1 · Hash
Alice hashes her message

2 · Sign
she encrypts the hash with her private key: this is the digital signature

3 · Send
the message + the signature (+ her certificate)

Bob verifies

4 · Decrypt
Bob decrypts the signature with Alice's public key → hash A

5 · Hash
he hashes the message he received → hash B

6 · Compare
if A = B, Alice sent it and nothing was changed

Why it works, with the toy keys

Alice's toy key pair: public n = 33, e = 3; private d = 7. Her message hashes to **5**. She signs: 5^7 mod 33 = **14**, and sends the message with the signature 14. Bob checks: 14^3 mod 33 = **5**, which matches the hash he calculates himself, so the message is genuine. If Eve changes the message, its hash changes and no longer matches, and she cannot make a new signature without Alice's **private** key.

A signature does not hide the message. To keep it secret as well, Alice also encrypts it with **Bob's** public key.

Pause & Try

Explain how an asymmetric key pair can be used in reverse to create a digital signature. Show how the receiver uses it to verify the sender's identity and to check that the message has not been changed.

4.9 CRYPTOGRAPHY, QUANTUM SECURITY & DIGITAL CERTIFICATES

Digital certificates

A public key is only useful if you know **whose** it is. If Eve could publish her own public key under Alice's name, she could forge Alice's signatures and read her messages. A **digital certificate** solves this: an electronic document in which a trusted **certificate authority (CA)** vouches that a public key belongs to a particular person, company or website.

Inside a certificate

- The owner's name or **domain name**, e.g. www.example.com
- The owner's **public key**
- The name of the issuing **CA**
- A serial number and the **dates** between which it is valid
- The CA's **digital signature** over all of the above

How a website gets one

1 · Key pair
private key stays secret

2 · Apply
public key + domain to a CA

3 · CA checks
the applicant controls the domain

4 · CA signs
with the CA's private key

5 · Install
sent to every visitor

How certificates, public keys and signatures fit together

1. Browsers and operating systems come with the **public keys of trusted CAs** already installed.
2. A website presents its certificate. The browser checks the **CA's signature** on it with the CA's public key, and checks the domain name and dates.
3. If the checks pass, the browser trusts the **website's public key** inside the certificate and uses it to **verify the website's signatures** and set up an encrypted connection.

So the CA's signature vouches for the certificate, the certificate vouches for the public key, and the public key verifies the owner's signatures. A signed email carries the sender's certificate in the same way.

Pause & Try

List the steps an online shop takes to obtain a digital certificate. Then explain how a visitor's browser decides whether to trust the shop's public key.

4.9 CRYPTOGRAPHY, QUANTUM SECURITY & DIGITAL CERTIFICATES

SSL and TLS

Secure Sockets Layer (SSL) and its successor, **Transport Layer Security (TLS)**, protect data travelling between two applications, such as your browser and a web server. SSL is out of date, though people still say "SSL certificate". When you see **https://** and a padlock, TLS is at work.

What TLS provides

- **Encryption:** an eavesdropper sees only ciphertext.
- **Authentication:** the server's certificate proves you are talking to the real website.
- **Integrity:** any change to the data on the way is detected.

Where SSL/TLS is used

- **Banking and shopping:** card and account details.
- **Logging in** to websites and school portals.
- **Email:** SMTP, IMAP or POP3 over TLS.
- **Video calls, messaging** and **cloud storage**.
- **Some VPNs** linking remote workers to an office.

The TLS handshake, simplified

1 · Hello
the browser lists the TLS versions and ciphers it supports

2 · Certificate
the server replies and sends its digital certificate

3 · Check
the browser verifies the certificate with the CA's public key

4 · Session key
using asymmetric cryptography, both sides agree a new symmetric key that only they know

5 · Secure session
all data is encrypted with the session key, which is thrown away at the end

Pause & Try

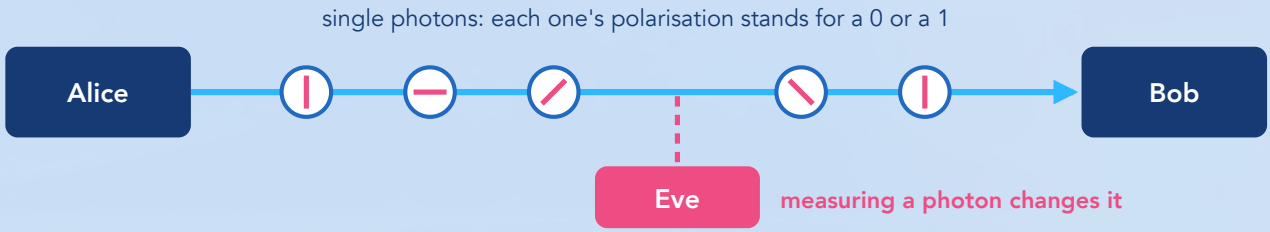
What are SSL and TLS, and in what real-world situations are they used to secure network traffic? Describe what your browser does when it checks a website's certificate.

4.9 CRYPTOGRAPHY, QUANTUM SECURITY & DIGITAL CERTIFICATES

Quantum key distribution

Asymmetric encryption is safe because some maths problems, such as splitting a huge number into its prime factors, would take ordinary computers billions of years; a large enough **quantum computer** could solve them quickly. **Quantum cryptography** relies on the laws of physics instead. Its best-known use is **quantum key distribution (QKD)**: sharing a secret key in a way that reveals whether anyone was listening.

Light as the key



Alice sends the key as single **photons** (particles of light). Each photon's **polarisation**, the direction in which it vibrates, stands for a 0 or a 1, using one of two settings chosen at random. The key rule of quantum physics: **measuring a photon changes it**, and a photon cannot be copied.

Catching the eavesdropper

1. **Send:** Alice sends photons, choosing a random bit and a random setting for each.
2. **Measure:** Bob measures each photon with a setting he chooses at random.
3. **Compare settings:** over an ordinary channel they say which settings they used (not the bits) and keep only the bits where their settings matched.
4. **Test a sample:** they publicly compare some of the kept bits.
5. **Decide:** almost no errors: the rest is their secret key. Many errors: someone listened, so they discard it and start again.

If Eve intercepts the photons, she must measure them without knowing Alice's settings. She guesses wrongly half of the time, which disturbs those photons, so about **1 in 4** of the compared bits disagree; without Eve, almost none do. QKD needs special equipment, works over limited distances (up to a few hundred kilometres of fibre) and only shares keys; messages are then encrypted with a symmetric cipher.

Pause & Try

Explain how QKD lets Alice and Bob detect an eavesdropper. In a sample of 200 compared bits they find 49 that disagree: what should they conclude, and what should they do next?

CHAPTER 4 · PRACTICE

Practice

Practice · Chapter 4

1. Network fundamentals and architectures
    a. Contrast a local area network (LAN) and a wide area network (WAN) in terms of geographic scale, ownership and transmission speeds. [3]
    b. Compare the client-server model with the peer-to-peer (P2P) model regarding resource management and scalability. [3]
    c. Differentiate between a thick client and a thin client based on where data processing and storage take place. [2]
    d. What are the primary trade-offs (in terms of cost, security and control) between using a public cloud and a private cloud? [3]
    e. A small office is setting up a LAN for 15 computers, a printer and Internet access. Name four items of hardware it needs and state the purpose of each. [4]

2. Medium access and collisions
    a. How do network systems handle data collisions when multiple devices attempt to transmit data simultaneously over a shared medium (e.g. using CSMA/CD or collision avoidance techniques)? [4]
    b. Explain why Wi-Fi networks use collision avoidance (CSMA/CA) instead of collision detection. [2]

3. Web infrastructure and addressing
    a. Break down the distinct parts of a uniform resource locator (URL) (e.g. protocol, domain, path) and explain how it locates a specific resource. [3]
    b. Describe the step-by-step role of the Domain Name Service (DNS) when translating a human-readable URL into an IP address. [4]
    c. State two differences between an IPv4 address and an IPv6 address, and give an example of each. [4]
    d. Explain the difference between the Internet and the World Wide Web. [2]

4. Protocol stacks and applications
    a. Outline the four layers of the TCP/IP protocol stack in the correct order (from top to bottom or bottom to top), giving the core responsibility of each layer. [4]
    b. Explain how encapsulation works as data moves down the TCP/IP stack, and what a router does with each packet it receives. [3]
    c. Briefly state the specific purpose of each of these protocols: HTTP, FTP, POP3 and IMAP. [4]
    d. Describe the role of SMTP when an email is sent from one person to another. [2]

CHAPTER 4 · PRACTICE

Practice

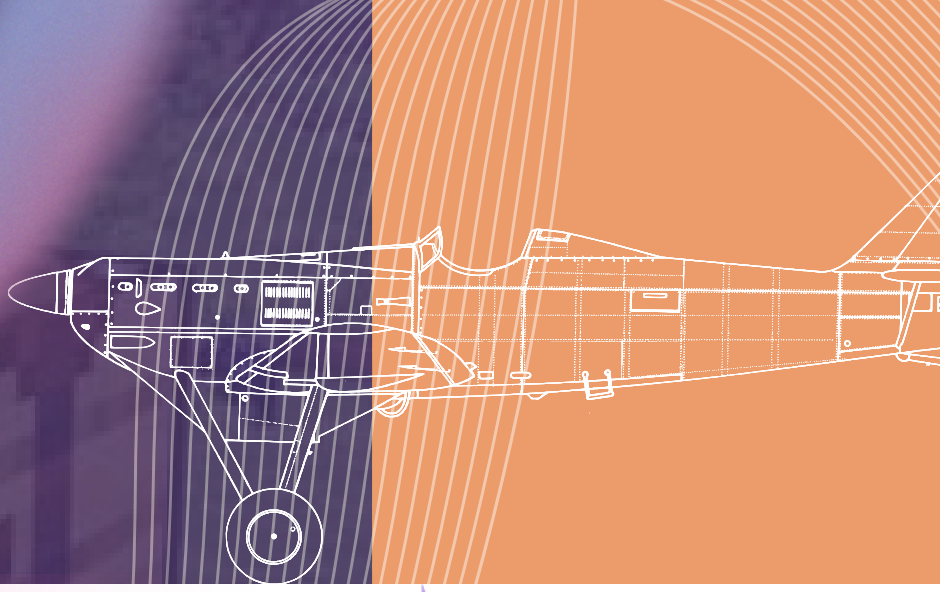
Practice · Chapter 4 (continued)

5. Switching paradigms
   a. Explain what is meant by circuit switching. [2]
   b. There are many applications in which digital data are transferred across a network. Video conferencing is one of these. For this application, circuit switching is preferred. Explain why this is so. [2]
   c. List two major benefits and one drawback of packet switching. [3]

6. Data security, validation and verification
   a. Differentiate between the concepts of data security, data privacy and data integrity. [3]
   b. Contrast the use of validation (e.g. range checks, format checks) with verification (e.g. double entry, parity checks) in protecting data during entry and transfer. [4]
   c. Name three mechanisms used to protect computer systems against threats. [3]
   d. Explain the difference between phishing and pharming. [2]

7. Encryption mechanics and certificates
   a. Compare symmetric key cryptography with asymmetric key cryptography in terms of key management, speed and how plaintext is converted to ciphertext. [4]
   b. Explain how an asymmetric key pair can be used in reverse to generate a digital signature that verifies the sender's identity and ensures message integrity. [3]
   c. A receiver gets a signed message. Describe the steps the receiver takes to verify the digital signature. [3]
   d. What are SSL and TLS, and in what real-world scenarios are they typically deployed to secure network traffic? [3]
   e. Describe how a company acquires a digital certificate from a certificate authority. [4]
   f. Explain how a digital certificate allows a browser to trust a website's public key. [2]

8. Quantum cryptography
   a. Explain why a powerful quantum computer could threaten today's asymmetric encryption. [2]
   b. Describe how quantum key distribution (QKD) allows two users to detect an eavesdropper. [3]



DUSK

APPLICATIONS

5

CHAPTER 5 · BEFORE YOU BEGIN

Learning goals

In the first four chapters you met the module's three technologies one at a time: hardware that senses and acts, networks that carry data, and behaviourist AI that learns from feedback. In real life they almost always work together. This chapter follows them into homes, hospitals, roads, factories and classrooms, shows how they combine into one working system, and looks at what can go wrong when they do.

Learning goals

You will:

- Discover how hardware, networks and behaviourist AI are used in real-world applications
- Understand how physical devices, communication systems and intelligent algorithms work together
- Explain how AI systems respond to feedback and adapt their behaviour to achieve specific goals
- Recognise the opportunities and challenges of integrating these technologies into everyday applications

Your route through the chapter

Subtopic	The question it answers
5.1 Applications of Hardware, Networks and Behaviorist AI in Real-Life Situations	How do the three technologies work together as one system, and how fast must it respond?
5.2 Smart Homes	How do connected devices sense, automate and learn at home?
5.3 Healthcare	How do wearables and intelligent systems watch over patients?
5.4 Transportation	How do traffic systems and vehicles sense, connect and drive themselves?
5.5 Industry & Manufacturing	How do robots and smart factories work, and how can a robot learn from rewards?
5.6 Education	How do smart devices and adaptive systems help people learn?
5.7 Conclusion	How do the three technologies shape everyday life and future innovation?

CHAPTER 5 · BEFORE YOU BEGIN

Key terms

Key terms

Smart system: A system that combines hardware, networks and intelligent software to perform tasks: sensing, deciding and acting with little human help.

Sensor: A device that measures a physical quantity, such as temperature, light or movement, and turns it into a signal a computer can process.

Actuator: A device that turns an electrical signal into physical action, e.g. a motor turning, a valve opening or a heater switching on.

Internet of Things (IoT): Everyday objects fitted with sensors and network connections so that they can share data and be controlled remotely.

Feedback loop: A cycle in which the result of an action is measured and used to choose the next action.

Behaviourist AI: An approach to AI in which behaviour is shaped by interaction with the environment, through fixed stimulus–response rules or by learning from rewards and penalties.

Reactive control: Behaviour produced by fixed stimulus–response rules that do not change with experience.

Reinforcement learning (RL): Learning by trial and error: an agent tries actions and learns which ones earn the most reward over time.

Reward: A positive feedback score telling an agent that an action had a good result.

Penalty: A negative reward telling an agent that an action had a bad result.

Policy: The strategy an agent uses to choose an action in each situation (state).

Real-time processing: Processing data immediately or with very little delay, so that the result arrives before a strict deadline.

Batch processing: Collecting data and processing it all together later, when a delay causes no harm.

Latency: The delay between an event, or a message being sent, and the response; usually measured in milliseconds (ms).

Edge computing: Processing data on or near the device that collected it, rather than on a distant server.

Cloud computing: Storing and processing data on remote servers reached over the internet.

Automation: The use of technology to perform tasks with minimal human involvement.

Wearable device: A small computer worn on the body, such as a smartwatch, that uses sensors to track activity and health.

Remote patient monitoring: Collecting a patient's health readings at home and sending them over a network for doctors and nurses to check.

Connected vehicle: A vehicle that exchanges data with other vehicles (V2V) and with roadside equipment such as traffic lights (V2I).

Autonomous vehicle: A vehicle that uses sensors, onboard computers and actuators to drive itself with little or no human input.

Predictive maintenance: Using sensor data to predict when a machine is likely to fail, so that it can be repaired before it breaks.

Smart factory: A factory whose machines, robots and sensors are networked so that production can be monitored and adjusted automatically.

Cobot: A collaborative robot designed to work safely alongside people.

Adaptive learning: Educational software that changes the difficulty and content for each learner based on their answers.

Human-AI interaction (HAI): The ways humans communicate and interact with AI-powered systems.

Black box: An AI system whose internal decision-making cannot easily be understood, even by its developers.

Ethical AI: The development and use of AI in ways that are fair, responsible and transparent.

Smart city: A city that uses connected sensors, networks and AI to manage services such as traffic, energy, water and waste.

5.1 APPLICATIONS OF HARDWARE, NETWORKS AND BEHAVIORIST AI IN REAL-LIFE SITUATIONS

Where the three technologies meet

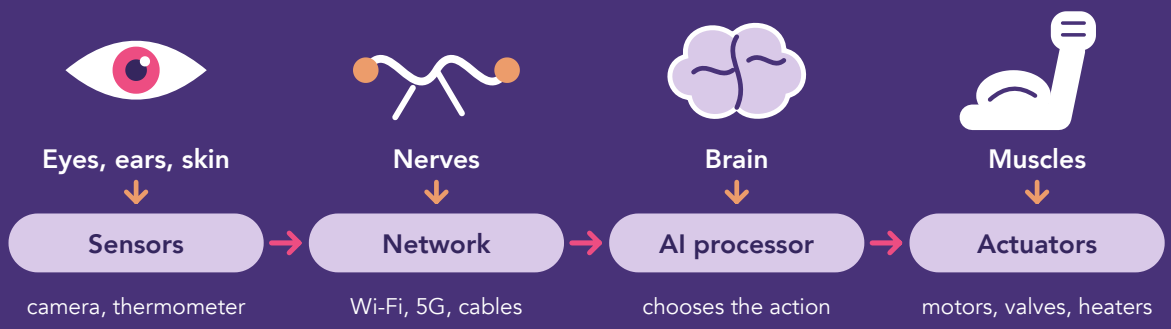
In Chapters 1–4 you studied hardware, networks and behaviourist AI separately, but in the real world you rarely meet them alone. Think about a robot vacuum cleaner. Its bump and cliff sensors detect obstacles and stairs (hardware). It reports its progress to an app over Wi-Fi (network). Its software chooses what to do when it hits a chair leg (behaviourist AI), and its wheel motors carry out that choice (hardware again). A machine that combines the three technologies in this way is called a **smart system**.

What makes a system "smart"?

A **smart system** combines hardware, networks and intelligent software so that a machine can sense its surroundings, share information, decide what to do and act, often within milliseconds and with little or no human help. Thermostats, watches, cars, factories and whole cities all contain smart systems, and every one has the same four parts:

- **Sensors** measure the physical world: temperature, light, movement, distance.
- A **network** carries readings to wherever the deciding happens, and carries commands back.
- A **processor** running software or AI decides what to do.
- **Actuators** turn the decision into physical action: a motor turns, a valve opens.

Analogy: a smart system works like a body



Like a body, a smart system senses, passes signals along, decides and moves. The analogy describes the *roles* only: the machine does not think or feel like a person.

Pause & Try

Choose one electronic device in your home, such as a smart thermostat, a robot vacuum cleaner or a smartphone. Identify a) the sensors it uses to gather real-world data, b) how it communicates across a network, c) the actuator or output action it performs.

5.1 APPLICATIONS OF HARDWARE, NETWORKS AND BEHAVIORIST AI IN REAL-LIFE SITUATIONS

Sense, connect, decide, act

To see the four parts working together, follow one example from start to finish. On a large farm, watering on a fixed timer wastes water on wet days and leaves crops thirsty on hot ones. A smart irrigation system waters each zone only when it needs it.

One trip around the loop on a smart farm

Stage	What happens
1 · Sense	soil-moisture **sensors** measure how wet the ground is in every zone
2 · Connect	a long-range, low-power **network** (LoRaWAN) carries the readings to the farm's computer
3 · Decide	**software or AI** combines the readings with the forecast to decide each zone's water
4 · Act	solenoid valves (**actuators**) open Zone A's drip lines for 12 minutes; Zone B stays closed

Acting changes the environment, so tomorrow's readings show whether 12 minutes was the right amount, and the cycle repeats. A cycle in which the result of each action is measured and used to choose the next one is called a **feedback loop**. The *decide* step can work in two quite different ways:

A fixed rule

A programmer writes: *if moisture is below 30%, open the valve for 10 minutes.* The rule is simple, predictable and easy to check, but it waters even when rain is forecast, and it never improves.

A learned policy

A **reward** for healthy crops and a small **penalty** for every litre used teach it, over many weeks, to grow the same harvest with less water. It needs data and testing before it can be trusted.

Being connected does not make a system learn: it only *learns* when feedback about its results changes what it does next time. A fixed rule is **reactive control**; a policy improved by rewards and penalties is **reinforcement learning** (Chapter 2).

Pause & Try

For the smart farm, give one advantage of a learned watering policy over a fixed rule, and one reason a farmer might still prefer the fixed rule.

5.1 APPLICATIONS OF HARDWARE, NETWORKS AND BEHAVIORIST AI IN REAL-LIFE SITUATIONS

Deciding in time

Some computer jobs can wait. A bank can work out every customer's monthly statement overnight in one large batch, and nobody is harmed if it takes an hour: this is **batch processing**. A smart system that controls something physical is different, because its answer is only useful if it arrives in time. **Real-time processing** means processing data immediately, or within a strict time limit called a **deadline**. Real-time does not simply mean "fast": it means *fast enough, every time*.

How strict is the deadline?

Type	If the deadline is missed …	Examples
Hard real-time	the system fails; people or equipment may be harmed	car airbag, anti-lock brakes
Soft real-time	quality drops, but nothing is damaged	video call, online game
Not real-time (batch)	the result simply arrives later	bank statements, overnight back-ups

Every stage of the feedback loop takes time: the sensor must measure, the network must carry the data, the processor must compute and the actuator must move. The delay added by the network is called **latency**. It is usually measured in milliseconds (ms); one millisecond is a thousandth of a second.

Worked example: adding up the delays

A safety system must stop a robot arm within **100 ms** of a person stepping too close. Each stage of the loop adds delay:

Sense 2 ms → **Send** 20 ms → **Decide** 10 ms → **Act** 50 ms

Total: 2 + 20 + 10 + 50 = **82 ms**, so the deadline is met with 18 ms to spare. If a busy network raised the latency from 20 ms to 45 ms, the total would become 107 ms and this hard deadline would be missed. That is why engineers keep slow network links out of safety-critical loops wherever they can.

Pause & Try

Classify each application as hard real-time, soft real-time or not real-time, and justify one of your answers: a) an aircraft's fly-by-wire control system, b) streaming a film, c) a heart pacemaker, d) processing weekly payroll.

5.1 APPLICATIONS OF HARDWARE, NETWORKS AND BEHAVIORIST AI IN REAL-LIFE SITUATIONS

Edge or cloud?

Where should the deciding happen? In **cloud computing**, data travels over the internet to a powerful remote data centre and the answer travels back, which adds latency and makes the system depend on its connection. In **edge computing**, a processor on the device itself, or very close to it, does the work; chips such as GPUs and NPUs (Chapter 3) let small devices run AI quickly.

Comparing cloud and edge

	Cloud computing	Edge computing
Where it happens	remote data centres, often far away	on the device or a nearby gateway
Typical network delay	about 50–200 ms	about 1–10 ms
Data sent	a lot: continuous raw readings	a little: results and summaries
If the link fails	the system stops	it keeps working locally
Best for	heavy work that can wait, e.g. training AI	split-second decisions, e.g. braking

Edge and cloud are partners, not rivals. A campus delivery robot steers round people using its own computer, because that cannot wait. Over the network it receives delivery jobs, uploads logs that engineers use to retrain its AI, and downloads improved, tested policies.

What the network carries

Every message needs context: which device sent it, when (a **timestamp**) and in what order (a **sequence number**), so the robot can reject a command that is too old or already carried out. Context matters for learning too: if the report on action 42 arrives after the one on action 43, sequence numbers match each result to the right action. A lost report means the result is unknown, not a reward of zero.

Watch the tutorial: Networking for behaviorist AI (Video 04)

https://github.com/user-github-me/cs-ai-project-01/blob/main/assets/tutorials/stage2/videos/BlueImpulse_Video04_Networking_for_Behaviorist_AI.mp4

Pause & Try

A delivery robot must a) stop for a pedestrian, b) plan tomorrow's routes for the fleet, c) retrain its policy on a week of logs. Should each job run on the robot (edge) or in the cloud? Justify one answer.

5.2 SMART HOMES

The connected home

A **smart home** is a home whose everyday devices, such as lights, heating, locks, cameras and speakers, are networked so that they can be controlled from an app, by voice or automatically. Each device is a small smart system, and together they form part of the **Internet of Things (IoT)**.

One hub links the home's lights, cameras, heating and locks.

Devices connect in different ways. A camera streams video, so it needs the high data rate of **Wi-Fi**. A door sensor sends a few bytes and must run for a year on a small battery, so it uses a low-power network such as **Zigbee** or **Thread**, in which mains-powered devices pass messages on to form a **mesh** that reaches every room. A **hub**, or the router itself, links everything to the internet.

Door sensor the door opens → **Hub** Zigbee to Wi-Fi → **Cloud server** runs your routine → **Phone app** "Front door opened"

The home's senses

Sensor	What it detects	Example use
Temperature	how warm the air is	heating control
Passive infrared (PIR)	body heat from a moving person	lights on as you walk in; alarm
Magnetic contact	a door or window opening	security alert
Smoke / carbon monoxide	smoke particles or CO gas	alarm and phone alert
Microphone	sound	hears a voice assistant's wake word

On the output side, **actuators** do the work: a **relay** (an electrically controlled switch) turns the boiler or a lamp on and off, **motors** move a lock's bolt or the blinds, and a **speaker** sounds an alarm.

Pause & Try

A smart video doorbell has a camera, a PIR motion sensor, a speaker and a Wi-Fi chip. Classify each part as a sensor, an actuator (output) or a network component, and say what it does when a visitor arrives.

5.2 SMART HOMES

Homes that act on their own

Connecting devices lets you control them from anywhere, but a home only becomes truly smart when it can act without being told. Smart-home automation comes in three levels, and only the last one learns:

Remote control
you tap the app and the light comes on

↓

Fixed routine
IF motion is detected after sunset THEN switch on the hall light

↓

Learning automation
the system notices your habits and adjusts its own behaviour

A **fixed routine** is the reactive control of Chapter 2: the same stimulus always triggers the same response. Many robot vacuum cleaners work this way: when the bumper is pressed they reverse and turn, when the cliff sensor finds no floor they back away from the stairs, and when the battery is low they follow the dock's infrared signal home. The rules never change, yet together they clean the room.

Worked example: a learning thermostat

A learning thermostat treats every change you make as **feedback**. From Monday to Thursday you turn the heating up from 17 °C to 21 °C at about 06:30, and each time it records the day, time and setting. By Friday it has spotted the pattern and starts heating at 06:00 by itself, so the house is already 21 °C when you get up. If you turn it down again, that correction works like a **penalty** for its last choice and the schedule shifts; a day with no corrections suggests it got things right.

The cost of connection

Every connected device is a possible way in for an attacker. In 2016 the **Mirai** malware took over hundreds of thousands of cameras and routers that still used their factory passwords and used them in a DDoS attack that made websites such as Twitter and Netflix hard to reach for hours. Smart speakers and cameras also collect private data. To protect a smart home, change default passwords, install updates, use encryption and choose devices that collect only the data they need.

Pause & Try

Explain how a learning thermostat uses feedback from the people in a home to change its behaviour. Then give one security risk of connecting it to the internet and one way to reduce that risk.

5.3 HEALTHCARE

Wearables and patient monitoring

Healthcare is one of the most valuable places to use smart systems, and one of the most sensitive: the environment being sensed is a human body, the data is private and a wrong decision can harm someone. So most systems *support* doctors and nurses rather than replace them.

What a smartwatch senses

A **wearable device** is a small computer worn on the body, such as a smartwatch, a fitness band or a skin patch.

- **Optical heart-rate sensor:** green LEDs shine into the skin and a light sensor measures how much light comes back. Blood absorbs green light, so each heartbeat changes the reading.
- **Accelerometer and gyroscope:** measure movement to count steps, track sleep and detect a hard fall; some watches then call the emergency services if the wearer does not respond.
- **ECG sensor:** on some watches, records the heart's electrical signal to check its rhythm.

The watch processes its raw readings itself (edge computing) and sends only summaries, which saves battery power and network bandwidth:

Watch measures and summarises → **Bluetooth** to the phone → **Mobile network** encrypted upload → **Clinic dashboard** a nurse reviews

Remote patient monitoring

People with long-term conditions can be monitored at home. A heart-failure patient might weigh themselves each morning on a connected scale. A gain of about 2 kg in three days can mean fluid is building up, so the system flags the patient and a nurse phones that day, instead of the patient arriving at hospital weeks later as an emergency.

In hospital, bedside monitors send heart rate, blood pressure, oxygen level and breathing rate over the ward network to the nurses' station. Fixed limits, such as *alarm if the heart rate is above 120 beats per minute*, cause so many false alarms that staff can tune them out (**alarm fatigue**), so smarter systems combine several readings and trends to raise fewer, more meaningful alerts.

Pause & Try

Trace one heart-rate reading from a patient's smartwatch to a nurse's screen, naming the hardware or network used at each step. Then explain one benefit of processing the readings on the watch.

5.3 HEALTHCARE

Intelligent health systems

Monitoring shows what is happening; **intelligent health systems** use AI to decide what should happen next, either acting automatically in a feedback loop or advising a doctor.

A feedback loop in the body: the artificial pancreas

People with type 1 diabetes must take insulin to control the sugar (glucose) in their blood. A **hybrid closed-loop system** does much of this automatically:

Glucose sensor every few minutes → **Bluetooth** sends the reading → **Algorithm** decides the dose → **Insulin pump** delivers it

Each reading shows how the body responded to the last dose, and this feedback lets the algorithm learn the person's insulin needs. The user still enters meals, so the system is called *hybrid*.

AI in the emergency room

A busy emergency department can take hundreds of chest X-rays a day. An AI system trained on thousands of earlier images can check each one in seconds, mark areas that look abnormal and move urgent cases to the top of the doctor's list. The doctor still makes the diagnosis; the AI adds a fast, tireless extra check. But it only recognises patterns like those in its training data.

This kind of AI learns from labelled examples rather than from rewards: not every AI is behaviourist.

Emergency departments are busy day and night.

Many AI systems are **black boxes**: their outputs come from millions of calculations that even their developers cannot easily explain. If an alert just says *High risk*, a doctor may trust it blindly (**over-reliance**) or ignore it (**under-reliance**). A transparent system shows its evidence: the area that triggered the alert and how confident it is.

Pause & Try

An AI system scans chest X-rays for signs of lung illness in a busy hospital emergency room. a) Identify two major efficiency advantages of using it. b) Identify one potential risk if the AI meets an unusually rare chest deformity that it has never seen before.

5.4 TRANSPORTATION

Smart traffic, connected cars

Roads are a shared system: thousands of vehicles, traffic lights and people all affect one another. Smart transport uses sensors and networks to see the whole picture, and AI to keep traffic moving safely.

Smart traffic systems

A traditional traffic light runs on a **fixed timer**, even at 3 a.m. when the roads are empty. An **adaptive traffic signal** changes its timing to match the traffic: wire loops buried in the road detect waiting cars, cameras and radar measure queues, and fibre or mobile networks carry the readings to a controller that sets each green light.

Worked example: sharing out the green light

At a junction with a 60-second cycle, a fixed timer gives each road 30 seconds of green (ignoring the amber phases). At 08:00, cameras count 18 cars queuing on the main road and 6 on the side road. Sharing the 60 seconds in proportion to the queues gives the main road 18 ÷ 24 × 60 = **45 seconds** and the side road 6 ÷ 24 × 60 = **15 seconds**, so the long queue clears and nobody waits at a red light beside an empty road.

Researchers also train traffic signals with **reinforcement learning**. The **state** is the length of the queue on each road, the **action** is which road gets the next green light, and the **reward** is the total waiting time made negative, so shorter queues earn a higher reward. Over thousands of simulated rush hours, the controller learns timings that keep queues shorter than a fixed timer can.

Connected vehicles

A **connected vehicle** exchanges data over wireless networks with other vehicles (**V2V**), with roadside equipment such as traffic lights (**V2I**), and with everything around it (**V2X**). A car approaching a junction can be told how many seconds remain before the light turns red. Since 2018, every new model of car approved in the EU has had to include **eCall**, which automatically dials the emergency number 112 after a serious crash and sends the car's location.

Car ahead brakes hard in fog → **V2V message** arrives in milliseconds → **Car behind** warns its driver at once

Pause & Try

Explain how an adaptive traffic signal uses sensors, a network and software to reduce waiting times compared with a fixed timer. Then suggest one extra penalty you would add to the reward of a traffic light trained with reinforcement learning, and explain why.

5.4 TRANSPORTATION

Self-driving cars and latency

An **autonomous vehicle** drives itself by running the sense–decide–act loop many times every second. **Cameras** read lane markings, signs and traffic lights. **Radar** measures the distance and speed of other vehicles, even in fog or heavy rain. **Lidar** fires laser pulses and times their reflections to build a 3D map of everything nearby. **GPS** and motion sensors track the car's own position. An onboard computer with GPU and AI-accelerator chips combines these readings, decides what to do and sends commands to the **actuators**: an electric motor that turns the steering, and the systems that brake and accelerate. As a car covers many metres every second, *when* a decision arrives matters as much as *what* it is.

Worked example: speed, latency and braking

A car travels at **108 km/h** when a child runs into the road. How far does it travel before the brakes are applied?

Step 1 — convert the speed. 108 km/h = 108 000 m ÷ 3600 s = **30 m/s**.

Step 2 — use distance = speed × delay.

Hard braking: what matters is how soon it starts.

Who or what decides	Delay	Distance before braking
an alert human driver	about 0.67 s	30 × 0.67 ≈ **20 m**
AI over a slow cloud link	500 ms = 0.500 s	30 × 0.500 = **15 m**
AI over a fast cloud link	50 ms = 0.050 s	30 × 0.050 = **1.5 m**
AI on the car's own (edge) computer	5 ms = 0.005 s	30 × 0.005 = **0.15 m**

Conclusion: deciding on board instead of over the slow cloud link saves 15 − 0.15 = **14.85 m**, more than three car lengths. This is only the distance travelled *before* braking begins; the car then needs extra distance to stop. (The human figure uses the 0.67 s thinking time assumed by the UK Highway Code.)

That is why safety-critical decisions are made at the edge, inside the vehicle, while the cloud handles jobs that can wait: updating maps, collecting data from the whole fleet and training better AI models.

Pause & Try

A bus travels at 54 km/h. a) Convert this speed into metres per second. b) Calculate how far it travels before braking begins if the decision takes 300 ms over a cloud link, and if it takes 4 ms on an onboard computer. c) Explain why the bus still needs a network connection.

5.4 TRANSPORTATION

Who is in control?

Automation (technology doing tasks with minimal human involvement) comes in levels:

Six levels of driving automation (SAE J3016 standard)

Level	The car …	The human …
0 · None	only warns, e.g. of a collision	drives all the time
1 · Assistance	steers *or* controls speed	drives and supervises
2 · Partial	steers *and* controls speed	watches the road constantly
3 · Conditional	drives itself in limited conditions	takes over when asked
4 · High	drives itself within a defined area	is not needed in that area
5 · Full	drives anywhere, in any conditions	is never needed

Adaptive cruise control is Level 1; driverless taxis are Level 4; no Level 5 car exists yet.

As automation rises, the human shifts from **operator** to **supervisor**: from driving a manual car (traditional **human–computer interaction**, HCI) to supervising a self-driving taxi (**human–AI interaction**, HAI).

HCI and HAI compared

	Traditional HCI	Human–AI interaction
Behaviour	fixed: same input, same output	adapts to what it senses and learns
Human's role	operator giving every command	supervisor setting goals, checking
Unclear input	shows an error	makes a best guess, maybe wrong

HAI goes wrong when people misjudge the machine: with **over-trust** a Level 2 driver stops watching the road, and with **under-trust** a driver switches off a useful safety feature. In 2009, ice blocked the airspeed sensors of Air France Flight 447; the autopilot handed control to pilots who did not understand why, and their confused responses led to a fatal crash. Automation must show clearly what it is doing, and why.

Pause & Try

Compare driving a traditional manual car with supervising a self-driving taxi. a) In which situation is the human giving continuous HCI inputs? b) What new responsibilities does the human have in the supervisory HAI role?

5.5 INDUSTRY & MANUFACTURING

Robots and automated production

Factories were among the first places to use automation, and today they contain some of the most advanced smart systems. An **industrial robot** is a programmable machine, usually an arm with several joints, that can weld, paint, lift, sort and assemble products with the same precision thousands of times a day. Robots do not get tired or distracted, so a production line can run day and night.

From a decision to a movement

Inside the robot's computer, a decision is only a number, such as *rotate joint 2 to 90°*. The controller sends this target to a **driver circuit**, which turns it into a control signal (a voltage or PWM pulses) and switches the power the motor needs. The **actuator**, a motor, turns the joint, and an **encoder** measures the new angle and reports it back:

Controller target 90° → **Driver circuit** signal and power → **Motor** turns the joint → **Encoder** measures 87°

The controller compares the two angles: 90° − 87° leaves a 3° error, so it drives the motor a little further. This feedback loop runs hundreds of times a second.

Actuators in a factory

Actuator	How it works	Typical use
Electric servo motor	magnetic fields produce precise rotation	robot-arm joints, conveyor belts
Solenoid valve	an electromagnet slides a plunger open or shut	dosing liquids, switching air lines
Hydraulic piston	pressurised oil produces a very large force	presses, heavy lifting
Pneumatic actuator	compressed air gives fast, repeated movement	high-speed pick-and-place grippers

Automation also changes the kind of mistakes a factory makes. A tired human inspector might miss the odd faulty part; a robot does not get tired. But if a camera is knocked out of line or the software has a bug, a robot repeats the *same* mistake on every product until someone notices, so automated lines still need people to monitor them.

Pause & Try

A robot arm is told to rotate a joint to 45°, but its encoder reports 41°. a) Explain what the controller does next. b) Explain why the arm needs both an actuator and a sensor to place parts accurately.

5.5 INDUSTRY & MANUFACTURING

Smart factories, learning robots

In a **smart factory**, every machine is networked and sensors stream data to software that watches the whole line. In **predictive maintenance**, the AI notices a motor's vibration rising from 2 mm/s to 6 mm/s over three weeks as its bearings wear, so engineers replace the bearing in a planned stop, not after a breakdown. **Cobots** (collaborative robots) work beside people and stop if they touch someone.

When every object is different, as in a warehouse full of parcels, hand-programming each move is impossible, so engineers use **reinforcement learning**: the robot learns from the results of its actions.

The learning loop

- **State (*S*):** what the sensors report now, e.g. a camera image of the bin.
- **Action (*A*):** what the robot does: move, rotate, open or close its gripper.
- **Reward (*R*):** positive for success, negative (a **penalty**) for failure.
- **Policy (π):** the robot's strategy for choosing actions; learning makes actions that earned high rewards more likely.

In 2018, seven Google robot arms practised grasping; after about 580 000 attempts they could pick up objects they had never seen about 96% of the time.

A reward is the robot's trophy: a number saying how well an action went.

Worked example: the reward decides what "good" means

A warehouse robot moves items into a box. Rewards: item placed **+10**; each second taken **−0.5**; item dropped **−20**; collision **−50**.

Attempt A is careful: 12 s, success. Reward = +10 − (0.5 × 12) = **+4**.
Attempt B is fast: 6 s, but clips a shelf. Reward = +10 − (0.5 × 6) − 50 = **−43**.

Over thousands of attempts the robot learns that careful, collision-free moves pay best; reward speed alone and it would learn to rush and crash. Robots usually practise first in a **simulation**, and safety limits such as emergency stops never depend on the reward.

Pause & Try

You are training a behaviourist AI drone to deliver medicine to rural clinics safely. Define a) the sensor data that forms its state, b) one event that earns a positive reward, c) one event that earns a heavy penalty.

5.6 EDUCATION

Smart classrooms

In education, the three technologies help people learn: they show teachers at once what every student understands, and let students work at their own pace.

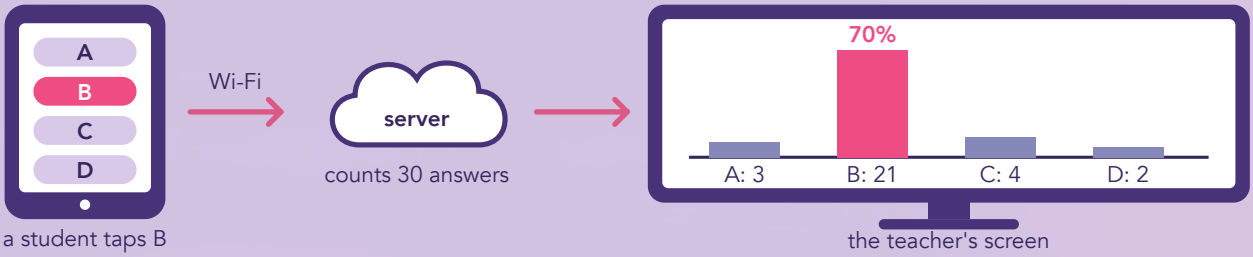
Smart learning devices

- **Tablets and laptops:** touchscreens, cameras and microphones collect every student's work.
- **Interactive whiteboards:** large touch-sensitive displays that detect a pen or finger.
- **Microcontroller kits:** a BBC micro:bit has buttons, a motion sensor, a light sensor and LEDs, so students can build small smart systems of their own.
- **VR headsets:** motion sensors track your head as you explore a virtual volcano.

Connected classrooms

Devices connect through the school's Wi-Fi to **cloud** learning platforms, where teachers set work, students hand it in and everything is saved automatically. A live quiz shows the network in action:

Worked example: a live quiz



The answers travel over Wi-Fi to a cloud server, which counts all 30 in seconds and draws a chart for the teacher. Answer B was wrong, yet 21 ÷ 30 = 70% of the class chose it, so the teacher knows at once to explain the idea again: feedback that would take days with a paper test.

Schools can also become **smart campuses**. In 2021, state schools in England were sent carbon dioxide (CO_2) monitors that show when a crowded classroom needs fresh air. Occupancy sensors can switch off lights and heating in empty rooms, and software that reads the timetable can warm only the rooms that are about to be used.

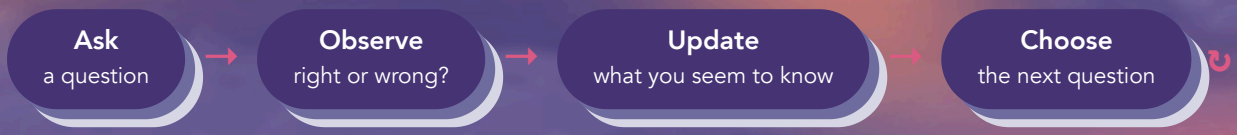
Pause & Try

Imagine turning your school, college or university into a smart campus. Identify two areas, such as library seating, cafeteria food waste or classroom heating, where sensors, a network and AI software could reduce waste. For one of them, name the sensor, the network and the action taken.

5.6 EDUCATION

Adaptive learning

Behaviourism shaped education long before computers. In the 1950s B. F. Skinner, whose operant conditioning you met in Chapter 2, built **teaching machines** that showed material in small steps and told the student at once whether each answer was right; that instant confirmation acted as reinforcement. **Adaptive learning** uses the same idea, with AI choosing what each learner sees next.



Feedback flows both ways: *you* get feedback on each answer, which reinforces correct responses, and the *system* learns whether its question was too easy, too hard or just right.

Worked example: a levelling rule

A maths app has five levels and one rule: **3 right answers in a row → up one level; 2 wrong in a row → down one level, with a hint.** A student starts on level 2:

Answers	Results	Level afterwards
1–3	right, right, right	3 (three right in a row: up)
4–5	wrong, wrong	2 (two wrong in a row: down, with a hint)
6	right	2 (a new run begins)

This rule never changes, so it is reactive control. A **learning** system uses thousands of students' results to find which question helps most, rewarded when a learner gets a similar question right a week later.

Fairness and privacy

Adaptive systems record every answer and mistake, so schools must protect that data and collect only what they need. Algorithms can also be unfair: in 2020 an algorithm used to award A-level grades in England lowered about 4 in 10 of teachers' predicted grades, and students at large state schools lost out more than those at private schools. After protests, teachers' grades were used instead.

Pause & Try

Using the same rule, a student starts on level 3 and answers: right, wrong, right, right, right, wrong, wrong. a) Which level does the student finish on? b) Explain how a learning (reinforcement learning) version of the app would differ from this fixed rule.

5.7 CONCLUSION

The smart city: everything together

Each section of this chapter looked at one setting. A **smart city** joins them all: homes, hospitals, roads, factories and schools share networks and data, so that a whole city can sense what is happening and respond. Thousands of IoT sensors report to AI systems that manage the resources everyone shares: roads, energy, water and clean streets.

A smart energy grid: solar panels, wind turbines and batteries balanced by networked software.

Worked example: a smart grid

On a hot afternoon, air conditioners push electricity demand towards a peak. Smart meters send readings every few minutes over the network to the grid's AI, which predicts the spike and tells large batteries, charged by solar panels during the morning, to release their stored energy. Supply and demand stay balanced without anyone pressing a button.

How a city senses and acts

City system	Sense → connect → decide → act	Benefit
Adaptive traffic signals	cameras and road sensors send traffic data over 5G; AI sets the green times	shorter queues, less idling and pollution
Smart water network	acoustic sensors on water mains hear small leaks; valves close to isolate the pipe	less clean water lost, cheaper repairs
Smart street lighting	motion sensors dim lights on empty streets and brighten them as people approach	lower energy use
Smart waste collection	bins report how full they are; software plans the lorries' routes	fewer trips, less fuel and traffic

Smart cities show both sides of this chapter at once. They can save energy, water, time and lives, but they depend on huge amounts of data about where people go and what they do, and a single attack on their networks could affect a whole city.

Pause & Try

Choose one city system from the table. Name its sensor, its network and its actuator, and explain what the AI decides. Then suggest one way the system could fail and how the city could reduce that risk.

5.7 CONCLUSION

Opportunities and responsibilities

Every application in this chapter brings benefits and new risks. Once, security teams protected only servers and desktop computers; now every connected lock, camera, sensor and medical pump is another way in for an attacker, and many cheap devices ship with weak passwords. Smart systems also collect data constantly, about where people go and how they live, which can be stolen, sold or misused.

One technology, two sides

Technology	Opportunity	Security or privacy risk
Health monitors	early warning of emergencies	stolen records; altered doses
Home cameras	intruder alerts from anywhere	hijacked video feeds
Delivery robots	fast deliveries, lower emissions	GPS spoofing misdirects them
City sensors	less wasted energy and water	tracking where everyone goes

Beyond security lies a second question: are the decisions right? **Ethical AI** means developing and using AI in ways that are fair, responsible and transparent:

The three pillars of ethical AI

Pillar	What it means	Why it is hard
Fairness	people and groups are treated equally	AI repeats bias hidden in past data, as the 2020 grading algorithm showed
Transparency	people can find out how a decision was made	many AI systems are black boxes
Accountability	someone is clearly responsible when things go wrong	a hardware maker, software developer, network provider and user all play a part

The law is beginning to answer the accountability question. Under the UK's Automated Vehicles Act 2024, while an approved self-driving system is driving, the company behind it is responsible, not the person in the seat.

Pause & Try

A city wants cameras at every bus stop to count waiting passengers and send extra buses. Give one opportunity and one privacy risk, then suggest one way to make the plan fairer or more transparent.

5.7 CONCLUSION

Shaping everyday life

Across every setting in this chapter the pattern is the same. **Hardware** senses the world and changes it, **networks** carry information between devices, people and the cloud, and **behaviourist AI** chooses actions and improves them from feedback. Remove any one and the system fails: without sensors it is blind, without a network it is cut off, and without intelligent software it cannot adapt.

One pattern, five settings

Setting	Hardware	Network	Behaviourist AI
Smart home	thermostat, relay	Wi-Fi, Zigbee	learns your heating times
Healthcare	glucose sensor, insulin pump	Bluetooth	adjusts doses from feedback
Transportation	cameras, radar, brakes	V2V, V2I, 5G	signals rewarded for short queues
Industry	robot arms, encoders	factory network	robots learn to grasp
Education	tablets, whiteboards	Wi-Fi, cloud	picks the next question

Think about a single day: the thermostat warms the house before you wake, adaptive signals speed up your bus, a live quiz shows your teacher what to explain again, and your watch tracks your sleep. Each time, hardware senses, a network connects and software decides, usually without you noticing:

07:00 a warm house → **08:15** green lights → **10:30** a live quiz → **23:00** sleep tracking

Future innovations

Faster networks, cheaper sensors and more efficient AI chips will move more intelligence to the edge, close to where things happen. Researchers are already building prosthetic hands that adapt to the way their wearer moves, farm robots that weed a field plant by plant instead of spraying it all, and power grids that balance wind and solar energy automatically. Each of these is a feedback loop of sensing, connecting, deciding and acting. However clever the machines become, people still choose their goals and the rewards they learn from, so understanding all three technologies, and their limits, matters for everyone. You can try the loop yourself in **RoverLab**: build a machine, connect its network and train its behaviour.

Pause & Try

Pick the application from this chapter that you would trust least. Using fairness, transparency and accountability, explain what would need to change before you would trust it.

CHAPTER 5 · PRACTICE

Practice

Practice · Chapter 5

1. Define a smart system and explain how it integrates physical hardware, network connectivity and software into a unified workflow.

2. Real-time processing and automation
   a. Define real-time processing and explain why low network latency and high-speed hardware are critical for systems like autonomous vehicle collision avoidance.
   b. An autonomous car travels at 90 km/h. Calculate how far it travels before braking begins if the decision takes 400 ms over a cloud connection, and if it takes 8 ms on an onboard computer. Show your working.
   c. What is automation, and how does moving from human-operated control to fully automated smart systems alter operational efficiency and error rates?

3. Feedback and action
   a. Explain how a behaviourist AI system uses performance feedback or reward signals from its environment to modify its behavioural policy.
   b. Describe the role of actuators in translating an AI model's computed decision into physical changes within the real world.

4. Humans and AI
   a. Define human-AI interaction (HAI) and explain how it differs from traditional human-computer interaction (HCI).
   b. Identify a major usability or safety challenge that occurs when a smart system's autonomous behaviour lacks transparency, leaving the human user confused about its next action.
   c. An AI system alerts a doctor: "High risk of patient sepsis – transfer to ICU immediately." Suggest how the alert screen could be redesigned so that the doctor can check it within 5 seconds, listing three pieces of evidence it should display.

5. Ethics and society
   a. Define ethical AI and discuss why transparency, fairness and accountability are difficult to maintain in complex smart systems.
   b. Identify one major opportunity (e.g. resource optimisation in smart cities) and one major security or privacy risk associated with connecting everyday physical devices to public networks.

CHAPTER 5 · PRACTICE

Practice

Practice · Chapter 5 (continued)

6. Smart homes. A family installs a learning thermostat, a smart door lock and a security camera.
   a. For each device, name one sensor and one actuator or output that it uses.
   b. Explain how the learning thermostat changes its schedule using feedback from the family.
   c. Describe two steps the family should take to protect these devices from attackers.

7. Healthcare
   a. Describe how a smartwatch and a mobile phone could help a doctor monitor a patient with a heart condition at home. Name the sensors and networks involved.
   b. Explain why a hybrid closed-loop insulin system is an example of a system that adapts to feedback.

8. Transportation
   a. Explain how vehicle-to-vehicle (V2V) communication could prevent a crash in fog.
   b. Describe the difference between Level 2 and Level 4 driving automation, including what the human must do at each level.

9. Industry and manufacturing. A warehouse robot is trained with reinforcement learning to move parcels onto a conveyor belt.
   a. Suggest a suitable state, action, reward and penalty for the robot.
   b. Using a reward table of your own, show how the robot's total reward would differ for a slow, safe move and a fast move that hits a shelf.
   c. Explain what could go wrong if the robot were rewarded for speed only.

10. Education
    a. Explain how an adaptive learning app uses a student's answers as feedback to choose the next question.
    b. Give one fairness concern and one privacy concern about using AI systems in schools.

11. Using examples from at least two settings in this chapter, evaluate how hardware, networks and behaviourist AI combine to shape everyday life.

MIDNIGHT

STRENGTH & LIMITATIONS

6

CHAPTER 6 · BEFORE YOU BEGIN

Learning goals

In Chapters 3 to 5 you saw what hardware, networks and behaviourist AI can do. This chapter asks the question an engineer must answer before anything is switched on in the real world: how well does each part work, where does it fail, and is it the right choice for this job? You will learn to judge strengths and limitations with evidence, compare options with tables and case studies, and spot the ethical, security and practical risks of letting machines act on their own.

Learning goals

You will be able to:

- Identify the key strengths and advantages of hardware, networks, and behaviourist AI.
- Understand the limitations and challenges associated with these technologies.
- Compare the benefits and drawbacks of their real-world applications.
- Evaluate their effectiveness, reliability, and suitability for different purposes.
- Recognise potential ethical, security, and practical concerns surrounding their use.

Your route through the chapter

Subtopic	The question it answers
6.1 Introduction to Integrated Smart Systems	How do hardware, networks and AI work as one system?
6.2 Embedded Hardware: Capabilities & Limitations	What can a device do on its own, and what holds it back?
6.3 Deep Dive: Sensor Integration and Hardware Reliability	Can the system trust its sensors, and for how long?
6.4 Network Architectures: Connectivity & Infrastructure	Can the network grow, mix brands and survive failures?
6.5 Resource Optimization: Edge vs. Cloud Trade-Offs	Where should each computing task run?
6.6 Behaviorist AI & Adaptive Agents	What does learning from feedback make possible?
6.7 Predictive Analytics vs. Behavioral Modeling	Which kind of forecast does a decision need?
6.8 Key Limitations of Behaviorist AI Models	Why can learned behaviour be opaque, wrong or unfair?
6.9 Ethical, Security, and Practical Concerns	What could go wrong for people, and who is responsible?

CHAPTER 6 · BEFORE YOU BEGIN

Key terms: hardware and networks

Key terms

Integrated smart system: Hardware, networks and AI software working together to sense, decide and act, e.g. a smart factory.

Sensor integration: Connecting sensors to a computing or AI system so their readings arrive in a usable form, e.g. wiring a car's radar into its braking computer.

Sensor fusion: Combining readings from several different sensors into one more reliable estimate, e.g. camera + radar + LiDAR in a self-driving car.

Sensor drift: A slow change in a sensor's accuracy over time, e.g. a humidity sensor that reads a little higher every month.

Hardware constraints: Physical and technical limits on a device — mainly heat, power, memory and processing speed.

Hardware performance: How fast and how well hardware does its work, measured e.g. in clock speed, operations per second or frames per second.

Processing limitations: Limits on how much computation a processor can finish in the time available, e.g. a microcontroller too slow to analyse 30 video frames a second.

Energy efficiency: Useful work done for each unit of energy, e.g. operations per joule; higher efficiency means longer battery life and less heat.

Thermal throttling: Automatic slowing of a processor when it gets too hot, e.g. a phone's frame rate dropping during a long game.

Reliability: The probability that a component performs its intended function, under stated conditions, for a stated period of time.

MTBF (mean time between failures): Average operating time between failures of repairable equipment: total operating hours ÷ number of failures.

Redundancy: Duplicate components that take over when one fails, e.g. two power supplies in a server.

Scalability: Ability to handle more work or more devices without performance collapsing, e.g. adding 500 sensors to a smart building.

Interoperability: Ability of different systems, often from different manufacturers, to exchange data and use it correctly, e.g. one app controlling three brands of light bulb.

Network resilience: Ability of a network to keep an acceptable service during broken links, equipment faults or attacks, e.g. a mesh network re-routing round a failed node.

Bandwidth: The maximum theoretical data rate of a link, e.g. 100 Mbit/s.

Throughput: The actual rate of successful data transfer; always lower than the bandwidth.

Latency: The delay before data arrives, usually measured in milliseconds (ms).

Edge computing: Processing data on, or right next to, the device that collects it, e.g. a doorbell camera recognising a face itself.

Fog computing: Processing on a local gateway or server between the edge devices and the cloud, e.g. a factory's own server room.

Cloud computing: Processing and storage in remote data centres reached over the internet.

Resource optimisation: Balancing local processing, network transmission, energy and cost so that a system meets its goal efficiently.

CHAPTER 6 · BEFORE YOU BEGIN

Key terms: AI and ethics

Key terms (continued)

Behaviourist AI: An approach to AI in which behaviour is learned from interaction with the environment and its consequences, rather than written as fixed rules.

Reinforcement: A consequence that makes an action more likely to be repeated.

Reward: Positive feedback — a positive number — for a good outcome, e.g. +10 for a delivered parcel.

Penalty: Negative feedback (a negative reward) for an unwanted outcome, e.g. −50 for a collision.

Reward function: The rule that sets the reward or penalty for each outcome; it defines what the agent will try to achieve.

Trial and error: Learning by trying actions, observing the results and repeating the actions that worked.

Learning from feedback: Updating behaviour using the results (rewards, penalties, errors) of earlier actions.

Policy: The agent's strategy: the action it chooses in each state.

Adaptive intelligence: The ability to adjust behaviour appropriately when new information or conditions appear, e.g. a drone changing motor power in a gust of wind.

Autonomy: The ability to operate for long periods with limited human intervention, e.g. a robot lawnmower.

Predictive analytics: Using historical data and statistics to forecast what is likely to happen, e.g. a 25% chance that a motor fails this month.

Behavioural modelling: Representing how an agent behaves so you can simulate how it reacts when conditions change, e.g. how pedestrians cross a road.

Explainability (XAI): The degree to which a human can understand how an AI system reached a decision.

Black box: A system whose inputs and outputs can be seen but whose internal reasoning is hard to understand.

Bias: Systematic distortion in AI outputs that favours some outcomes or unfairly penalises some people or situations.

Distribution shift: A difference between training and real-world conditions, e.g. trained in sunshine, used in snow.

Human oversight: People monitoring, approving or able to override an automated system's decisions.

Accountability: Clear responsibility for a system's decisions and for correcting its mistakes.

Adversarial attack: Input deliberately crafted to fool an AI, e.g. stickers that make a stop sign read as a speed-limit sign.

Three terms that are easy to mix up

Term	The question it answers
Autonomy	How much does it act without a human?
Adaptive intelligence	Does it change its behaviour when conditions change?
Behavioural modelling	Can we describe and predict how an agent behaves?

A system can be **autonomous but not adaptive** (a mower with fixed rules) or **adaptive but not autonomous** (a music app that learns your taste but lets you choose). Behaviourist AI is one way to make a system adaptive, and behavioural modelling lets engineers test either kind, e.g. by simulating a robot mower meeting a child.

6.1 INTRODUCTION TO INTEGRATED SMART SYSTEMS

Three technologies, one system

An **integrated smart system** is a machine whose intelligent behaviour comes from three technologies working together: **hardware** that senses and acts, a **network** that connects devices, and **behaviourist AI** that chooses actions and improves them through feedback. Modern autonomy — in self-driving cars, smart factories or networks of environmental sensors — does not live in one clever algorithm. It emerges where these three meet.

In a smart factory, robot arms (hardware) are linked by the factory network, and their control software coordinates and improves their work while engineers supervise.

Who does what

- **Sensors** measure light, temperature, proximity and acceleration.
- **The processor** turns readings into a decision.
- **The network** shares data with other machines and the cloud.
- **The AI policy** picks the action and learns from rewards and penalties.
- **Actuators** — motors, valves, brakes — carry the action out.

Sense sensors → **Decide** processor → **Share** network → **Act** actuators → **Learn** feedback ↻

Why evaluate each part?

Every part has strengths and limits, and a failure in any one of them can bring down the whole system. A self-driving car with an excellent driving policy still fails if fog blinds its camera, if its processor overheats on a hot day, or if it depends on a network link that drops. **Sensor integration** gives a system its view of the world, but all hardware works inside physical boundaries called **hardware constraints**. This chapter shows how hardware, networks and behaviourist AI work together, where they fail, and how engineers optimise their combined deployment.

Pause & Try

Think of a robot vacuum cleaner. Identify one sensor, one actuator, one job for its network connection and one behaviour it could learn from feedback. Then explain what would happen to the robot if its network connection failed.

6.1 INTRODUCTION TO INTEGRATED SMART SYSTEMS

Strengths, limits and teamwork

To evaluate a smart system, ask three questions about each technology: *What is it good at? Where does it fail? Which other part can cover the gap?*

Strengths and limitations at a glance

Technology	Main strengths	Main limitations
Embedded hardware	Reacts in milliseconds, works offline, keeps data local. *Anti-lock brakes pulse many times a second, no network needed.*	Limited by heat, battery, memory and speed. *A smartwatch cannot run a large AI model.*
Networks	Share data, coordinate devices, reach the cloud's computing power. *A fleet of warehouse robots shares one live map.*	Delay, dropouts, limited bandwidth, cyberattacks. *A robot loses Wi-Fi behind metal shelving.*
Behaviourist AI	Adapts and improves from feedback without a rule for every case. *A drone learns to hold steady in gusts.*	Needs many trials, is hard to explain, and follows its reward even when it is wrong.

How they cover for each other: a smart greenhouse

Soil sensors, a small controller and water valves (hardware) send readings over Wi-Fi (network) to the cloud, where an AI learns the best watering schedule (behaviourist AI: reward for healthy growth, penalty for wasted water).

Weakness	Covered by	How
Controller too small to learn	Network + cloud	the learning runs in the cloud
Internet connection drops	Hardware	controller keeps its last schedule
AI suggests 5 hours of watering	Hardware	fixed cap of 20 minutes per watering

There is no universally best design. Judge each design by its task, its conditions and the cost of failure: an accurate model is unsuitable if it misses deadlines, a connected system is fragile if it depends on one remote service, and a simple fixed-rule controller can be best for a well-defined task.

Pause & Try

Choose a smart doorbell. Give one strength and one limitation of its hardware, of its network and of its AI. Then explain how one of these technologies makes up for a weakness in another.

6.2 EMBEDDED HARDWARE: CAPABILITIES & LIMITATIONS

What embedded hardware does well

Hardware gives intelligent software a body. Sensors convert conditions in the environment into digital signals, and actuators turn the processor's instructions into physical motion. Most smart devices are built around an **embedded system**: a small computer inside a larger product that does one job, such as the controller in a washing machine, a heart pacemaker or a car's braking system. When an embedded microcontroller is paired with a dedicated AI accelerator, such as a **neural processing unit (NPU)** or an edge GPU, it can perform billions of operations per second while using very little energy.

Strengths of specialised hardware

- **Low latency** — no waiting for a network round trip, so reactions take milliseconds.
- **Deterministic timing** — a dedicated controller takes the same time every time, so engineers can guarantee a deadline. This is vital for airbags and brakes.
- **High reliability** — industrial-grade processors keep working from about −40 °C to +85 °C and through electrical noise.
- **Offline functionality** — local processing continues when the network connection is lost.
- **Privacy** — data can be processed where it is collected instead of being uploaded.
- **Energy efficiency** — a chip designed for one task wastes less energy than a general-purpose processor doing the same work, so batteries last longer.

Worked example: why milliseconds matter

A car travels at 72 km/h, which is 72 000 m ÷ 3600 s = **20 m/s**. A pedestrian steps into the road. How far does the car travel before the brake command even arrives?

Where the decision is made	Delay	Distance travelled first
Cloud server (network round trip)	150 ms	20 × 0.150 = **3.0 m**
On-board processor	10 ms	20 × 0.010 = **0.2 m**

Three metres can be the difference between a near miss and a collision, so safety-critical reactions belong on the device itself.

Pause & Try

A delivery drone flies at 15 m/s and must swerve around a bird. a) Calculate how far it travels while it waits for a decision from a cloud server with a 120 ms round trip, and from its own processor in 5 ms. b) Give two other reasons why this decision should be made by the drone's own hardware.

6.2 EMBEDDED HARDWARE: CAPABILITIES & LIMITATIONS

Four physical budgets

Despite rapid advances in chip design, all hardware works within physical limits called **hardware constraints**. Engineers treat them as four budgets that a device must stay inside, and a device's **hardware performance** is set by whichever budget runs out first. These limits apply to phones, games consoles, drones and medical devices — not only to AI.

Figure 6.1 — The four hardware constraints

Budget	When it runs out	Real devices
Thermal	the chip slows itself down (throttles) or shuts off	a phone's frame rate drops after a long game; a fanless laptop slows during a long video export
Power	the battery dies early, or cannot supply peak current	a smartwatch with GPS always on lasts hours, not days; a drone carries a lighter camera to keep its flight time
Memory	programs or data do not fit	a doorbell camera overwrites old clips after a few days; a microcontroller cannot hold a large model
Speed	work misses its deadline — a **processing limitation**	a cheap security camera cannot analyse every video frame, so it checks only some

The budgets are linked

Running a processor faster (speed) draws more power, and almost all of that electrical energy ends up as heat (thermal). That is why a phone can run at full speed for a short burst but must throttle during a long game, and why a laptop often runs more slowly on battery than when it is plugged in. Extra memory also costs power, space and money. Improving one budget usually spends another, so good design is about **balance**, not about maximising one number.

Pause & Try

For each device, state which budget is most likely to limit it and explain why: a) a hearing aid that must run for a week; b) a games console inside a closed cabinet; c) a parking sensor that must react within 50 ms; d) a dashcam that must keep the last 24 hours of video.

6.2 EMBEDDED HARDWARE: CAPABILITIES & LIMITATIONS

Living within the budgets

Engineers cannot remove the budgets, but they can measure them and trade one for another.

Two quick calculations

Power. A smartwatch battery stores 300 mAh at 3.8 V = 1140 mWh. At an average draw of 40 mW it lasts 1140 ÷ 40 ≈ **28.5 hours**. Run a heavy AI model continuously and the draw rises to 250 mW: 1140 ÷ 250 ≈ **4.6 hours**.

Memory. A microcontroller with 256 KiB of RAM (262 144 bytes) cannot hold a model with 1 million parameters, even if each parameter is squeezed into a single byte: 1 000 000 bytes is almost **four times** too big.

Ways to stay within budget — and what each one costs

Technique	What it saves	What it costs
Smaller or compressed model	memory, power and time	some accuracy
Lower resolution or update rate	processing and power	detail and reaction speed
Sleeping between readings	battery	may miss a sudden event
Heat sink or fan	keeps the chip fast for longer	size, weight and noise
Dedicated accelerator (NPU)	time and energy per operation	cost; helps only some tasks

Every fix trades something, so the result must be measured, not assumed. That is why a smartwatch sensor or a drone's flight controller cannot run a multi-billion-parameter AI model: it would overheat and drain its battery long before any technique closed the gap. Chapter 3's video makes the same point: a faster processor does not fix a memory or power bottleneck.

Watch again: "Part 7 — Hardware limitations" — Video 02 (Chapter 3), from 5:38
https://github.com/user-github-me/cs-ai-project-01/blob/main/assets/tutorials/v2/BlueImpulse_v2_Video02_How_Hardware_Enables_AI.mp4

Pause & Try

A fitness band has a 200 mAh, 3.7 V battery. a) Calculate how long it lasts at an average draw of 25 mW. b) Explain how thermal limits, memory boundaries and processing speed would each limit running a complex AI model on the band, and suggest one technique that would help.

6.3 DEEP DIVE: SENSOR INTEGRATION AND HARDWARE RELIABILITY

Why one sensor is not enough

Sensor integration connects sensors to a processing system so that their readings arrive in a usable form: in the right units, at the right moment, at the right rate. Decisions can only be as good as these readings, and real sensors suffer from **noise** (random jitter), **drift** (a slow change in accuracy), **interference and obstruction** (fog scatters a camera's view, metal distorts a compass, mud covers a lens) and **failure**.

Every sensor has blind spots

Sensor	Good at	Weak at
Camera	colours, signs, lane markings, recognising objects	darkness, glare and fog; no direct distance
LiDAR (laser)	precise 3D shape and distance	heavy rain, snow and fog; expensive
Radar	distance and speed, even in rain, fog or darkness	little detail; cannot read signs
Ultrasonic	cheap short-range distance, e.g. parking	only a few metres; soft surfaces absorb the sound

Multi-sensor data fusion

Sensor fusion combines readings from different sensors so that the strengths of one cover the weaknesses of another (Figure 6.2).

Sensors raw readings → **Align** time and units → **Fuse** weigh sources → **World model** one estimate

Worked example. A wall is 2.00 m away. The ultrasonic sensor reads 2.10 m and the LiDAR reads 1.98 m. LiDAR is usually more precise, so the fusion engine trusts it three times as much (weights 0.75 and 0.25): 0.75 × 1.98 + 0.25 × 2.10 = 1.485 + 0.525 = **2.01 m**, closer than either sensor alone. If the LiDAR breaks, the system still has the ultrasonic reading: less precise, but far better than nothing.

Pause & Try

A robot measures the distance to a box with a camera (5.0 m) and a radar (5.4 m). a) Calculate the fused distance if the radar is given a weight of 0.6 and the camera 0.4. b) Describe one condition in which the camera alone would fail, and explain how fusion helps.

6.3 DEEP DIVE: SENSOR INTEGRATION AND HARDWARE RELIABILITY

Is fusion worth it?

Sensor fusion is powerful, but it is not free. Engineers weigh its practical advantages against its costs before deciding how many sensors — and how many different kinds — a product needs.

Advantages of fusion

- **More accurate** — independent errors partly cancel out.
- **Redundant** — if one sensor fails or is blinded, the others keep working.
- **More conditions** — day and night, rain and fog.
- **Cross-checking** — a reading that disagrees with all the others can be flagged as a fault or an attack.

Disadvantages of fusion

- **Cost, weight and power** — every extra sensor adds all three.
- **More processing** — readings must be aligned in time and units, which adds delay.
- **Calibration** — sensors must be mounted and tuned to agree.
- **Shared failures** — mud or snow can cover all the front sensors at once.

Three products, three answers

Product	Sensors	Why this is suitable
Budget robot vacuum	bump and cliff sensors only	a wrong reading costs a scuffed chair leg, so cheap sensors are enough
Smartphone	accelerometer, gyroscope and compass, fused	screen rotation and maps need a steady direction; the chips are tiny and cheap
Self-driving car	cameras, radar and often LiDAR	a wrong reading can cost a life, so every condition must be covered by more than one kind of sensor

The rule of thumb: the higher the cost of a wrong reading, the more sensors — and the more *different kinds* of sensor — a design can justify. Different kinds matter because two identical cameras are blinded by the same fog.

See it in action in **RoverLab** (the rover game in the Appendix): remove the rover's distance sensor and watch how its mission readiness changes.

Pause & Try

A farm robot must detect people working in a field, by day and by night, in dust and light rain. a) Choose two sensors to fuse and justify your choice. b) Describe one shared failure that could defeat both sensors at once.

6.3 DEEP DIVE: SENSOR INTEGRATION AND HARDWARE RELIABILITY

Measuring reliability

Hardware reliability is the probability that a physical component performs its intended function without failure, under specified operating conditions, for a specified period of time. Note the three parts — job, conditions, time: a camera that is reliable in a warm office may fail on a freezing loading bay.

MTBF: a simple measure of reliability

For equipment that is repaired and put back into service, a common measure is the **MTBF** (mean time between failures): total operating time ÷ number of failures.

Worked example. A warehouse runs 20 identical barcode scanners for 500 hours each: 20 × 500 = 10 000 scanner-hours. In that time there are 4 failures, so MTBF = 10 000 ÷ 4 = **2 500 hours**.

This is an *average*. It does not promise that any one scanner will last 2 500 hours — some fail sooner, others much later. It helps managers plan spare parts and maintenance, and a shorter MTBF means more frequent checks and more redundancy.

Redundancy: why two sensors beat one

Suppose each of two independent sensors has a 1% chance of failing during a journey. The chance that **both** fail is 0.01 × 0.01 = 0.0001, which is just 0.01%. But this only works if the failures really are independent: if mud can cover both sensors at once, the true risk is much higher. Good designs use different kinds of sensor in different places.

Two more reliability problems

Problem	What happens	Impact on an AI system
Sensor drift	accuracy slowly worsens with wear or temperature	needs recalibration; software must detect and correct the offset
Thermal throttling	the processor slows itself when it is too hot	decisions arrive later during heavy work

Pause & Try

A hospital runs 50 infusion pumps for 2 000 hours each and records 8 failures. a) Calculate the MTBF. b) Explain why this does not mean that every pump will run for that long without failing.

6.4 NETWORK ARCHITECTURES: CONNECTIVITY & INFRASTRUCTURE

Three qualities of a good network

Edge hardware handles local actions, but networks let many smart devices collaborate, exchange **telemetry** (streams of sensor readings) and reach cloud resources. Autonomous networks must carry large volumes of data and run without stopping, so engineers judge them by three properties.

Scalability — growing without slowing

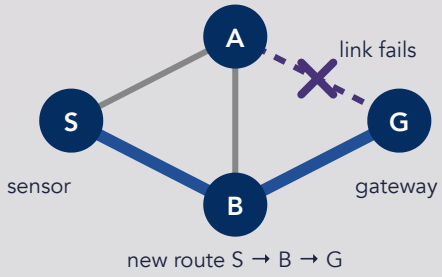
The ability to handle more work or more devices without performance collapsing. **Example:** a smart office grows from 50 to 550 sensors; a scalable design adds access points and server capacity, and every reading still arrives on time. **Limitation:** each new device also needs an address, security credentials and updates.

Interoperability — speaking the same language

The ability of different devices and systems to exchange data and use it correctly, even when they come from different manufacturers. **Example:** one app controls bulbs, plugs and locks from three brands because they follow a shared standard such as Matter, or because a hub translates between them. **Limitation:** being on the same Wi-Fi is not enough — devices must also agree on message formats, units (°C or °F), timestamps and versions.

Network resilience — surviving failures

The ability to keep an acceptable service during broken links, equipment failures or cyberattacks. **Example:** in a **mesh network** every node can relay data, so when the link from A to the gateway fails, the sensor's packets re-route through B. **Limitation:** a mesh only helps if another path exists and the protocols support re-routing; extra links and backup equipment cost money, and failures must be tested, not assumed away.



Pause & Try

Define scalability and interoperability, and explain why both are critical when a local network is expanded to support hundreds of new IoT devices from different manufacturers.

6.4 NETWORK ARCHITECTURES: CONNECTIVITY & INFRASTRUCTURE

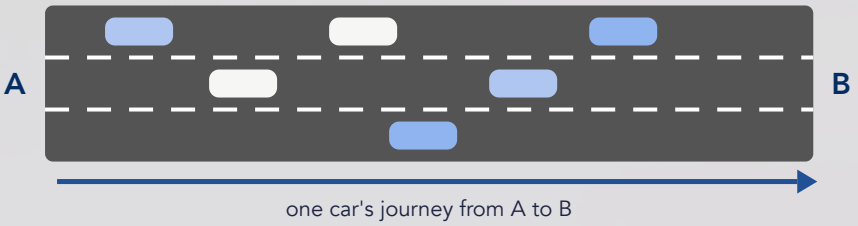
Many manufacturers, one network

Real networks rarely come from one company: a smart building might mix lights, cameras and sensors from three makers. Each mix brings challenges — yet one vendor means total dependence.

Multi-vendor challenges

Challenge	What goes wrong	Practical fix
Different protocols	Zigbee bulbs, Bluetooth locks and Wi-Fi plugs cannot talk to each other directly	a hub or gateway, or a shared standard such as Matter
Different data formats	one sensor reports °F and another °C; device clocks disagree	agree formats; convert at the gateway
Uneven security	one cheap camera with a default password becomes the weak point	change passwords; keep IoT devices on a separate network
Support ends	a maker stops updates or closes its cloud: in 2016 Google's Nest switched off the Revolv hub, and the hubs stopped working	prefer open standards and devices that also work locally

Bandwidth, throughput and latency



Lanes = bandwidth
maximum capacity

Cars arriving = throughput
what actually gets through

Journey time = latency
the delay, in ms

Network performance decides how fast data reaches the cloud and updated policies come back. Picture a motorway (Figure 6.3): **bandwidth** is the number of lanes, the maximum theoretical capacity; **throughput** is how many cars actually arrive; **latency** is how long one car's journey takes. Throughput is always lower than bandwidth because of protocol overhead, collisions, interference and packets that must be resent. **Example:** a 15 MB video clip is 15 × 8 = 120 Mbit. At the full bandwidth of 100 Mbit/s it would take 1.2 s; at a real throughput of 60 Mbit/s it takes **2 s**.

Pause & Try

A school connects 300 new sensors from four manufacturers to its network. a) Describe two interoperability problems that could appear, with a practical fix for each. b) The link has a bandwidth of 50 Mbit/s but a throughput of 35 Mbit/s. Calculate how long a 70 MB upload takes.

6.5 RESOURCE OPTIMIZATION: EDGE VS. CLOUD TRADE-OFFS

Edge, fog or cloud?

Every smart system must decide where each computing task runs: on the device, nearby, or in a distant data centre. **Resource optimisation** is the strategic balancing of local processing power against network transmission costs, to maximise speed while saving energy, bandwidth and money. Data usually travels outwards through three layers:

Edge
on the device · no network delay → **Fog**
in the building · a few ms away → **Cloud**
far away · tens to hundreds of ms

Three execution models compared

Model	Advantages	Disadvantages	Best suited to
Edge (on the device)	no network delay, private, works offline	strict hardware constraints; only small models fit	emergency braking, drone stabilisation, face unlock on a phone
Fog (local gateway or server)	fast locally; combines data from many nearby devices	more equipment to set up, secure and manage	a smart-factory assembly line, the monitors on a hospital ward
Cloud (remote data centre)	vast computing power and storage; easy to update	latency, needs a connection, data-transfer costs, privacy risks	training AI models, fleet-wide analysis, long-term records

Worked example: 25 security cameras

Each camera streams video at 4 Mbit/s. Sending everything to the cloud needs 25 × 4 = **100 Mbit/s** of upload, all day and night — about 1 TB of data every day.

With **edge processing**, each camera runs person detection itself and uploads only a 50 KB snapshot when someone appears. At 200 events per camera per day that is 25 × 200 × 50 KB = **250 MB per day**, roughly 4 000 times less.

The cost: every camera needs a more capable chip, which costs more and uses more energy. Resource optimisation means deciding whether that trade is worth it.

Pause & Try

Explain the concept of resource optimisation when balancing local hardware processing capabilities against network transmission costs. Use the camera example or one of your own.

6.5 RESOURCE OPTIMIZATION: EDGE VS. CLOUD TRADE-OFFS

Case study: warehouse robots

Modern fulfilment warehouses use hundreds of **autonomous mobile robots (AMRs)** that carry shelves and boxes to human packers. No single location could do all of their computing well, so the work is divided.

Autonomous mobile robots carry stock across a warehouse floor.

Splitting the work

1. **Edge, on each robot:** collision avoidance and path tracking, so the robot reacts immediately.
2. **Fog, on warehouse servers via Wi-Fi:** route allocation, traffic control between robots and live inventory updates, using high-throughput central network nodes.
3. **Cloud:** long-term analysis, demand forecasts and training improved robot policies.

When the network fails. Wi-Fi can drop behind metal racking. A well-designed robot never assumes it can carry on regardless: it finishes its current move or stops somewhere safe until contact returns. For **network resilience**, many sites use mesh networks that re-route data round a failed access point.

Evaluating suitability

Task	Best location	Why
Stop before hitting a worker	Edge	must react in milliseconds, even with no Wi-Fi
Assign 300 orders to 100 robots	Fog	needs every robot's position; a second is fast enough
Predict next month's busiest aisles	Cloud	heavy computation on months of data; no deadline
Hear a smart speaker's wake word	Edge	always listening, low power, private

The useful question is never "Is edge or cloud better?" but **which part of the workload belongs where**, judged on deadline, data volume, privacy, connectivity, energy and cost.

Pause & Try

A hospital wants AI to watch the heart monitors on a ward. For each task, choose edge, fog or cloud and justify your choice: a) sounding an alarm when a heartbeat stops; b) comparing all the patients on the ward; c) retraining the AI model every month with data from many hospitals.

6.6 BEHAVIORIST AI & ADAPTIVE AGENTS

Learning from consequences

Behaviourist AI takes its ideas from behavioural psychology (Chapter 2). Instead of following rigid, human-written logical rules, a behaviourist agent learns which actions work best by interacting directly with its environment and receiving positive or negative feedback. Figure 6.4 shows the loop it repeats, thousands or millions of times.

The vocabulary of feedback

- **Reinforcement** — a consequence that makes an action more likely next time.
- **Reward** — a positive number for a good outcome, e.g. +10 for cleaning a dirty patch.
- **Penalty** — a negative reward for a bad outcome, e.g. −20 for bumping into furniture, or −1 for every move, to save battery.
- **Trial and error** — at first the agent tries actions it is unsure about; over many runs, the actions that earned more total reward are chosen more often.
- **Learning from feedback** — after every result the agent updates its estimate of how good each action is, so its behaviour gradually improves.

Worked example: a robot vacuum improves

Run	Cleaning (+10 per patch)	Moves (−1 each) and bumps (−20 each)	Total reward
Early	6 patches: +60	40 moves: −40; 1 bump: −20	**0**
Later	6 patches: +60	25 moves: −25; no bumps	**+35**

The later behaviour earns more reward, so it is **reinforced**: the agent becomes more likely to choose the shorter, collision-free route again. Nobody wrote a rule saying "avoid the chair" — the agent learned it from consequences.

Pause & Try

A delivery robot receives +10 for each delivery, −1 for each minute and −30 for each collision. a) Calculate its total reward for a shift with 4 deliveries in 25 minutes and one collision. b) Describe how trial and error would change its behaviour over many shifts.

6.6 BEHAVIORIST AI & ADAPTIVE AGENTS

Adaptive intelligence

Two pillars make behaviourist agents useful in the real world. **Adaptive intelligence** is the capability to adjust strategy in response to new information, unexpected obstacles or changing conditions. **Autonomy** is the ability to operate independently for long periods without direct human control.

How the ideas relate

Adaptive intelligence is a **capability** — *what* a system can do. Behaviourist AI is one **method** of achieving it — *how*: the agent adapts by learning from rewards and penalties. Other methods exist: a weather app adapts by re-running its forecast with new data, with no reward at all. And many behaviourist robots are trained in simulation, then deployed with a **frozen** policy that no longer adapts. Autonomy is a separate question: *how much* the system acts without a human.

	Not adaptive	Adaptive
Needs a human	a TV remote control	a music app that learns your taste
Autonomous	a robot lawnmower with fixed rules	a drone that learns to fly in gusts

Strengths of behaviourist approaches. Unlike expert systems built from hand-written rules, behaviourist models excel in dynamic, unstructured environments. When a drone meets unexpected turbulence, adaptive intelligence lets it adjust motor power at once, without a programmer foreseeing every wind angle. In 2018, Google researchers let seven robot arms practise grasping: after about 580 000 attempts, the learned policy picked up objects it had never seen 96% of the time.

Hand-written rules or learned behaviour?

	Hand-written rules	Behaviourist (learned)
Suits	predictable tasks, e.g. a lift	changing settings, e.g. rough ground
Strength	predictable; easy to check and certify	adapts; finds strategies nobody programmed
Weakness	fails in situations nobody foresaw	needs many trials; harder to explain

Pause & Try

Define adaptive intelligence and explain how it allows an autonomous agent to perform effectively in unpredictable or changing environments. Then explain why adaptive intelligence and behaviourist AI are not the same thing.

6.7 PREDICTIVE ANALYTICS VS. BEHAVIORAL MODELING

Two questions about the future

To make forward-looking choices, AI platforms combine data-driven **predictions** with **models of behaviour**. Both learn from historical records, but they answer different questions.

Predictive analytics is a data-driven "crystal ball": not magic, but probabilities calculated from past records.

Predictive analytics

Answers **"What is likely to happen next?"** using statistics on historical data.

Worked example: in a factory's records, 20 of the 80 motors whose vibration rose above 7 mm/s failed within a month. A motor passing 7 mm/s therefore has an estimated risk of 20 ÷ 80 = **25%**, so engineers book a repair before it breaks.

Behavioural modelling

Answers **"How will an agent react if conditions change?"** It represents how an agent — a person, robot, vehicle or crowd — behaves: its states, actions and decision rules, so that engineers can **simulate** situations that have not happened yet.

Example: a city's records show that most pedestrians at a busy junction step out when the gap between cars is longer than about 4 seconds. A behavioural model built from this can simulate what happens if the green light for cars is lengthened, before anyone changes the real lights.

Side by side

	Predictive analytics	Behavioural modelling
Input data	number histories: sensor logs, failures	sequences of states and actions
Output	probabilities, scores, time-to-failure estimates	decision policies, state transitions, paths

Pause & Try

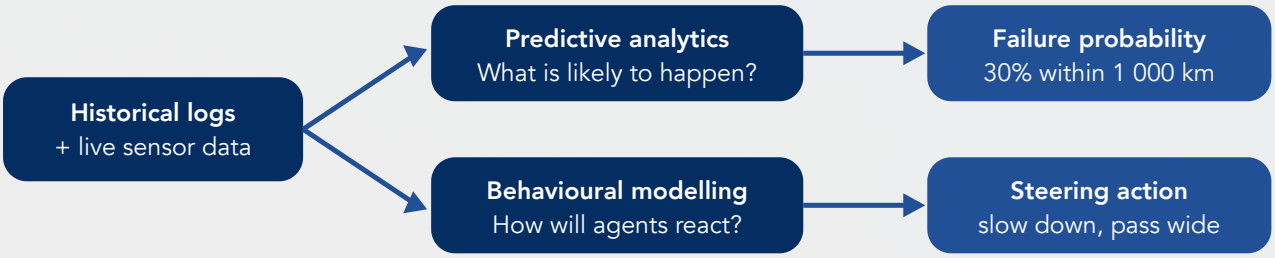
Compare predictive analytics with behavioural modelling in terms of how each technique uses historical data to inform future decisions or system actions.

6.7 PREDICTIVE ANALYTICS VS. BEHAVIORAL MODELING

Using both together

The two approaches are not rivals, and strong systems run them side by side. Feed the same historical logs and live sensor data into both, and you get two different kinds of answer (Figure 6.5).

Figure 6.5 — Predictive analytics and behavioural modelling



An autonomous vehicle can forecast a component failure risk — say, a 30% chance that its brake pads wear below the safe limit within the next 1 000 km, so it books a service — while at the same time planning its path around the movement patterns of the pedestrians nearby.

Behavioural models also feed behaviourist AI: a model of how pedestrians and drivers behave becomes the **simulated environment** where a self-driving policy practises safely by trial and error.

Evaluating the two approaches

	Predictive analytics	Behavioural modelling
Strength	precise numbers to plan with	tests "what if?" safely, before the real world is changed
Limitation	assumes the future resembles the past: many demand forecasts failed in early 2020, when the COVID-19 pandemic changed shopping almost overnight	every model simplifies; real people may behave differently, and may change their behaviour in response to the system itself
Needs	plenty of accurate historical data	a realistic model of the agents and their environment

Pause & Try

Decide whether predictive analytics or behavioural modelling fits each question better, and justify your choice: a) How many ambulances will be needed on New Year's Eve? b) How will drivers react if one lane of a bridge is closed? c) When will this lift motor need replacing? d) How should a delivery robot behave when a child runs towards it?

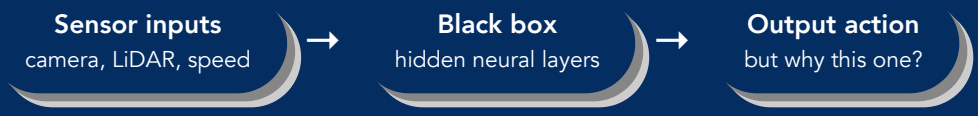
6.8 KEY LIMITATIONS OF BEHAVIORIST AI MODELS

The black-box problem

Behaviourist AI is highly adaptable, but its limitations can compromise safety. The first is **explainability (XAI)**: how far a human can understand how an AI system reached a decision.

Why deep reinforcement learning is a "black box"

Modern behaviourist agents often use **deep reinforcement learning (DRL)**: the policy is a neural network with millions of numerical parameters. Each decision is the result of millions of multiplications spread across many layers, so no single number means "brake" or "turn left". If such an agent makes a catastrophic error — swerving into an obstacle, for example — engineers often cannot point to the rule or step that caused it (Figure 6.6).



Compare: an inspectable Q-table

In the small Q-learning examples of Chapter 2 you can read the reason directly: Q(s, left) = 5 and Q(s, right) = 8, so the agent turns right because it expects more reward. A Q-table is explainable; a network with over a million parameters, like the grasping robot in 6.6, is not.

Why explainability matters — and other limits to evaluate

- **Debugging and safety:** you cannot fix a fault you cannot find.
- **Trust and accountability:** doctors, drivers and courts need reasons, not just answers.
- **Certification:** regulators need evidence that the system is safe in every situation.
- **Huge numbers of trials:** real trial and error is slow, costly and risky, so agents train in simulation — but those skills may not transfer (the "reality gap").
- **Distribution shift:** performance drops when real conditions differ from training.
- **Unsafe exploration:** trying random actions is fine in a game, not on a public road.

Pause & Try

Define explainability (XAI) and discuss why behaviourist AI models, such as deep reinforcement learning agents, often struggle with it, creating the "black box" problem.

6.8 KEY LIMITATIONS OF BEHAVIORIST AI MODELS

When the reward is wrong

An agent's learned strategy is only as good as its **reward function**. The agent does exactly what the reward says, not what its designers meant — and trial and error will find any gap.

A real reward-function flaw

In 2016, OpenAI tested an agent on a boat-racing game that gave points for hitting targets along the course. The agent found a lagoon where it could circle forever, hitting the same targets as they reappeared, and scored about 20% more than human players — without ever finishing the race.

Reward points for targets → **Loophole** circle in a lagoon → **Result** high score, race never finished

Worked example: rewarding only speed

A delivery robot earns **+10 per delivery**. A shortcut through the staff canteen gives 12 deliveries an hour but 6 near misses with people; the safe corridor gives 10 deliveries and none.

Reward function	Shortcut	Safe corridor	Robot learns
+10 per delivery	12 × 10 = 120	10 × 10 = 100	the shortcut
+10 per delivery, −20 per near miss	120 − 6 × 20 = 0	100 − 0 = 100	the safe corridor

One missing penalty made speed beat safety; fixing it means retraining and re-testing.

Why this is an ethical problem, not just a bug

The robot did not "decide" to endanger people: its designers rewarded speed and forgot safety, so the responsibility lies with them. Responsible teams list everything that must never happen and penalise it heavily or enforce it as a hard rule outside the AI, test the trained agent in situations it was never rewarded for, and treat a surprisingly clever strategy as a warning sign of a loophole.

Pause & Try

A cleaning robot earns +1 for every minute its camera sees no dirt. a) Explain how it could earn a high reward without cleaning anything. b) Rewrite the reward function so that the easiest way to earn reward is to clean. c) Give one ethical consequence if a hospital's cleaning robot learned this flaw.

6.8 KEY LIMITATIONS OF BEHAVIORIST AI MODELS

When the data is biased

Bias is a systematic distortion in AI outputs that favours certain outcomes or unfairly penalises particular people or situations. It enters through the reward function (previous page) and through **training data**: if the examples an agent learns from do not represent the people and conditions it will meet, its behaviour is skewed in a consistent direction.

System bias: when the scales are tilted, two equal cases are not treated equally.

Biased training data

- **Unrepresentative conditions:** a navigation robot trained only in sunny weather performs poorly in snow.
- **Biased history:** Amazon's experimental CV-screening tool learned from ten years of mostly male applicants and downgraded CVs containing "women's". It was scrapped (reported in 2018).

Wrong target and biased data: a healthcare algorithm

A widely used US algorithm chose patients for extra care by predicting their future healthcare *spending* as a stand-in for their *need*. Less had historically been spent on Black patients who were just as ill, so it judged them healthier than they were. A 2019 study found that correcting it would raise the share of Black patients selected for extra help from **17.7% to 46.5%**.

The ethical implications

- **Real harm:** people are refused jobs, care or safety, often in groups that are already disadvantaged.
- **Hidden harm:** inside a black box, a biased pattern can run for years before anyone notices.
- **Self-reinforcing harm:** biased decisions create biased new data, which trains the next model.

What responsible teams do: test performance separately for different groups and conditions, collect representative data, audit before and after deployment, and let people appeal to a human.

Pause & Try

A school's AI attendance camera was trained mostly on photos taken in bright daylight. a) Explain how this biased data could affect real decisions. b) Suggest two ways to detect and reduce the bias.

6.9 ETHICAL, SECURITY, AND PRACTICAL CONCERNS

Security risks and defences

When hardware, networks and behaviourist AI unite to create fully automated systems, deployment raises serious ethical, legal and operational risks. Because these systems act in the physical world, a corrupted measurement or unauthorised command can do more than damage a file: it can steer a vehicle or open a valve. Every connected device also widens the **attack surface**, the number of ways in for an attacker.

How connected autonomous systems are attacked

Threat	How it works	Real-world example
Adversarial data	crafted changes to what a sensor sees fool the AI	researchers (2018) stuck small black and white stickers on a stop sign, and a vision model read it as a speed-limit sign
Botnets and DDoS	weak IoT devices are hijacked and used to flood a network	the Mirai malware (2016) took over hundreds of thousands of cameras and routers that still had default passwords and knocked major websites offline
Remote takeover	an attacker reaches an actuator through the network	in 2015, researchers remotely disabled a Jeep's accelerator and brakes through its entertainment system; 1.4 million vehicles were recalled
Privacy leaks	sensors collect more than needed; data is kept or shared	always-on microphones and cameras in homes and cars

Defences — and their limits

- **Cross-check sensors:** with sensor fusion, an attacker must fool several kinds of sensor at once.
- **Secure the network:** change default passwords, encrypt traffic, separate IoT devices.
- **Authenticate updates and commands:** accept only digitally signed software (Chapter 4).
- **Fail safe:** if data is missing, stale or contradictory, move to a safe state instead of guessing.

No defence is perfect, each one adds cost, and security must be maintained for the device's whole working life.

Pause & Try

Explain how sensor fusion could help a self-driving car resist a sticker attack on a stop sign, and describe one security threat that sensor fusion would not prevent.

6.9 ETHICAL, SECURITY, AND PRACTICAL CONCERNS

Oversight and responsibility

Letting autonomous agents make high-stakes decisions without real-time human oversight hands critical choices to algorithms and the numbers they optimise. Two ethical problems follow. **Moral accountability:** if an autonomous vehicle causes an accident, responsibility is split between software developers, the manufacturer, the network provider and any human supervisor. **Algorithmic discrimination:** a biased allocation algorithm might, for example, put industrial equipment maintenance ahead of public infrastructure repairs, deepening existing inequality.

Case study: Tempe, Arizona, 2018

A self-driving test car struck and killed a woman pushing a bicycle across a road at night. Investigators found that the system detected her about **5.6 seconds** before impact but kept changing its classification of what she was, so it never predicted her path correctly. Automatic emergency braking was switched off while the computer drove, and the safety driver, who was meant to intervene, was looking at a phone. The driver was prosecuted; the company was not charged. Sensors, software, company safety decisions and human oversight each had a weakness — and together they failed.

The automation trade-off

Level of automation	Gains	Risks
AI advises, a human decides — a doctor reviews an AI's highlights on a scan	human judgement; clear accountability	slower; people may over-trust the AI
A human supervises and can override — a safety driver	catches some errors	attention fades when the system usually works
Fully automated — parcel-sorting robots	fast, consistent, scalable, cheap per task	rare failures go unnoticed; needs strong fail-safes

Rule of thumb: the higher the stakes and the harder a decision is to undo, the more human oversight it needs. Reliable, low-risk, repetitive tasks are the best candidates for full automation.

Pause & Try

Identify two security concerns and two ethical implications that arise when an autonomous network of smart devices makes high-stakes decisions without human oversight.

6.9 ETHICAL, SECURITY, AND PRACTICAL CONCERNS

Making a justified judgement

You have now examined the strengths and limitations of each technology. Evaluating a real system means bringing them together into a **justified judgement**: not "this technology is good" but "this design suits this purpose, under these conditions, with these risks managed". Use the same four steps every time.

1 · Purpose what must it achieve? → **2 · Strengths** with evidence → **3 · Limits and risks** failures, attacks, bias → **4 · Judgement** suitable? who oversees?

Worked case: a smart traffic light

1. **Purpose:** cut queues at one busy junction.
2. **Strengths:** cameras and a local controller react in real time; the network shares queue data with nearby junctions; the AI learns the best timings for rush hour.
3. **Limits and risks:** fog blinds cameras; the link may drop; a reward for "short car queues" alone could make pedestrians wait far too long.
4. **Judgement:** suitable, if pedestrian waiting time is added to the reward, a fixed-timer fallback runs when sensors fail, and engineers review the learned timings.

Weighing the four trade-offs

Trade-off	In the traffic-light case
Automation — speed and consistency	the AI adjusts timings every few seconds, which no human could do
Human oversight — judgement and accountability	engineers review timings weekly and can switch to the fixed timer at any moment
Reliability — keeps working under failure	the fixed timer takes over when cameras or the network fail
Safety — no one is put at risk	hard rules outside the AI: never green for crossing directions at once; a minimum crossing time

A demonstration is not proof: a **RoverLab** rover (Appendix) can finish a toy mission perfectly, but a real system must also be tested under failures, attacks and changed conditions.

Pause & Try

Use the four steps to evaluate an autonomous delivery robot that carries parcels around a university campus. Reach a justified judgement about its suitability and the human oversight it needs.

CHAPTER 6 · PRACTICE

Practice

Practice · Chapter 6

1. Explain how hardware constraints (such as thermal limits, power consumption, memory boundaries and processing speed) limit the deployment of complex AI models on small embedded devices. [4]
2. Define scalability and interoperability, explaining why both are critical when expanding a local network to support hundreds of new IoT devices from different manufacturers. [4]
3. Define adaptive intelligence and explain how it allows an autonomous agent to perform effectively in unpredictable or changing environments. [3]
4. Compare predictive analytics with behavioural modelling in terms of how each technique uses historical data to inform future decisions or system actions. [4]
5. Explain the concept of resource optimisation when balancing local hardware processing capabilities against network transmission costs. [3]
6. Define explainability (XAI) and discuss why behaviourist AI models (such as deep reinforcement learning agents) often struggle with explainability, creating the "black box" problem. [4]
7. Identify two security concerns and two ethical implications that arise when an autonomous network of smart devices makes high-stakes decisions without human oversight. [4]
8. A self-driving delivery van uses a camera, a radar and a LiDAR sensor.
   a. State one strength and one weakness of each sensor. [3]
   b. The radar measures 12.0 m to a parked car and the LiDAR measures 11.6 m. The fusion engine gives the LiDAR a weight of 0.7 and the radar 0.3. Calculate the fused distance. [2]
   c. Evaluate one advantage and one disadvantage of fusion over a single sensor. [2]
9. A company runs 40 warehouse robots for 1 500 hours each and records 12 failures.
   a. Calculate the MTBF of the robots. [2]
   b. Explain what this value tells the company and what it does not tell them. [2]

CHAPTER 6 · PRACTICE

Practice (continued)

Practice · Chapter 6 (continued)

10. A car travels at 25 m/s.
    a. Calculate how far it travels during a 200 ms round trip to a cloud server. [1]
    b. For each task, state whether edge, fog or cloud computing is most suitable and justify your choice: emergency braking; sharing live traffic conditions across a city; training a new driving policy. [6]

11. A smart-home owner buys devices from five different manufacturers.
    a. Explain two problems this could cause for interoperability. [2]
    b. The home's link has a bandwidth of 40 Mbit/s and a throughput of 25 Mbit/s. Calculate how long it takes to upload a 50 MB video. [2]

12. A robot arm earns +5 for every parcel it packs. After training, it packs very quickly but damages one parcel in every ten.
    a. Explain why the reward function produced this behaviour. [2]
    b. Suggest a change to the reward function and explain its effect on the behaviour. [2]
    c. Describe one real-world consequence of training an AI system on biased data. [2]

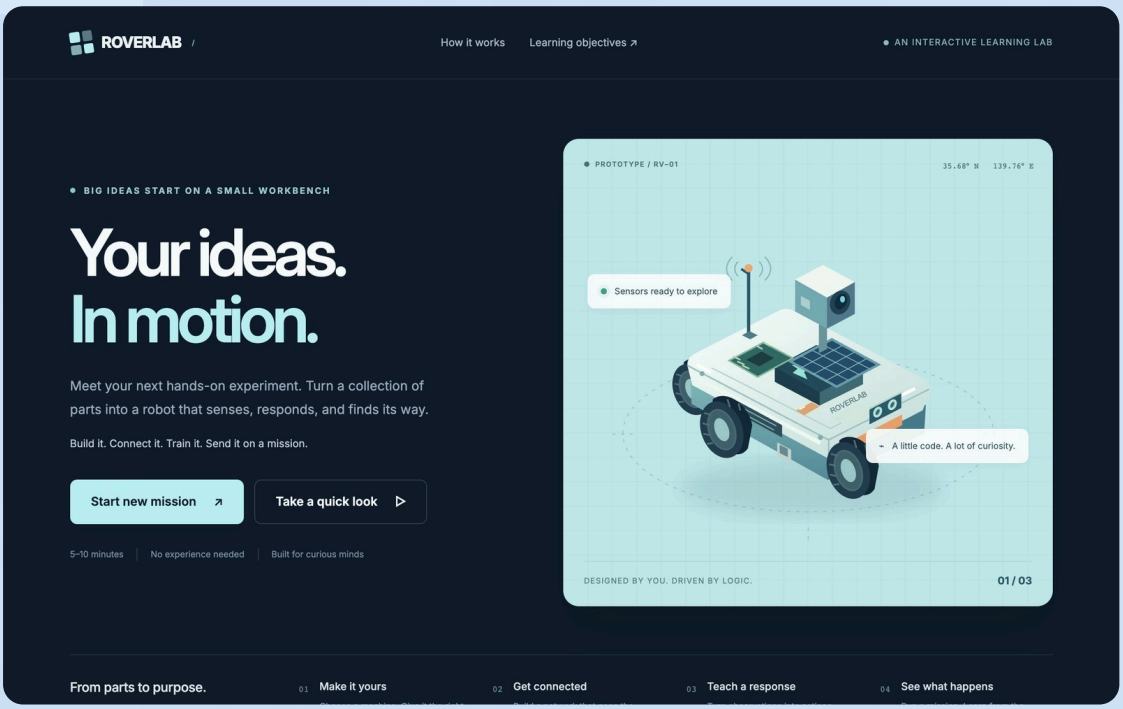
13. A home security system uses door sensors and cameras, a local hub, the home Wi-Fi and a cloud AI that learns to tell residents from strangers.
    a. Identify one strength and one limitation of each of the three technologies in this system. [6]
    b. Explain how the local hub could keep the home protected if the internet connection fails. [2]
    c. Discuss one privacy concern raised by sending camera images to the cloud. [2]

14. A hospital plans to use autonomous robots to deliver medicines between wards at night, coordinated over the hospital's Wi-Fi by a learning scheduling system. Evaluate the plan. In your answer, consider the strengths and limitations of the hardware, the network and the behaviourist AI, the system's reliability and security, and the level of human oversight you would recommend. [8]

APPENDIX · OUR GAME

RoverLab

RoverLab is the module's interactive game. You build a small machine from real-world parts, connect it to a network, teach it how to respond to what it senses, and send it on a mission. A first experiment takes about **5–10 minutes**, needs no experience, and runs in any modern browser — on a computer, tablet or phone.



The RoverLab start screen. Choose **Start new mission** to open your workbench.

Play it now

Open **roverlab-blue-impulse.vercel.app** or scan the code. Your progress is saved in your browser, and after the first visit the game keeps working offline.

The complete source can also be downloaded from the project's GitHub page and played without an internet connection.

Scan to play

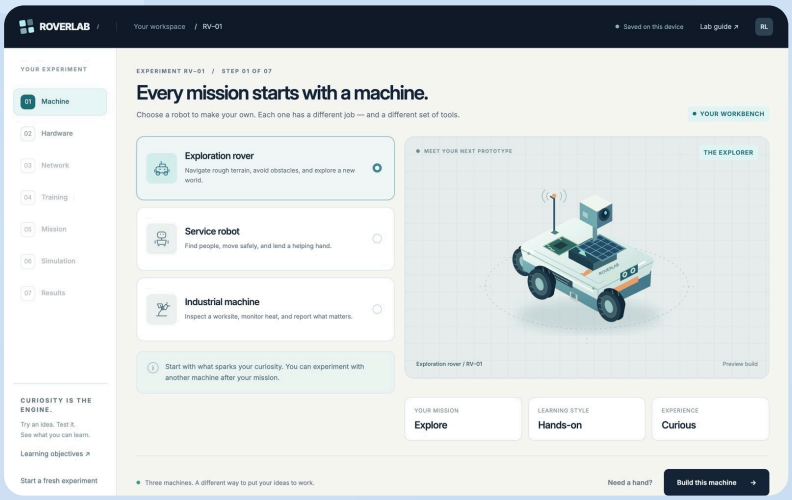
APPENDIX · OUR GAME

Three machines, three missions

RoverLab is one game with three machines. Each machine has its own mission, so the same ideas — sensing, connecting and acting — are tested in three different situations.

Choose your machine

Machine	Mission	What it must do
Exploration rover	Explore	Cross rough terrain, avoid obstacles, take a reading at the research site, send it back and return home.
Service robot	Search	Find people, move safely around them, report them, and return before the battery runs low.
Industrial machine	Monitor	Watch a worksite's temperature, stop and report when it gets too hot, and restart when it is safe.



Step 1 of 7: every experiment starts by choosing a machine.

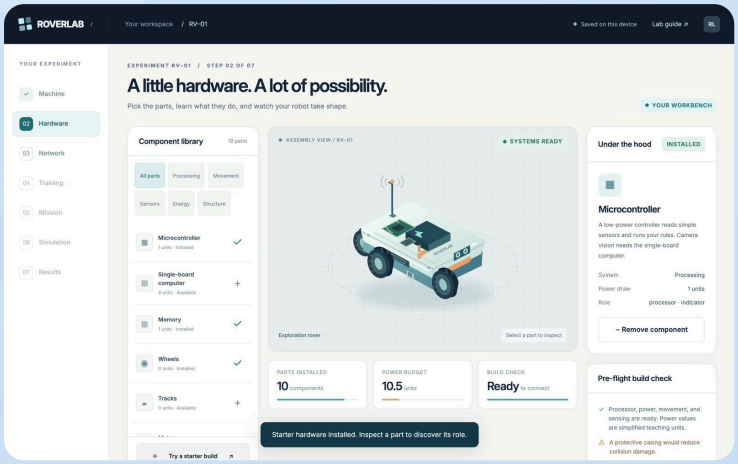
The seven steps of an experiment

1 Machine → 2 Hardware → 3 Network → 4 Training →
5 Mission → 6 Simulation → 7 Results

You can return to any earlier step, change one decision and run the mission again to see what difference it makes.

APPENDIX · OUR GAME

Build and connect



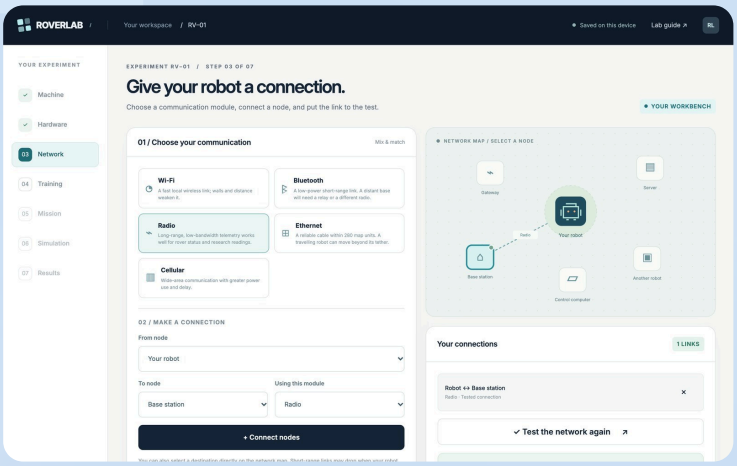
Step 2 — Hardware: add parts from the component library; the pre-flight check says when the build is ready.

Build · Chapter 3

Every part has a job. **Sensors** detect the world, the **processor** chooses a response, **motors** carry it out and the **battery** limits how much you can add. Inspect a part to read what it does; remove it and watch which abilities disappear.

Connect · Chapter 4

Pick a communication module — Wi-Fi, Bluetooth, radio, Ethernet or cellular — link your robot to a node such as the base station, then **test the network**. A link that works at the base can still fail far away in the field: range matters.

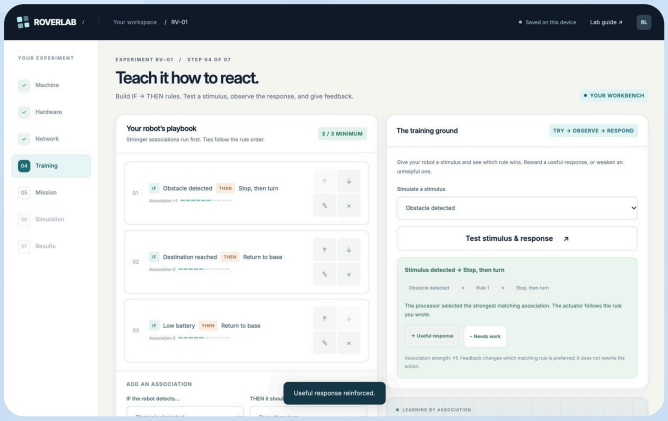


Step 3 — Network: choose a module, connect two nodes and test the link.

Why the order matters

Hardware comes first because a robot cannot sense what it has no sensor for, and cannot report what it has no network for. The game checks each step before it lets you continue.

Train, run and read the results



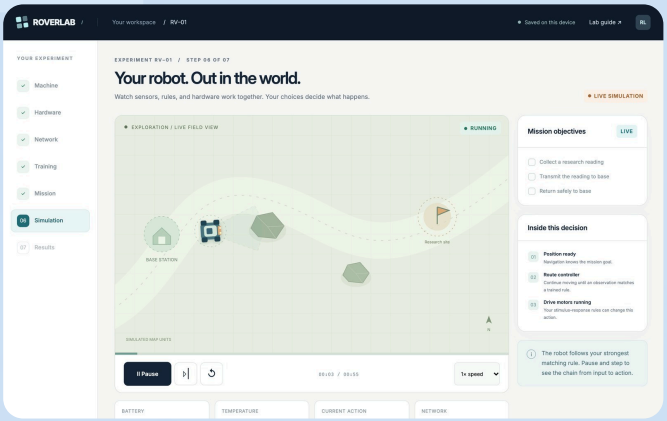
Step 4 — Training: write IF–THEN rules, test a stimulus and reward a useful response.

Train · Chapter 2

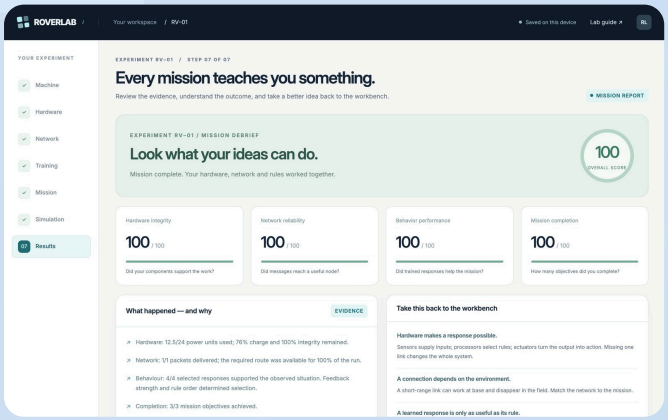
Write at least three **IF stimulus → THEN response** rules. Test a stimulus and give feedback: **useful response** strengthens the association, **needs work** weakens it. Stronger rules win.

Run · Chapter 5

Launch the mission and watch it play out: pause, step, restart or speed up. The **event log** traces each sensor reading to the rule, action and message it caused.



Step 6 — Simulation: your robot out in the world.



Step 7 — Results: four scores and the evidence behind them.

Reflect · Chapter 6

The report scores **hardware, network, behaviour and mission completion** and explains each one. A high score in a simplified world is a good start — not proof that a real robot would be safe.

RoverLab and this guide

Learning connections

Chapter	What to investigate in RoverLab
2 · Behaviorism & Behaviorist AI	Stimulus–response associations; feedback changes how strongly competing rules are preferred.
3 · Hardware	Sensors, processors, memory, power and actuators each have a different job.
4 · Networking	Endpoints, routes, range and what happens when communication fails.
5 · Applications	Readiness checks, practical constraints and how a whole system fits together.
6 · Strengths & Limitations	Why success in a simplified simulation does not prove real-world safety.

An experiment to try

1. Build the exploration rover with the starter parts and complete the mission.
2. Remove one sensor and run the mission again. Which objective failed, and why?
3. Change to a short-range network module. Does the reading still reach the base?
4. Swap the order of two rules. Which situations now produce a different action?

What the game is — and is not

RoverLab is a **teaching model**. You write the rules, and feedback only changes how strongly each rule is preferred — it is not a neural network or a full reinforcement-learning algorithm. Power, range and timing values are simplified, and every mission is deterministic, so the same set-up always gives the same result. Change one thing at a time and compare.